

TEC—4 计算机组成原理实验系统

教师实验指导书

清华大学科教仪器厂

2004 年 11 月

目 录

前言.....	2
TEC—4 计算机组成原理实验系统.....	3

基本实验

运算器组成实验.....	13
双端口存储器原理实验.....	18
数据通路组成实验.....	22
微程序控制器组成实验.....	28
CPU 组成和机器指令执行实验.....	40
中断原理实验.....	45

课程设计

硬布线控制器的设计与调试.....	50
流水微程序控制器的设计与调试.....	61
流水硬布线控制器的设计与调试.....	70

附录.....	78
---------	----

前 言

计算机组成原理是大学计算机专业的一门专业基础课程。清华同方股份有限公司和北京邮电大学白中英教授合作，开发出 TEC—4 计算机组成原理实验系统。在这个实验设备上，既能够做计算机组成原理课程的实验，又能够进行计算机系统结构课程的流水实验。该实验设备计算机模型简单、实用，运算器数据通路、控制器、控制台各部分划分清晰，为学生提供了很多的动手条件，有利于提高学生学习的主动性和创造性。

为了更好地使用 TEC—4 计算机组成原理实验系统，我们编写了这本《教师用实验指导书》。它是 TEC—4 计算机组成原理实验系统的配套资料。本书提供了 9 个实验，其中 6 个基本实验，3 个课程设计。对每个实验提供了接线图、实验步骤及实验结果；对 3 个课程设计提供了参考方案。当然，每个实验的接线方法或者源程序绝不是唯一的，实验结果也会有所不同。本书提供的接线图和源程序只是许多接线方法、源程序中的一种，只能作为参考。这些接线图、源程序和实验结果都经过实际实验验证。本书的目的是希望减少教师设计、准备、调试实验的劳动，让教师把主要精力集中在提高教学质量上。各位老师在使用中如果发现错误和不妥之处，欢迎提出宝贵意见，以便将来改进。

除了本书外，TEC—4 计算机组成原理实验系统还有另一本配套资料《计算机组成原理题解、题库和实验》，这是一本科学出版社出版的学生用书。该书由北京邮电大学的白中英教授和我们共同编写，可作为 TEC—4 计算机组成原理实验系统的使用说明书。两本资料配合使用，效果更好。

敬请读者提出宝贵意见。

清华同方股份有限公司
教学仪器设备公司
2001 年 1 月

第一节 TEC—4 计算机组成原理实验系统

TEC—4 计算机组成原理实验系统由北京邮电大学计算机学院、清华同方教学仪器设备公司、深圳拓普威电子技术有限公司联合研制。它是一个 8 位计算机模型实验系统，可用于大专、本科、硕士研究生计算机组成原理课程、计算机系统结构课程的教学实验，对提高学生的动手能力、提高学生对计算机整体和各组成部分的理解、提高学生的计算机系统综合设计能力都会有很大帮助。

一、TEC—4 计算机组成原理实验系统特点

1. 计算机模型简单、实用，运算器数据通路、控制器、控制台各部分划分清晰。
2. 计算机模型采用了数据总线和指令总线双总线体制，能够实现流水控制。
3. 控制器有微程序控制器或者硬布线控制器两种类型，每种类型又有流水和非流水两种方案。
4. 寄存器堆由 1 片 ispLSI1016 组成，运算器由 1 片 ispLSI1024 组成，设计新颖。
5. 实验台上包括了 1 片在系统编程芯片 ispLSI1032，学生可用它实现硬布线控制器。
6. 该系统能做运算器组成、双端口存储器、数据通路、微程序控制器、中断、CPU 组成与机器指令执行、流水微程序控制器、硬布线控制器、流水硬布线控制器等多种实验。
7. 电源部分采用模块电源，重量轻，具有抗电源对地短路能力。
8. 采用自锁紧累接接线方式，接线可靠。

二、TEC—4 计算机组成原理实验系统的组成

TEC—4 计算机组成原理实验系统由下述六部分组成：

1. 控制台
2. 数据通路
3. 控制器
4. 用户自选器件试验区
5. 时序电路
6. 电源部分

下面分别对各组成部分予以介绍。

三、电源

电源部分由一个模块电源、一个电源插座、一个电源开关和一个红色指示灯组成。电源模块通过四个螺栓安装在实验台下面。它输出+5V 电压，最大负载电流 3 安培，内置自恢复保险功能，具有抗+5V 对地短路能力。电源插座用于接交流 220 伏市电，插座内装有保险丝。电源开关用于接通或者断开交流 220 伏市电。当电源模块输出+5V 时，点亮+5V 红色指示灯。

四、时序发生器

时序发生器产生计算机模型所需的时序。时序电路由一个 1MHz 晶体振荡器、2 片 GAL22V10 (U6 和 U7) 组成，位于控制存储器的右边。根据本机设计，执行一条微指令需要 4 个时钟周期 T1、T2、T3、T4，执行一条指令通常需要取指、送操作数、运算、写结果四个节拍，因此本机的基本时序如下：

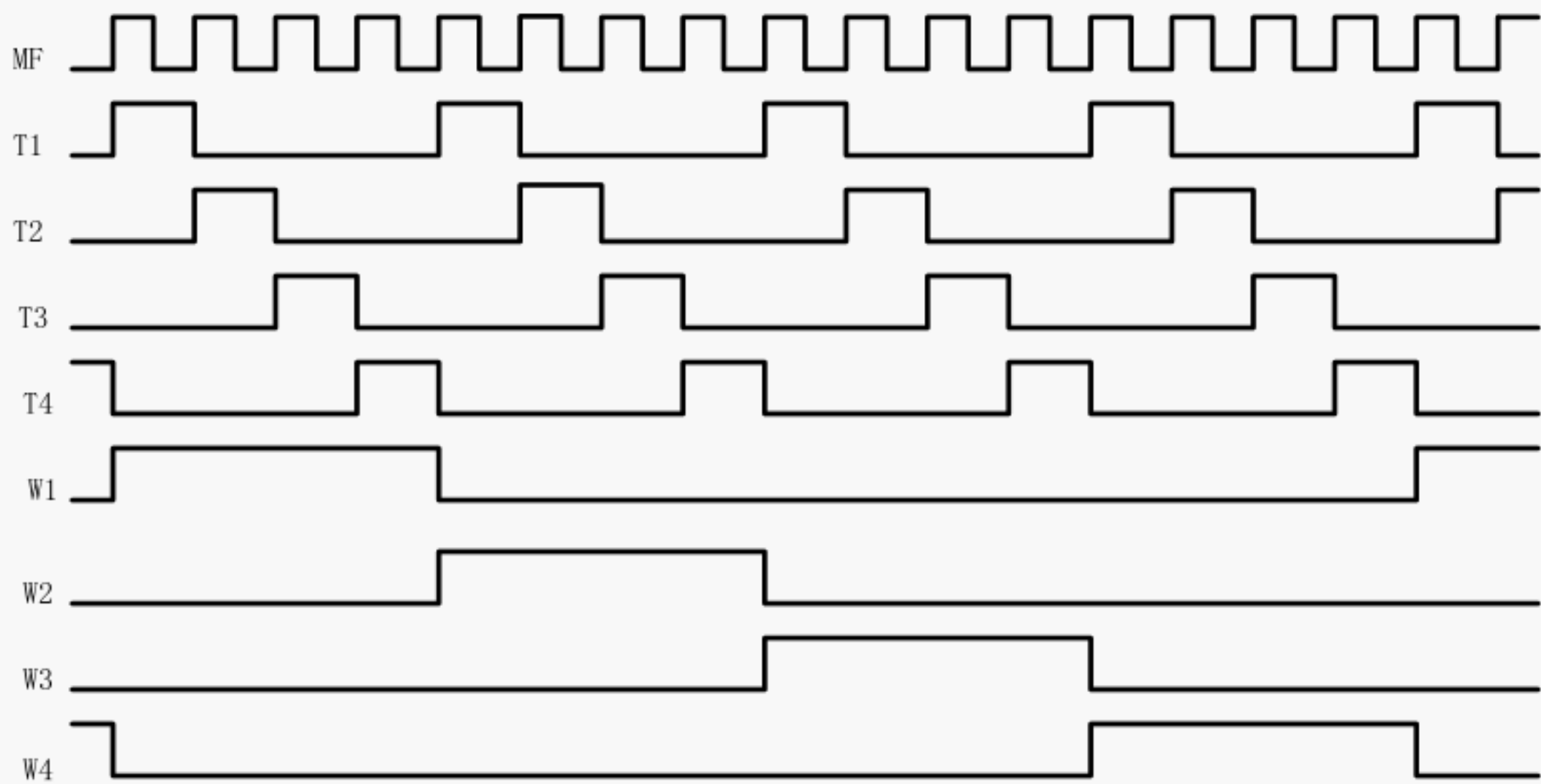


图3 基本时序图

图中，MF 是晶体振荡器产生的 1MHz 基本时钟，T1、T2、T3、T4 是数据通路和控制器中各寄存器的时钟脉冲，印制板上已将它们和有关的寄存器连接。T1、T2、T3、T4 既供微程序控制器时使用，也供硬布线控制器使用。W1、W2、W3、W4 只供硬布线控制器作指令节拍信号使用。

五、数据通路

数据通路的设计是 TEC—4 计算机组成原理实验系统最有特色的部分。首先它采用了数据总线和指令总线双总线形式，使得流水实验能够实现。它还使用了大规模在系统可编程器件作为运算器和寄存器堆，使得设计简单明了，可修改性强。数据通路位于实验系统的中部。

图 4 是数据通路总体图，下面介绍图中个主要部件的作用。

1. 运算器 ALU

运算器 ALU 由一片 ispLSI1024 (U47) 组成，在选择端 S2、S1、S0 控制下，对数据 A 和 B 进行加、减、与、直通、乘五种运算，功能如下：

表 1 运算器功能表

选 择			操 作
S2	S1	S0	
0	0	0	A & B
0	0	1	A & A (直通)
0	1	0	A + B
0	1	1	A - B
1	0	0	A(低 4 位) X B(低 4 位)

进位 C 只在加法运算和减法运算时产生。加运算中，C 表示进位；减运算中，C 代表借位。加、减运算产生的进位（借位）在 T4 的上升沿送入 C 寄存器保存。与、乘、直通操作

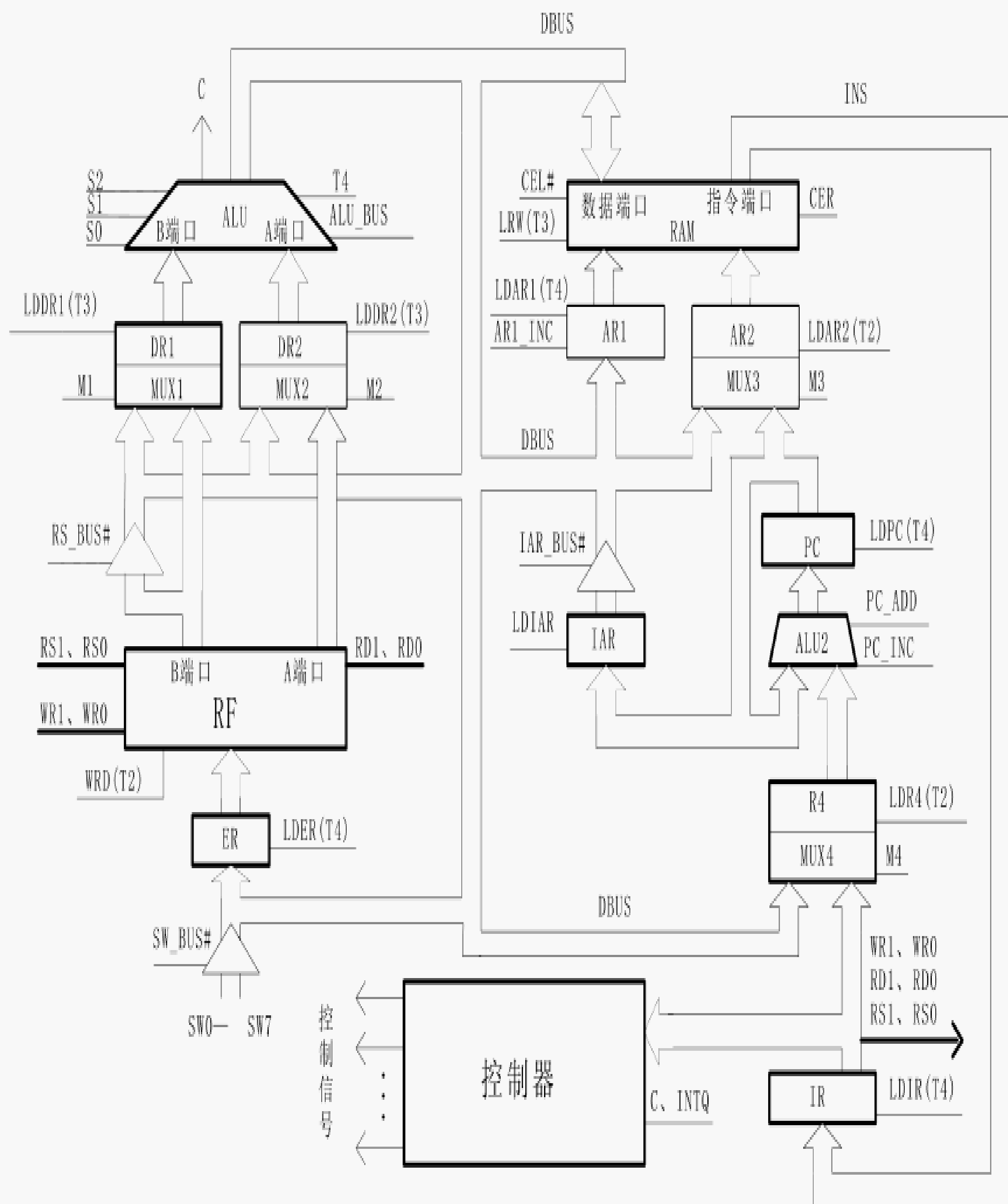


图4 数据通路总体图

不影响进位 C 的状态，即进位 C 保持不变。

当 $ALU_BUS = 1$ 时，运算结果送往数据总线 DBUS。加、减运算产生的进位（借位）C 与控制台的 C 指示灯相连。

2. DR1 和 DR2

DR1 和 DR2 是运算操作数寄存器，DR1 和 ALU 的 B 数据口相连，DR2 和 ALU 的 A 数据口相连。DR1 和 DR2 各由 2 片 74HC298 (U23、U24、U21、U22) 组成。U23 是 DR1 的低 4 位，U24 是 DR1 的高 4 位；U21 是 DR2 的低 4 位，U22 是 DR2 的高 4 位。当 $M1=0$ 且 $LDDR1=1$ 时，在 T3 的下降沿，DR1 接收来自寄存器堆 B 端口的数据；当 $M1=1$ 且 $LDDR1=1$ 时，在 T3 的下降沿，DR1 接收来自数据总线 D_BUS 的数据。当 $M2=0$ 且 $LDDR2=1$ 时，在 T3 的下降沿，DR2 接收来自寄存器堆 A 端口的数据；当 $M2=1$ 且 $LDDR2=1$ 时，在 T3 的下降沿，DR2 接收来自数据总线 DBUS 的数据。

3. 多端口通用寄存器堆 RF

多端口通用寄存器堆 RF 由 1 片 ispLSI1016 (U32) 组成，它的功能和 MC14580 类似。寄存器堆中包含 4 个 8 位寄存器(R0、R1、R2、R3)，有三个控制端口。其中两个端口控制读操作，一个端口控制写操作，三个端口可同时操作。RD1、RD0 选择从 A 端口读出的寄存器，RS1、RS0 选择从 B 端口读出的寄存器，WR1、WR0 选择被写入的寄存器。WRD 控制写操作。当 $WRD = 0$ 时，禁止写操作；当 $WRD = 1$ 时，在 T2 的上升沿将来自 ER 寄存器的数据写入由 WR1、WR0 选中的寄存器。

A 端口的数据直接送往操作数寄存器 DR2，B 端口的数据直接送往操作数寄存器 DR1。除此之外，B 端口的数据还通过 1 片 74HC244 (U15) 送往数据总线 DBUS。当 $RS_BUS\# = 0$ 时，允许 B 端口的数据送到数据总线 DBUS 上；当 $RS_BUS\# = 1$ 时，禁止 B 端口的数据送到数据总线 DBUS。

4. 暂存寄存器 ER

暂存寄存器 ER (U14) 是 1 片 74HC374，主要用于暂时保存运算器的运算结果。当 $LDER = 1$ 时，在 T4 的上升沿，将数据总线 DBUS 上的数据打入暂存寄存器 ER。ER 的输出送往多端口通用寄存器堆 RF，作为写入数据使用。

5. 开关寄存器 SW_BUS

开关寄存器 SW_BUS(U38)是 1 片 74HC244，用于将控制台开关 SW7—SW0 的数据送往数据总线 DBUS。当 $SW_BUS\# = 1$ 时，禁止开关 SW7—SW0 的数据送往数据总线 DBUS；当 $SW_BUS\# = 0$ 时，允许开关 SW7—SW0 的数据送往数据总线 DBUS。

6. 双端口存储器 RAM

双端口存储器由一片 IDT7132 (U36) 及少量附加控制电路组成。IDT7132 是 2048 字节的双端口静态随机存储器，本机实际使用 256 字节。IDT7132 两个端口可同时进行读、写操作。在本机中，左端口的数据连接数据总线 DBUS，可进行读、写操作，右端口数据和指令总线 INS 连接，输出到指令寄存器 IR，作为只读端口使用。存储器 IDT7132 有 6 个控制引脚：CEL#、LRW、OEL#、CER#、RRW、OER#。CEL#、LRW、OEL#控制左端口读、写操作，CER#、RRW、OER#控制右端口读、写操作。CEL#为左端口选择引脚，低有效，为高时禁止左端口操作；LRW 为高时，左端口进行读操作，LRW 为低时，左端口进行写操作；OER#为低时，将左端口读出的数据放到数据总线 DBUS 上。CER#、RRW、OER#控

制右端口读、写操作的方式与 CEL#、LRW、OER#控制左端口读、写操作的方式类似，不过右端口读出的数据放到指令总线上而不是数据总线上。本机设计中，OER#已固定接地，RRW 固定接高电平，CER#由 CER 反相产生。当 CER=1 时，右端口读出数据，并放到指令总线 INS 上；当 CER=0 时，禁止右端口操作。左端口的 OEL#由 LRW 经反相产生，不需单独控制。当 CEL#=0 且 LRW=1 时，左端口进行读操作；当 CER#=0 且 LRW=0 时，在 T3 的上升沿开始进行写操作，将数据总线 DBUS 上的数据写入存储器。

7. 地址寄存器 AR1 和 AR2

地址寄存器 AR1 (U37) 和 AR2 (U27、U28) 提供双端口存储器的地址。AR1 是 1 片 GAL22V10，具有加 1 功能，提供双端口存储器左端口的地址。AR1 从数据总线 DBUS 接收数据。AR1 的控制信号是 LDAR1 和 AR1_INC。当 AR1_INC = 1 时，在 T4 的上升沿，AR1 的值加 1；当 LDAR1 = 1 时，在 T4 的上升沿，将数据总线 DBUS 的数据打入地址寄存器 AR1。AR2 由 2 片 74HC298 组成，有两个数据输入端，一个来自程序计数器 PC，另一个来自数据总线 DBUS。AR2 的控制信号是 LDAR2 和 M3。M3 选择数据来源，当 M3 = 1 时，选中数据总线 DBUS；当 M3 = 0 时，选中程序计数器 PC。LDAR2 控制何时接收地址，当 LDAR2 = 1 时，在 T2 的下降沿将选中的数据源上的数据打入 AR2。

8. 程序计数器 PC、地址加法器 ALU2、地址缓存器 R4

程序计数器 PC、地址加法器 ALU2、地址缓存器 R4 联合完成三种操作：PC 加载，PC+1，PC+D。R4 是一个由 2 片 74HC298 (U25、U26) 构成的具有存储功能的两路选择器。当 M4 = 1 时，选中数据总线 DBUS；当 M4 = 0，从指令寄存器 IR 的低 4 位 IR0—IR3 接收数据。当 LDR4 = 1 时，在 T2 的下降沿将选中的数据打入 R4。ALU2 由 1 片 GAL22V10 (U17) 构成，当 PC_ADD = 1 时，完成 PC 和 IR 低 4 位的相加，即 PC 加 D。程序计数器 PC 是 1 片 GAL22V10 (U18)，当 PC_INC = 1 时，完成 PC+1；当 PC_ADD = 1 时，与 ALU2 一起完成 PC+D 的功能；当 LDPC=1 时，接收从 ALU2 和 R4 来的地址，实际是接收来自数据总线 DBUS 的地址，这些新的程序地址在 T4 的上升沿打入 PC 寄存器。

9. 指令寄存器 IR

指令寄存器 IR 是一片 74HC374 (U20)。它的数据端从双端口存储器接收数据（指令）。当 LDIR = 1 时，在 T4 的上升沿将来自双端口存储器的指令打入指令寄存器 IR 保存。指令的操作码部分送往控制器译码，产生各种所需的控制信号。大多数情况下，指令的操作数部分应连到寄存器堆（用户自己连接），选择参与运算的寄存器。在某些情况下，指令的操作数部分也参与新的 PC 的计算。

本实验系统设计了 12 条基本的机器指令，均为单字长（8 位）指令。指令功能及格式如表 2 所示。表 2 中的 X 代表随意值，RS1、RS0 指的是寄存器堆的 B 端口选择信号 RS1、RS0，RD1、RD0 指的是寄存器堆的 A 端口选择信号 RD1、RD0，不过由于运算结果需写回，因此它也同时指 WR1、WR0，用户需将它们对应连接。另一点需说明的是，为了简化运算，指令 JC D 中的 D 是一个 4 位的正数，用 D3 D2 D1 D0 表示。

实验系统虽仅设计了 12 条基本的机器指令，但代表了计算机中常用的指令类型。必要时用户可扩充到 16 条指令或者重新设计指令系统。

10. 中断地址寄存器 IAR

中断地址寄存器 IAR (U19) 是一片 74HC374，用于保存中断发生时的断点地址。它直接使用 LDIAR 信号作为时钟脉冲。当 IAR_BUS# = 0 时，它将断点地址送到数据总线 DBUS 上，

D7	D6	D5	D4	D3	D2	D1	D0
操 作 码				源操作数		目标操作数	

表 2 机器指令格式

名称	助记符	功能	指令格式							
			R7	R6	R5	R4	R3	R2	R1	R0
加法	ADD Rd, Rs	Rd+Rs→Rd	0	0	0	0	RS1	RS0	RD1	RD0
减法	SUB Rd, Rs	Rd-Rs→Rd	0	0	0	1	RS1	RS0	RD1	RD0
乘法	MUL Rd, Rs	Rd*Rs→Rd	0	0	1	0	RS1	RS0	RD1	RD0
逻辑与	AND Rd, Rs	Rd&Rs→Rd	0	0	1	1	RS1	RS0	RD1	RD0
存数	STA Rd, [Rs]	Rd→[Rs]	0	1	0	0	RS1	RS0	RD1	RD0
取数	LDA Rd, [Rs]	[Rs]→Rd	0	1	0	1	RS1	RS0	RD1	RD0
无条件转移	JMP [Rs]	[Rs]→PC	1	0	0	0	RS1	RS0	X	X
条件转移	JC D	若 C=1 则 PC+D→PC	1	0	0	1	D3	D2	D1	D0
停机	STP	暂停运行	0	1	1	0	X	X	X	X
中断返回	IRET	返回断点	1	0	1	0	X	X	X	X
开中断	INTS	允许中断	1	0	1	1	X	X	X	X
关中断	INTC	禁止中断	1	1	0	0	X	X	X	X

以使用控制台上的数据指示灯观察断点地址。

以上介绍了数据通路的基本组成。数据通路所需的各控制信号，除了 T1、T2、T3、T4 已在印制板上连接好以外，其余的控制信号在数据通路的下方都有插孔引出，实验时只要将它们和控制器产生的对应信号正确连接即可。实验中提供的电路图上，凡引出、引入线端带有短粗黑标记的信号，都是需要用户自己连接的信号。

六、控制器

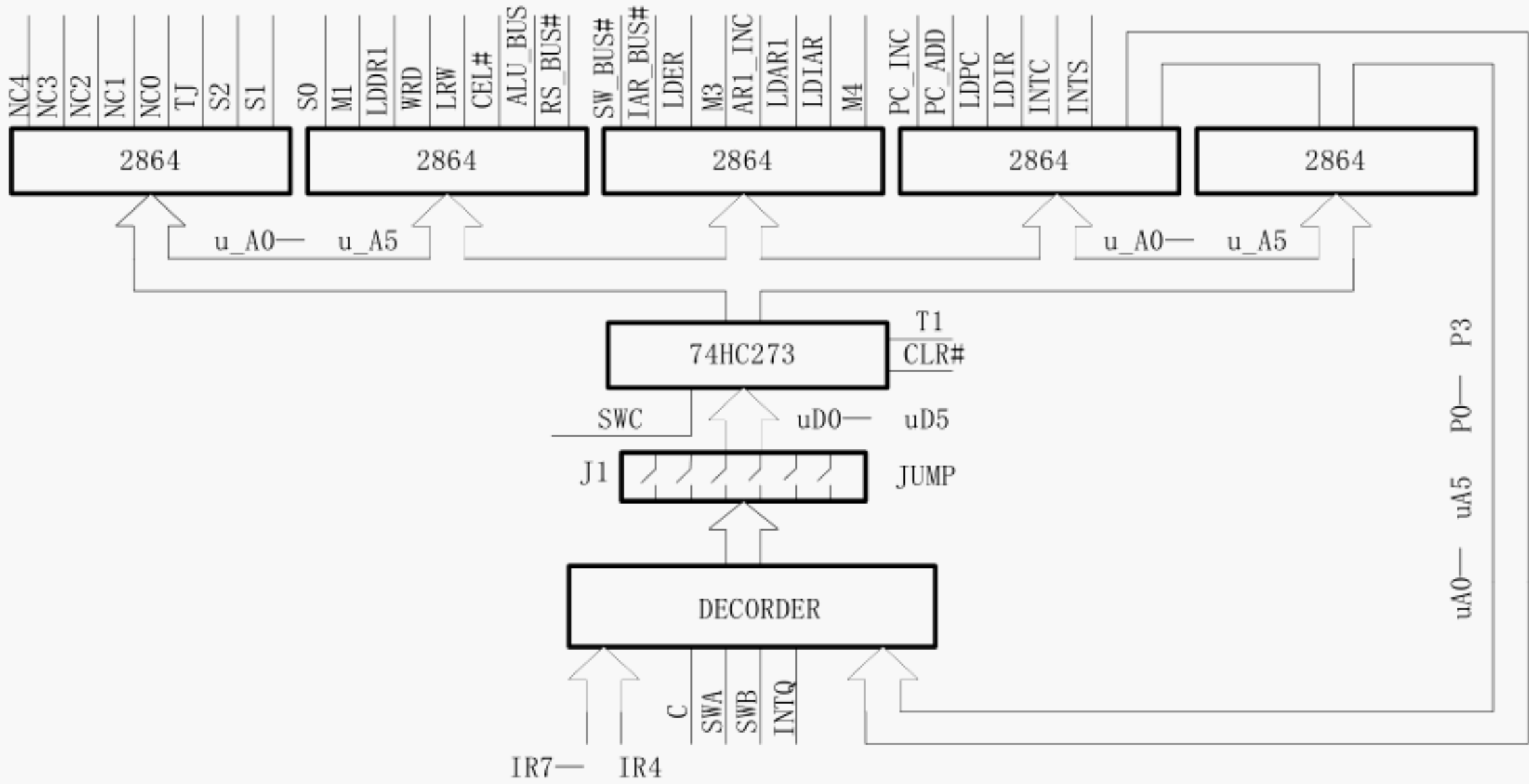


图5 控制器框图

控制器位于本实验系统的中上部，产生数据通路操作所需的控制信号。出厂时，提供了一个微程序控制器，使用户能够进行基本的计算机组成原理实验。在进行流水微程序控制器实验，硬布线控制器实验和流水硬布线控制器实验等课程设计时，用户可设计自己的控制器，部分或者全部代替出厂时提供的控制器。图 5 是控制器的框图。

1. 控制存储器

控制存储器由 5 片 28C64 (U8、U9、U10、U11、U12) 组成。28C64 是电擦除的可编程 ROM，存储容量为 8K 字节，本实验系统仅使用了 128 字节。微指令格式采用全水平型，微指令字长 35 位。其中顺序控制部分 10 位：后继微地址 $\mu A0-\mu A5$ ，判别标志 P0、P1、P2、P3；操作控制字段 25 位，全部采用直接表示法，用于控制数据通路的操作。

标志位 P3 和控制台开关 SWB、SWA 结合在一起确定微程序的分支，完成不同的控制台操作。标志位 P2 与指令操作码 (IR 的高 4 位 IR4、IR5、IR6、IR7) 结合确定微程序的分支，转向各种指令的不同微程序流程。标志位 P1 标志一条指令的结束，与中断请求信号 INTQ 结合，实现对程序的中断处理。标志位 P0 与进位标志 C 结合确定微程序的分支，实现条件转移指令。

操作控制字段 25 位，全部采用直接表示法，控制数据通路的操作。在设计过程中，根据微程序流程图对控制信号行了适当的综合与归并，把某些在微程序流程图中作用相同或者类似的信号归并为一个信号。下面列出微程序控制器提供的控制信号。信号名带后缀#者为低电平有效，否则为高电平有效。

INTS	置中断允许标志 INTE 为 1。
INTC	清除中断允许标志 INTE。
LDIR (CER)	为 1 时, 允许对 IR 加载, 此信号也可用于作为双端口存储器右端口选择 CER。
LDPC (LDR4)	为 1 时, 允许对程序计数器 PC 加载, 此信号也可用于作为 R4 的加载允许信号 LDR4。
PC_ADD	为 1 时, 进行 PC+D 操作。
PC_INC	为 1 时, 进行 PC+1 操作。
M4	当 M4 = 1 时, R4 从数据总线 DBUS 接收数据; 当 M4=0 时, R4 从指令寄存器 IR 接收数据。
LDIAR	为 1 时, 对中断地址寄存器 IAR 加载。
LDAR1 (LDAR2)	为 1 时, 允许对地址寄存器 AR1 加载, 此信号也可用于作为允许对地址寄存器 AR2 加载。
AR1_INC	为 1 时, 允许进行 AR1+1 操作。
M3	当 M3 = 1 时, AR2 从数据总线 DBUS 接收数据; 当 M3 = 0 时, AR2 从程序计数器 PC 接收数据。
LDER	为 1 时, 允许对暂存寄存器 ER 加载。
IAR_BUS#	低有效, 为 0 时将中断地址寄存器 IAR 送数据总线 DBUS。
SW_BUS#	低有效, 为 0 时将控制台开关 SW7—SW0 送数据总线 DBUS。
RS_BUS#	低有效, 为 0 时将寄存器堆 RF 的 B 端口送数据总线 DBUS。
ALU_BUS	为 1 时, 将 ALU 中的运算结果送数据总线 DBUS。
CEL#	低有效, 为 0 时允许双端口存储器左端口进行读、写操作。
LRW	当 LRW = 1 且 CEL# = 0 时, 双端口存储器左端口进行读操作; 当 LRW = 0 且 CEL# = 0 时, 双端口存储器左端口进行写操作。

WRD	为 1 时，允许对寄存器堆 RF 进行写操作。
LDDR1 (LDDR2)	为 1 时允许对操作数寄存器 DR1 加载。此信号也可用于作为对操作数寄存器 DR2 加载。
M1 (M2)	当 M1 = 1 时，操作数寄存器 DR1 从数据总线 DBUS 接收数据；当 M1 = 0 时，操作数寄存器 DR1 从寄存器堆 RF 接收数据。此信号也可用于作为操作数寄存器 DR2 的数据来源选择信号。
S2、S1、S0	选择运算器 ALU 的运算类型。
TJ	暂停微程序运行。
NC0、NC1、NC2 、NC3、NC4	备用

上述控制信号连同时序电路提供的时序、控制信号位于控制器的下边。

2. 微地址寄存器 μ AR (74HC273)

微地址寄存器 μ AR (74HC273) 对控制存储器提供微程序地址。当 CLR# = 0 时，将其复位到零，使微程序从 000000B 地址开始执行。在 T1 的上升沿将新的微程序地址 μ D0— μ D5 打入微地址寄存器 μ AR。控制台开关 SWC 直接连到 74HC273，作为 μ D6，用于实现读寄存器操作 KRR。

3. 跳转开关 JUMP

这是一组 6 个跳线开关 (J1)。当用短路子将它们连通时，微地址寄存器 μ AR 从本实验系统提供的微程序地址译码电路得到新的微程序地址 μ D0— μ D5。当他们被断开时，用户提供自己的新微程序地址 μ D0— μ D5。这样用户能够使用自己设计的微程序地址译码电路。

4. 微程序地址译码电路 DECORDER

微程序地址译码电路 DECORDER 产生后继微程序地址，它由 2 片 74HC32 (U2、U3) 和 2 片 74HC08 (U4、U5) 构成。微程序地址译码电路数据来源是：控制存储器产生的后继微程序地址 μ A0— μ A5，控制存储器产生的标志位 P0—P3，指令操作码 IR4—IR7，进位标志 C，中断请求标志 INTQ，控制台方式标志位 SWA、SWB。

七、控制台

控制台位于 TEC—4 计算机组成原理实验系统的下部，主要由若干指示灯和若干拨动开关组成，用于给数据通路置数、设置控制信号、显示各种数据使用。

1. SW7—SW0

数据开关，直接接到数据通路部分的数据总线 DBUS 上，用于向数据通路中的器件置数。开关拨到上面位置时输出 1，拨到下面位置时输出 0。SW7 是最高位，SW0 是最低位。

2. K15—K0

双位拨动开关。开关拨到上面位置时输出 1，拨到下面位置时输出 0。实验中用于模拟数据通路部分所需的电平控制信号。例如，将 K0 与 LDDR1 连接，则 K0 向上时，表示置 LDDR1 为 1；K0 向下时，表示置 LDDR1 为 0。

3. 数据指示灯 D7—D0

8 个红色发光二极管，用于显示数据总线 DBUS 或者指令寄存器 IR 的状态。D7 是最高位，D0 是最低位。双位开关 IR/DBUS 拨到 IR 位置时，显示指令寄存器 IR 的状态；双位开关 IR/DBUS 拨到 DBUS 位置时，显示数据总线 DBUS 状态。

4. 地址指示灯 A7—A0

8 个绿色发光二极管，用于显示双端口存储器的地址寄存器内容。A7 是最高位，A0 是最低位。双端口存储器 IDT7132 有两个地址端口，地址寄存器 AR1 提供左端口地址 A7L—A0L，地址寄存器 AR2 提供右端口地址 A7R—A0R。当双位开关 AR2/AR1 拨到 AR1 位置时，显示地址寄存器 AR1 的内容；当双位开关 AR2/AR1 拨到 AR2 位置时，显示地址寄存器 AR2 的内容。

5. 微地址指示灯 μ _A5— μ _A0

6 个黄色发光二极管，用于显示控制存储器的地址 μ _A5— μ _A0。 μ _A5 是最高位， μ _A0 是最低位。

6. 其他指示灯 P3、P2、P1、P0、IE、C

6 个黄色发光二极管用于显示 P3、P2、P1、P0、IE、C 的值。P3、P2、P1、P0 是控存的微代码位，用于条件分支产生下一个微地址。C 是加、减运算时产生的进位值。IE 是中断允许标志。当 IE = 1 时，允许中断；当 IE = 0 时，禁止中断。

7. 微动开关 CLR#、QD、INTR

这三个微动开关用于产生 CLR#、QD、INTR 单脉冲。按一次按钮 CLR#，产生一个负的单脉冲 CLR#，对全机进行复位，使全机处于初始状态，微程序地址置为 000000B。CLR#到时序和控制器的连接已在印制板上实现，控制存储器和数据通路部分不使用复位信号 CLR#。按一次 QD 按钮，产生一个正的 QD 启动脉冲。QD 和时序部分的连接已在印制板上实现。按一次 INTR 按钮，产生一个正的单脉冲，可用于作为中断请求信号。INTR 到时序部分的连接已在印制板上实现。这三个单脉冲都有插孔对外输出，供用户设计自己的控制器和时序电路时使用。

8. 单步、单拍、单指开关 DB、DP、DZ

DB（单步）、DP（单拍）、DZ（单指）是三种特殊的非连续工作方式。当 DP = 1 时，计算机处于单拍方式，按一次 QD 按钮，每次只执行一条微指令，发送一组 T1、T2、T3、T4 时序脉冲。当 DZ = 1 时，计算机处于单指方式。单指方式只对微程序控制器适用。在单指方式下，按一次 QD 按钮，计算机执行一条指令。当 DB = 1 时，机器处于单步方式。单步方式只对硬布线控制器适用。在单步方式下，按一次启动按钮 QD，发送一组 W1、W2、W3、W4 时序脉冲。在使用硬布线控制器时，每条指令需要一组 W1、W2、W3、W4 时序脉冲，因此单步方式实际上是硬布线控制器下的单指方式。DB、DP、DZ 这三个双位开关，任何时刻都只允许一个开关置 1，决不允许两个或三个开关同时置 1。当 DB=0 且 DP=0 且 DZ=0 时，机器处于连续工作方式。

9. 控制台方式开关 SWC、SWB、SWA

控制台方式开关 SWC、SWB、SWA 定义了 TEC—4 计算机组成原理实验系统的五种工作方式。在出厂时提供的标准控存中，五种工作方式定义如下：

SWC	SWB	SWA	工作方式
0	0	0	PR，启动程序
0	0	1	KRD，读双端口存储器

0	1	0	KWE, 写双端口存储器
0	1	1	KLD, 加载寄存器堆
1	0	0	KRR, 读寄存器堆

在按 CLR#按钮复位后, 根据 SWC、SWB、SWA 选择工作方式。

PR 是启动程序方式。在此方式下, 首先在 SW7—SW0 指定启动地址, 按启动按钮 QD 后, 启动程序运行。

KRD 是读双端口存储器方式。在此方式下, (1) 首先在 SW7—SW0 置好存储器地址; 按 QD 按钮, 则将此地址打入地址寄存器 AR1, 并读出该地址存储器内容到数据总线 DBUS。(2) 每按一次 QD 按钮, 地址寄存器 AR1 加 1, 并读出新地址存储器内容到数据总线 DBUS。依次进行下去, 直到按复位按钮 CLR#为止。

KWE 是写双端口存储器方式。在此方式下, (1) 首先在 SW7—SW0 置好存储器地址; 按 QD 按钮, 则首先将此地址打入地址寄存器 AR1, 然后等待输入数据。(2) 在 SW7—SW0 置好数据, 按 QD 按钮, 首先写数据到 AR1 指定的存储器单元, 然后地址寄存器 AR1 加 1, 等待新的输入数据。依次进行下去, 直到按复位按钮 CLR#为止。

KLD 是加载寄存器堆方式。此方式用于对寄存器堆加载。(1) 首先在 SW7—SW0 置好存储器地址, 按 QD 按钮, 则将此地址打入地址寄存器 AR1 和地址寄存器 AR2。(2) 在 SW7—SW0 置好数据, 数据的低 2 位 D1、D0 为寄存器堆中的寄存器号, 按一次 QD 按钮, 则写数据到 AR1 指定的存储器单元; 然后将写入的数据从右端口读出, 并送入指令寄存器 IR。(3) 在 SW7—SW0 置好数据, 该数据为写入寄存器的数据, 寄存器号由 IR 低 2 位指定。按 QD 按钮, 则首先将此数据写入寄存器 ER, 然后将 ER 中的数据写入指定的寄存器。(4) 返回 (2), 依次进行下去, 直到按复位按钮 CLR#为止。

KRR 是读寄存器堆方式。此方式用于读寄存器堆中的寄存器。(1) 首先在 SW7—SW0 置好存储器地址, 按 QD 按钮, 则将此地址打入地址寄存器 AR1 和地址寄存器 AR2。(2) 在 SW7—SW0 置好数据, 数据的 D3、D2 位为寄存器堆中的寄存器号, 按一次 QD 按钮, 则写数据到 AR1 指定的存储器单元; 然后将写入的数据从右端口读出, 并送入指令寄存器 IR。同时将 IR3、IR2 指定的寄存器送往数据总线 DBUS。拨动开关 IR/DBUS 可看到 IR 的值和 IR 指定的寄存器的值。(3) 返回 (2), 依次进行下去, 直到按复位按钮 CLR#为止。

八、用户自选器件试验区

本计算机组成原理实验系统提供了一个用户自选器件试验区, 供流水微程序控制器实验、硬布线控制器实验、流水硬布线控制器实验使用。自选器件试验区包括了 1 个 ispLSI1032 器件及下载插座, 把 PC 机和下载插座用出厂时提供的下载电缆相连, 在 PC 机上运行 ispEXPERT 软件, 即可对 ispLSI1032 器件编程和下载。利用 ispLSI1032 器件, 可满足这三个实验中应用的逻辑电路需要。另外, 为了增加灵活性和性, 用户自选器件试验区还提供了 10 个双列直插插座, 其中包括 2 个 24 引脚插座, 3 个 20 引脚插座, 2 个 16 引脚插座, 3 个 14 引脚插座。

除此之外, TEC—4 计算机组成原理实验系统中还提供了 3 个接地点, 供用示波器和万用表测试时使用。

第二节 运算器组成实验

一、实验目的

- (1) 掌握算术逻辑运算加、减、乘、与的工作原理。
- (2) 熟悉简单运算器的数据传送通路。
- (3) 验证实验台运算器的 8 位加、减、与、直通功能。
- (4) 验证实验台的 4 位乘 4 位功能。
- (5) 按给定数据，完成几种指定的算术和逻辑运算。

二、实验电路

图 6 示出了本实验所用的运算器数据通路图。ALU 由 1 片 ispLSI1024 构成。四片 4 位的二选一输入寄存器 74HC298 构成两个操作数寄存器 DR1 和 DR2，保存参与运算的数据。DR1 接 ALU 的 B 数据输入端口，DR2 接 ALU 的 A 数据输入端口，ALU 的输出在 ispLSI1024 内通过三态门发送到数据总线 DBUS7—DBUS0 上，进位信号 C 保存在 ispLSI1024 内的一个 D 寄存器中。当实验台下部的 IR/DBUS 开关拨到 DBUS 位置时，8 个红色发光二极管指示灯接在数据总线 DBUS 上，可显示运算结果或输入数据。另有一个指示灯 C 显示运算器进位信号状态。由 ispLSI1024 构成的 8 位运算器的运算类型由选择端 S2、S1、S0 选择，功能如表 3 所示：

表 3 运算器运算类型选择表

选 择			操 作
S2	S1	S0	
0	0	0	A & B
0	0	1	A & A (直通)
0	1	0	A + B
0	1	1	A - B
1	0	0	A(低 4 位) X B(低 4 位)

进位 C 只在加法运算和减法运算时产生，与、乘、直通操作不影响进位 C 的状态，即进位 C 保持不变。减法运算采用加减数的反码再加以 1 实现。在加法运算中，C 代表进位；在减法运算中，C 代表借位。运算产生的进位在 T4 的上升沿送入 ispLSI1024 内的 C 寄存器保存。

在 SW_BUS#信号为 0 时，参与运算的数据通过一个三态门 74HC244 (SW_BUS) 送到 DBUS 总线上，进而送至 DR1 或 DR2 操作数寄存器。输入数据可由实验台上的 8 个二进制数据开关 SW0—SW7 来设置，其中 SW0 是最低位，SW7 是最高位。开关向上时为 1，开关向下时为 0。

图中尾巴上带粗短线标记的信号都是控制信号，控制信号均为电位信号。T3、T4 是脉冲信号，印制板上已连接到实验台的时序电路产生的 T3、T4 信号上。S2、S1、S0、ALU_BUS、LD_DR2、LDDR1、M1、M2、SW_BUS#各电位控制信号用电平开关 K0—K15 来模拟。K0—K15 是一组用于模拟各控制电平信号的开关，开关向上时为 1，开关向下时为 0，每个开关无固定用途，可根据实验具体情况选用。S2、S1、S0、ALU_BUS、LDDR2、LDDR1 为高电平有效，SW_BUS#为低电平有效。M1=1 时，DR1 选择 D1—A1 作为数据输入端；M1=0 时，DR1 选择 D0—A0 作为数据输入端。当 LDDR1=1 时，在 T3 的下降沿，选中的数据被打入 DR1 寄存器。M2=1 时，DR2 选择 D1—A1 作为数据输入端；M2=0 时，DR2 选择 D0

—A0 作为数据输入端。当 LD_DR2=1 时，在 T3 的下降沿，选中的数据被打入 DR2 寄存器。

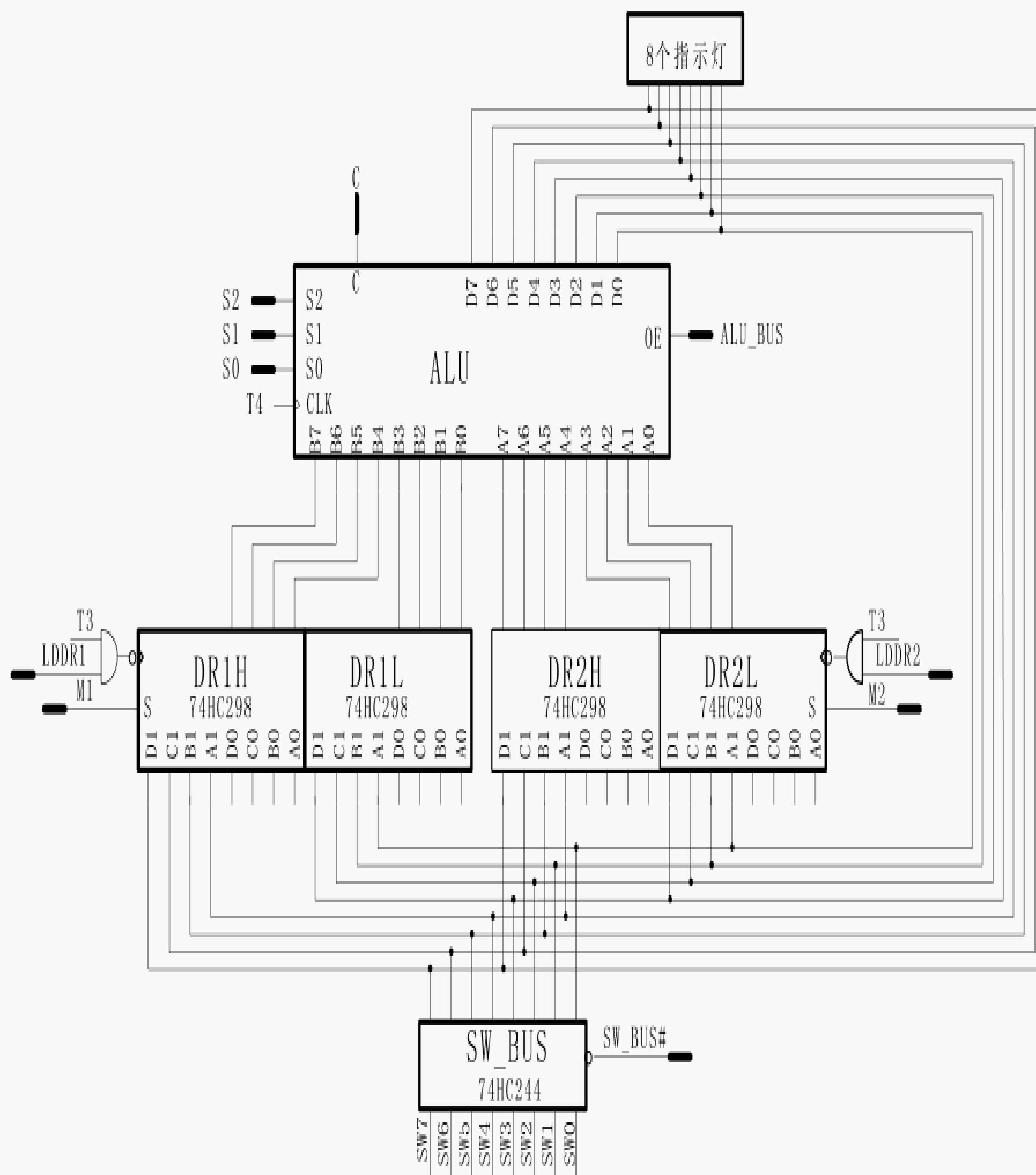


图6 运算器数据通路实验电路图

数据总线 DBUS 有 5 个数据来源：运算器 ALU，寄存器堆 RF，控制台开关 SW0—SW7，双端口存储器 IDT7132 和中断地址寄存器 IAR。在任何时刻，都不允许 2 个或者 2 个以上的数据源同时向数据总线 DBUS 输送数据，只允许 1 个(或者没有)数据源向数据总线 DBUS 输送数据。在本实验中，为了保证数据的正确设置和观察，请令 RS_BUS# = 1, LRW = 0, IAR_BUS# = 1。

为了在实验中，每次只产生一组 T1、T2、T3、T4 脉冲，需将实验台上的 DP、DB、DZ 开关进行正确设置。将 DP 开关置 1，将 DB、DZ 开关置 0，每按一次 QD 按钮，则顺序产生 T1、T2、T3、T4 各一个单脉冲。本实验中采用单脉冲输出。

三、实验设备

- (1) TEC—4 计算机组成原理实验系统 1 台
- (2) 双踪示波器一台
- (3) 直流万用表一只
- (4) 逻辑测试笔一支

四、实验任务

(1) 按图 6 所示，正确连接运算器模块与实验台上的电平开关 K0—K15。由于运算器的 C 和 C 指示灯、8 位数据开关 SW0—SW7、T3、T4 的连线已由印制电路板连好，故接线任务仅仅是完成有关控制信号与电平开关 K0—K15 的连线。正确设置开关 DZ、DB、DP。用数据开关 SW0—SW7 向 DR1 和 DR2 寄存器置数。

1. 置 ALU_BUS = 0，关闭 ALU 向数据总线 DBUS 的输出；置 SW_BUS# = 0，开启数据开关 SW0—SW7 向数据总线 DBUS 的输出。**注意，对于数据总线 DBUS（或者其他任何总线），在任一时刻，只能有一个数据源向它输出。**置 IR/DBUS 开关于 DBUS 位置，在数据开关 SW0—SW7 上设置各种数据，观察数据指示灯状态是否与数据开关状态一致。

2. 置 M1=1，选择 DBUS 作为 DR1 的数据源；置 LDDR1=1，按 QD 按钮，则将 DBUS 的数据打入 DR1。置 M2=1，选择 DBUS 作为 DR2 的数据源；置 LDDR2=1，按 QD 按钮，则将 DBUS 的数据打入 DR2。向 DR1 存入 0 1 0 1 0 1 0 1，向 DR2 存入 1 0 1 0 1 0 1 0。

3. 置 SW_BUS# = 1，关闭数据开关 SW0—SW7 对数据总线 DBUS 的输出；置 ALU_BUS = 1，开启 ALU 对 DBUS 的输出。选择 S2 = 0、S1 = 0、S1 = 1，使运算器进行直通运算，通过 DBUS 指示灯验证 DR2 中的内容是否为第 2 步设置的值。令 S2 = 0、S1 = 1、S0 = 0，使运算器进行加运算，通过 DBUS 指示灯验证 DR1 中的内容是否为第 2 步设置的值。在表 4 中填入控制信号状态与 DBUS 显示状态。

表 4 DR1、DR2 设置值检查

ALU_BUS	SW_BUS#	寄存器内容	S2 S1 S0	DBUS
		DR1 (01010101)、DR2 (10101010)		
		DR1 (01010101)、DR2 (10101010)		

(2) 验证运算器的算术运算和逻辑运算功能。

(2) 验证运算器的算术运算和逻辑运算功能。

1. 令 DR1 = 01100011B，DR2 = 10110100B，正确选择 S2、S1、S0，依次进行加、减、与、直通，乘实验，记下实验结果（数据和进位）并对结果进行分析。

2. 令 DR1 = 10110100B, DR2 = 01100011B, 正确选择 S2、S1、S0, 依次进行加、减、与、直通, 乘实验, 记下实验结果(数据和进位)并对结果进行分析。
3. 令 DR1 = 01100011B, DR2 = 01100011B, 正确选择 S2、S1、S0, 依次进行加、减、与、直通, 乘实验, 记下实验结果(数据和进位)并对结果进行分析。
4. 令 DR1 = 01001100B, DR2 = 10110011B, 正确选择 S2、S1、S0, 依次进行加、减、与、直通, 乘实验, 记下实验结果(数据和进位)并对结果进行分析。
5. 令 DR1=11111111B, DR2 = 11111111B, 正确选择 S2、S1、S0, 依次进行加、减、与、直通, 乘实验, 记下实验结果(数据和进位)并对结果进行分析。

(3) M1、M2 控制信号的作用是什么? 改变 M1、M2 的高低电平, 重复第(2)步, 观察出现什么问题?

五、实验步骤和实验结果

这是计算机组成原理课程中使用 TEC—4 计算机组成原理实验系统进行的第一个实验。除了实验本身的要求外, 一个很重要的任务是使学生熟悉 TEC—4 计算机组成原理实验系统, 从而掌握它, 学会使用它。

(1) 实验任务(1)的实验步骤及结果如下:

1. 接线

将 IAR_BUS#接 VCC, RS_BUS#接 VCC, LRW 接 GND, 禁止中断地址寄存器 IAR、寄存器堆 RF、双端口存储器向数据总线 DBUS 送数。也可以通过将 CEL#接 VCC 禁止双端口存储器向数据总线 DBUS 送数。

将 SW_BUS#接 K0, 将 ALU_BUS 接 K1, S0 接 K2, S1 接 K3, S2 接 K4, LDDR1 接 K5, LDDR2 接 K6, M1 接 VCC, M2 接 VCC。

2. 置开关 DB = 0, DZ = 0, DP = 1, 使实验系统处于单拍状态。
3. 合上实验台电源。按复位按钮 CLR#, 使实验系统处于初始状态。
4. 置 K0 (SW_BUS#) = 0, K1 (ALU_BUS) = 0。置开关 SW7—SW0 为 01010101B。将开关 IR/DBUS 拨到 DBUS 位置, 红色数据指示灯应显示 01010101B。它指示的是数据总线 DBUS 的值。
5. 置 K5 (LDDR1) = 1, K6 (LDDR2) = 0。按一次 QD 按钮, 将 01010101B 置入 DR1。
6. 置 K5 (LDDR1) = 0, K6 (LDDR2) = 1。将数据开关 SW7—SW0 置为 10101010B, 红色数据指示灯应显示 10101010B。按一次 QD 按钮, 将 10101010B 置入 DR2。
7. 置 K0 (SW_BUS#) = 1, 关闭数据开关 SW0—SW7 对数据总线 DBUS 的输出; 置 K1 (ALU_BUS) = 1, 开启 ALU 对 DBUS 的输出。置 K2 (S0) = 1、K3 (S1) = 0、K4 (S2) = 0, 使运算器进行直通运算。红色数据指示灯应显示 10101010B。这表示 DR2 置数正确。置 K2 (S0) = 0、K3 (S1) = 1、K4 (S2) = 0, 使运算器进行加运算, 红色数据指示灯应显示 11111111B。这表示 DR1 中的数确实是 01010101B。

(2) 以上是实验内容(1)的实验步骤和实验结果, 对于实验内容(2), 可仿照办理。欲察看 C 的值, 需按一次 QD 按钮。实验内容(2)的实验结果如下:

1. 令 DR1= 01100011B, DR2 = 10110100B, 加的结果是 00010111B, C = 1; 减的结果是 01010001B, C = 0; 与的结果是 00100000B, C 不变; 直通的结果是 10110100B, C 不变; 乘的结果是 00001100B, C 不变。
2. 令 DR1=10110100B, DR2 = 01100011B, 加的结果是 00010111B, C = 1; 减的结果是 10101111B, C = 1; 与的结果是 00100000B, C 不变; 直通的结果是 01100011B, C 不变; 乘的结果是 00001100B,

C 不变。

3. 令 $DR1 = 01100011B$, $DR2 = 01100011B$, 加的结果是 $11000110B$, $C = 0$; 减的结果是 $00000000B$, $C = 0$; 与的结果是 $01100011B$, C 不变; 直通的结果是 $01100011B$, C 不变; 乘的结果是 $00001001B$, C 不变。
4. 令 $DR1 = 01001100$, $DR2 = 10110011$, 加的结果是 $11111111B$, $C = 0$; 减的结果是 $01100111B$, $C = 0$; 与的结果是 $00000000B$, C 不变; 直通的结果是 $10110011B$, C 不变; 乘的结果是 $00100100B$, C 不变。
5. 令 $DR1 = 11111111B$, $DR2 = 11111111B$, 加的结果是 $11111110B$, $C = 1$; 减的结果是 $00000000B$, C 不变; 与的结果是 $11111111B$, C 不变; 直通的结果是 $11111111B$, C 不变; 乘的结果是 $11100001B$, C 不变。

(3) $M1$ 用于选择 $DR1$ 的数据输入源。 $M1=1$ 时, $DR1$ 选择 $D1—A1$ 作为数据输入端; $M1=0$ 时, $DR1$ 选择 $D0—A0$ 作为数据输入端。 $M2$ 用于选择 $DR2$ 的数据输入源。 $M2=1$ 时, $DR2$ 选择 $D1—A1$ 作为数据输入端; $M2=0$ 时, $DR2$ 选择 $D0—A0$ 作为数据输入端。在做实验内容 (2) 时, 如果将 $M1$ 或者 $M2$ 由接 VCC 改为接 GND , 则 $DR1$ 或者 $DR2$ 选择寄存器堆作为数据输入源。由于没有给寄存器堆中的寄存器置数, 因此寄存器的内容处于不可知状态。因此在 $M1 = 0$ 或者 $M2 = 0$ 时, 进行实验内容 (2), 结果将是无法预知的。

第三节 双端口存储器原理实验

一、实验目的

- (1) 了解双端口静态随机存储器 IDT7132 的工作特性及使用方法。
- (2) 了解半导体存储器怎样存储和读出数据。
- (3) 了解双端口存储器怎样并行读写，产生冲突的情况如何。

二、实验电路

图 7 示出了双端口存储器的实验电路图。这里使用了一片 IDT7132 (U36) (2048 X 8 位)，两个端口的地址输入 A8—A10 引脚接地，因此实际使用存储容量为 256 字节。左端口的数据部分连接数据总线 DBUS7—DBUS0，右端口的数据部分连接指令总线 INS7—INS0。一片 GAL22V10 (U37) 作为左端口的地址寄存器 (AR1)，内部具有地址递增的功能。两片 4 位的 74HC298 (U28、U27) 作为右端口的地址寄存器 (AR2H、AR2L)，带有选择输入地址源的功能。使用两组发光二极管指示灯显示地址和数据：通过开关 IR/DBUS 切换显示数据总线 DBUS 和指令寄存器 IR 的数据，通过开关 AR1/AR2 切换显示左右两个端口的存储地址。写入数据由实验台操作板上的二进制开关 SW0—SW7 设置，并经过 SW_BUS 三态门 74HC244 (U38) 发送到数据总线 DBUS 上。指令总线 INS 上的指令代码输出到指令寄存器 IR (U20)，这是一片 74HC374。

存储器 IDT7132 有 6 个控制引脚：CEL#、LRW、OEL#、CER#、RRW、OER#。CEL#、LRW、OEL# 控制左端口读、写操作，CER#、RRW、OER# 控制右端口读、写操作。CEL# 为左端口选择引脚，低有效。当 CEL# = 1 时，禁止左端口读、写操作；当 CEL# = 0 时，允许左端口读、写操作。当 LRW 为高时，左端口进行读操作；当 LRW 为低时，左端口进行写操作。当 OEL# 为低时，将左端口读出的数据放到数据总线 DBUS 上；当 OEL# 为高时，禁止左端口读出的数据放到数据总线 DBUS 上。CER#、RRW、OER# 控制右端口读、写操作的方式与 CEL#、LRW、OEL# 控制左端口读、写操作的方式类似，不过右端口读出的数据放到指令总线上而不是数据总线上。实验台上的 OEL# 由 LRW 经反相产生。当 CEL# = 0 且 LRW = 1 时，左端口进行读操作，同时将读出的数据放到数据总线 DBUS 上。当 CER# = 0 且 LRW = 0 时，在 T3 的上升沿开始进行写操作，将数据总线上的数据写入存储器。实验台上已连接 T3 到时序发生器的 T3 输出。实验台上 OER# 已固定接地，RRW 固定接高电平，CER# 由 CER 反相产生，因此当 CER = 1 且 LDIR = 1 时，右端口读出的指令在 T4 的上升沿打入 IR 寄存器。

存储器的地址由地址寄存器 AR1、AR2 提供，而 AR1 和 AR2 的内容根据数码开关 SW0—SW7 设置产生，并经三态门 SW_BUS 发送到数据总线时被 AR1 或 AR2 接收，三态门的控制信号 SW_BUS# 是低电平有效。数据总线 DBUS 有 5 个数据来源：运算器 ALU，寄存器堆 RF，控制台开关 SW0—SW7，双端口存储器 IDT7132 和中断地址寄存器 IAR。在任何时刻，都不允许 2 个或者 2 个以上的数据源同时向数据总线 DBUS 输送数据，只允许 1 个 (或者没有) 数据源向数据总线 DBUS 输送数据。在本实验中，为了保证数据的正确设置和观察，请令 RS_BUS# = 1, ALU_BUS = 0, IAR_BUS# = 1。AR1 的控制信号是 LDAR1 和 AR1_INC。当 LDAR1 = 1 时，AR1 从 DBUS 接收地址；当 AR1_INC = 1 时，使 AR1 中的存储器地址增加 1；在 T4 的上升沿，产生新的地址；LDAR1 和 AR1_INC 两者不可同时为 1。AR2 的控制信号是 LDAR2 和 M3。当 M3 = 1 时，AR2 从数据总线 DBUS 接收数据；当 M3 = 0 时，AR2 以 PC 总线 PC0—PC7 作为数据来源。当 LDAR2 = 1 时，在 T2 的下降沿，将新的 PC 值打入 AR2。

三、实验设备

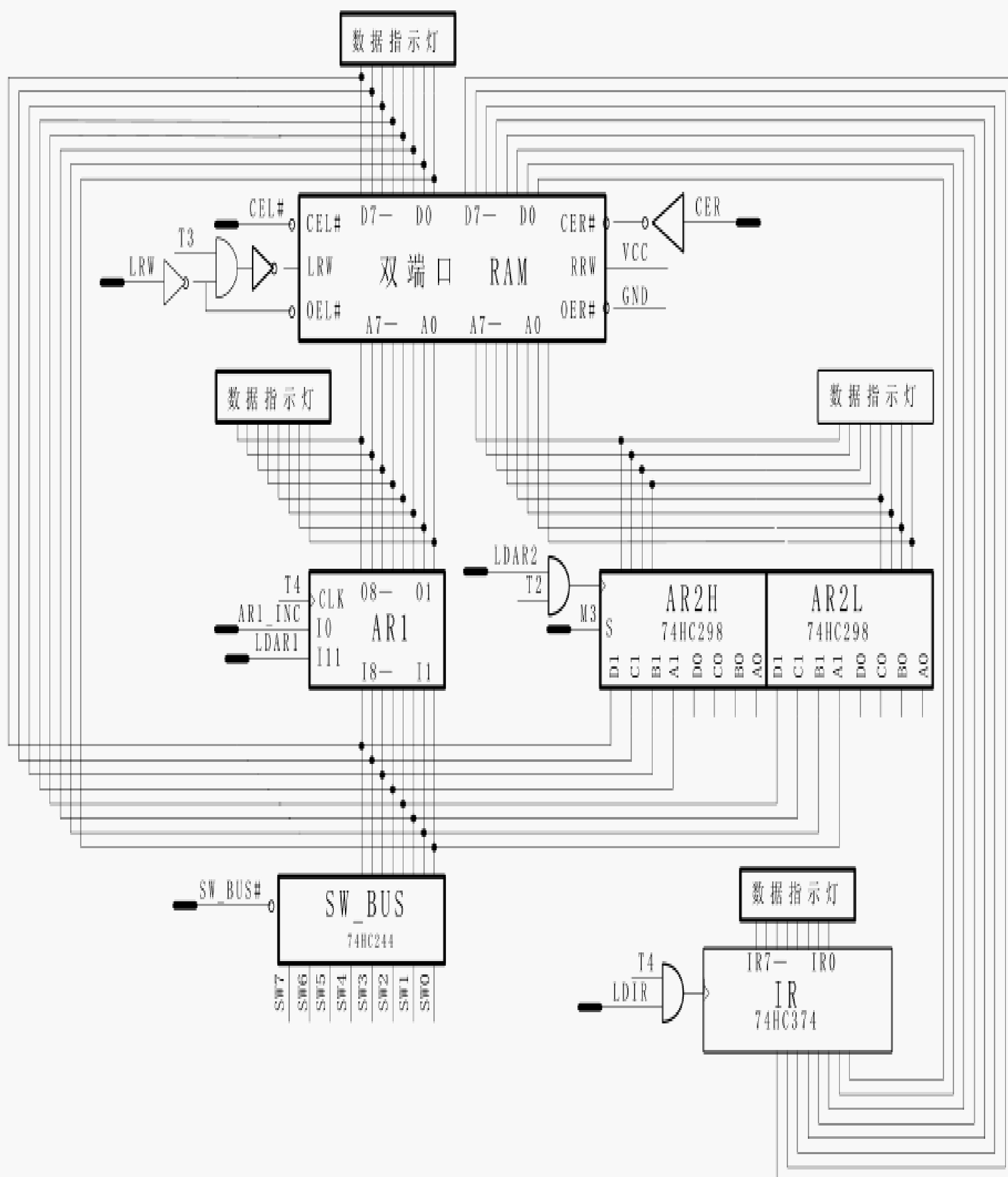


图7 双端口存储器实验电路图

- (1) TEC—4 计算机组成原理实验系统一台
- (2) 双踪示波器一台
- (3) 直流万用表一只
- (4) 逻辑测试笔一支

四、实验任务

(1) 按图 7 所示, 将有关控制信号和二进制开关对应接好, 仔细复查一遍, 然后接通电源。

(2) 将数码开关 SW0—SW7 (SW0 是最低位) 设置为 00H, 将此数据作为地址置入 AR1; 然后重新设置二进制开关控制, 将数码开关 SW0—SW7 上的数 00H 写入 RAM 第 0 号单元。依此方法, 在存储器 10H 单元写入数据 10H, 20H 单元写入 20H, 30H 单元写入 30H, 40H 单元写入 40H, 共存入 5 个数据。

使用双端口存储器的左端口, 依次读出存储器第 00H、10H、20H、30H、40H 单元中的内容, 观察上述各单元中的内容是否与该单元的地址号相同。请记录数据。**注意: 总线上禁止两个以上部件同时向总线输出数据。**当存储器进行读出操作时, 必须关闭 SW_BUS 三态门! 而当向 AR1 送入地址时, 双端口存储器不能被选中。

(3) 通过双端口存储器右端口 (指令端口), 依次把存储器第 00H、10H、20H、30H、40H 单元中的内容置入指令寄存器 IR, 观察结果是否与 (2) 相同, 并记录数据。

(4) 双端口存储器的并行读写和访问冲突测试。

置 $CEL\#=0$ 且 $CER=1$, 使存储器左、右端口同时被选中。当 AR1 和 AR2 的地址不不同时, 没有访问冲突; 地址相同时, 由于都是读出操作, 也不冲突。如果左、右端口地址相同且一个进行读操作、另一个进行写操作, 则发生冲突。要检测冲突, 可以用示波器测试 BUSYL 和 BUSYR 插孔 (分别是两个端口的“忙”信号输出)。BUSY 为 0 时不一定发生冲突, 但发生冲突时, BUSY 一定为 0。当某一个端口 (无论是左端口还是右端口) 的 $BUSY=0$ 时, 对该端口的写操作被 IDT7132 忽略掉。

五、实验步骤及实验结果

(1) 接线

IAR_BUS#接 VCC, ALU_BUS 接 GND, RS_BUS#接 VCC, 禁止中断地址寄存器、运算器、多端口寄存器堆 RF 向数据总线 DBUS 送数据。AR1_INC 接 GND, M3 接 VCC, 使地址寄存器 AR1 和 AR2 从数据总线 DBUS 取得地址数据。

$CEL\#$ 接 K0, LRW 接 K1, CER 接 K2, LDAR1 接 K3, LDAR2 接 K4, SW_BUS#接 K5, LDIR 接 K6。

置 $DP=1$, $DB=0$, $DZ=0$, 使实验台处于单拍状态。

合上电源。按复位按钮 CLR#, 使实验系统处于初始状态。

(2) 向存储器写数, 并读出进行检查。

1. 令 $K0(CEL\#)=1$, $K1(LRW)=1$, $K2(CER)=0$, $K3(LDAR1)=1$, $K4(LDAR2)=0$, $K5(SW_BUS\#)=0$, $K6(LDIR)=0$ 。将 IR/DBUS 开关拨到 DBUS 位置, 将 AR1/AR2 开关拨到 AR1 位置。置 SW7—SW0 = 00H, 按一次 QD 按钮, 将 00H 写入 AR1, 绿色的地址指示灯应显示 00H。

令 $K0(CEL\#)=0$, $K1(LRW)=0$, $K3(LDAR1)=0$, 按一次 QD 按钮, 则将 00H 数据写入存储器的 00H 单元。

依次重复进行, 在存储器 10H 单元写入数据 10H, 20H 单元写入 20H, 30H 单元写入 30H, 40H 单元写入 40H, 共存入 5 个数据。

2. 令 $K0(CEL\#)=1$, $K1(LRW)=1$, $K2(CER)=0$, $K3(LDAR1)=1$, $K4(LDAR2)=0$, $K5(SW_BUS\#)$

= 0, K6 (LDIR) = 0。将 IR/DBUS 开关拨到 DBUS 位置, 将 AR1/AR2 开关拨到 AR1 位置。置 SW7—SW0 = 00H, 按一次 QD 按钮, 将 00H 写入 AR1, 绿色的地址指示灯应显示 00H。令 K5 (SW_BUS#) = 1, 然后令 K3 (LDAR1) = 0, K0 (CEL#) = 0, K1 (LRW) = 1, 则读出存储器的 00H 单元的数据, 读出的数据显示在 DBUS 数据指示灯上, 应为 00H。照此方法, 可依次读出存储器单元 10H、20H、30H、40H 的数据。

(3) 读出存储器的数据, 写入 IR。

令 K0 (CEL#) = 1, K1 (LRW) = 1, K2 (CER) = 0, K3 (LDAR1) = 0, K4 (LDAR2) = 1, K5 (SW_BUS#) = 0, K6 (LDIR) = 0。将 IR/DBUS 开关拨到 IR 位置, 将 AR1/AR2 开关拨到 AR2 位置。置 SW7—SW0 = 00H, 按一次 QD 按钮, 将 00H 写入 AR2, 绿色的地址指示灯应显示 00H。令 K4 (LDAR2) = 0, K2 (CER) = 1, K6 (LDIR) = 1, 按一次 QD 按钮, 则从右端口读出存储器的 00H 单元的数据, 读出的数据写入指令寄存器 IR, 显示在 IR 数据指示灯上, 应为 00H。照此方法, 可从右端口依次读出存储器单元 10H、20H、30H、40H 的数据, 写入指令寄存器 IR。

(4) 双端口存储器的并行读写和访问冲突测试

1. 令 K0 (CEL#) = 1, K1 (LRW) = 1, K2 (CER) = 0, K3 (LDAR1) = 1, K4 (LDAR2) = 0, K5 (SW_BUS#) = 0, K6 (LDIR) = 0。将 AR1/AR2 开关拨到 AR1 位置。置 SW7—SW0 = 38H, 按一次 QD 按钮, 将 38H 写入 AR1, 绿色的地址指示灯应显示 38H。令 K3 (LDAR1) = 0, K4 (LDAR2) = 1, K5 (SW_BUS#) = 0, 将 AR1/AR2 开关拨到 AR2 位置。置 SW7—SW0 = 38H, 按一次 QD 按钮, 将 38H 写入 AR2, 绿色的地址指示灯应显示 38H。
2. 先令 K2 (CER) = 1, K0 (CEL#) = 1, 用示波器探头测试 BUSYL 插孔, BUSYL 应为高电平。保持 K2 (CER) 不变, 将 K2 (CEL#) 拨动到 0 位置, 示波器上的 BUSYL 信号从高电平变为低电平; 再将 K0 (CEL#) 拨到 1 位置, BUSYL 信号从低电平变为高电平。
3. 先令 K0 (CEL#) = 0, K2 (CER) = 0, 用示波器探头测试 BUSYR 插孔, BUSYR 应为高电平。保持 K0 (CEL#) 不变, 将 K2 (CER) 拨动到 1 位置, 示波器上的 BUSYR 信号也从高电平变为低电平; 再将 K2 (CER) 拨到 0 位置, BUSYL 信号也从低电平变为高电平。

第四节 数据通路组成实验

一、实验目的

- (1) 将双端口通用寄存器堆和双端口存储器模块联机;
- (2) 进一步熟悉计算机的数据通路;
- (3) 掌握数字逻辑电路中故障的一般规律, 以及排除故障的一般原则和方法;
- (4) 锻炼分析问题与解决问题的能力, 在出现故障的情况下, 独立分析故障现象, 并排除故障。

二、实验电路

图 8 示出了数据通路实验电路图, 它是将双端口存储器实验模块和一个双端口通用寄存器堆模块(RF)连接在一起形成的。双端口存储器的指令端口不参与本次实验。通用寄存器堆连接运算器模块, 本实验涉及其中的操作数寄存器 DR2。

由于双端口存储器 RAM 是三态输出, 因而可以将它直接连接到数据总线 DBUS 上。此外, DBUS 上还连接着双端口通用寄存器堆。这样, 写入存储器的数据可由通用寄存器堆提供, 而从存储器 RAM 读出的数据也可送到通用寄存器堆保存。

双端口存储器 RAM 已在第三节做过介绍, DR2 在第三节的实验中使用过。通用寄存器堆 RF(U32) 由一个 ISP1016 实现, 功能上与两个 4 位的 MC14580 并联构成的寄存器堆类似。RF 内含四个 8 位的通用寄存器 R0、R1、R2、R3, 带有一个写入端口和两个输出端口, 从而可以同时写入一路数据, 读出两路数据。写入端口取名为 WR 端口, 连接一个 8 位的暂存寄存器 (U14) ER, 这是一个 74HC374。输出端口取名为 RS 端口 (B 端口)、RD 端口 (A 端口), 连接运算器模块的两个操作数寄存器 DR1、DR2。RS 端口 (B 端口) 的数据输出还可通过一个 8 位的三态门 RS0(U15) 直接向 DBUS 输出。

双端口通用寄存器堆模块的控制信号中, RS1、RS0 用于选择从 RS 端口 (B 端口) 读出的通用寄存器, RD1、RD0 用于选择从 RD 端口 (A 端口) 读出的通用寄存器。而 WR1、WR0 则用于选择从 WR 端口写入的通用寄存器。WRD 是写入控制信号, 当 WRD = 1 时, 在 T2 上升沿时刻, 将暂存寄存器 ER 中的数据写入通用寄存器堆中由 WR1、WR0 选中的寄存器; 当 WRD = 0 时, ER 中的数据不写入通用寄存器中。LDER 信号控制 ER 从 DBUS 写入数据, 当 LDER=1 时, 在 T4 的上升沿, DBUS 上的数据写入 ER。RS_BUS#信号则控制 RS 端口到 DBUS 的输出三态门, 是一个低有效信号。以上控制信号各自连接一个二进制开关 K0—K15。

三、实验设备

- (1) TEC—4 计算机组成原理实验仪一台
- (2) 双踪示波器一台
- (3) 直流万用表一只
- (4) 逻辑测试笔一支

四、实验任务

- (1) 将实验电路与控制台的有关信号进行线路连接, 方法同前面的实验。
- (2) 用 8 位数据开关向 RF 中的四个通用寄存器分别置入以下数据: R0=0FH, R1=0F0H, R2=55H, R3=0AAH。
给 R0 置入 0FH 的步骤是: 先用 8 位数码开关 SW0—SW7 将 0FH 置入 ER, 并且选择 WR1=0、WR0=

0、WRD = 1，再将 ER 的数据置入 RF。给其他通用寄存器置入数据的步骤与此类似。

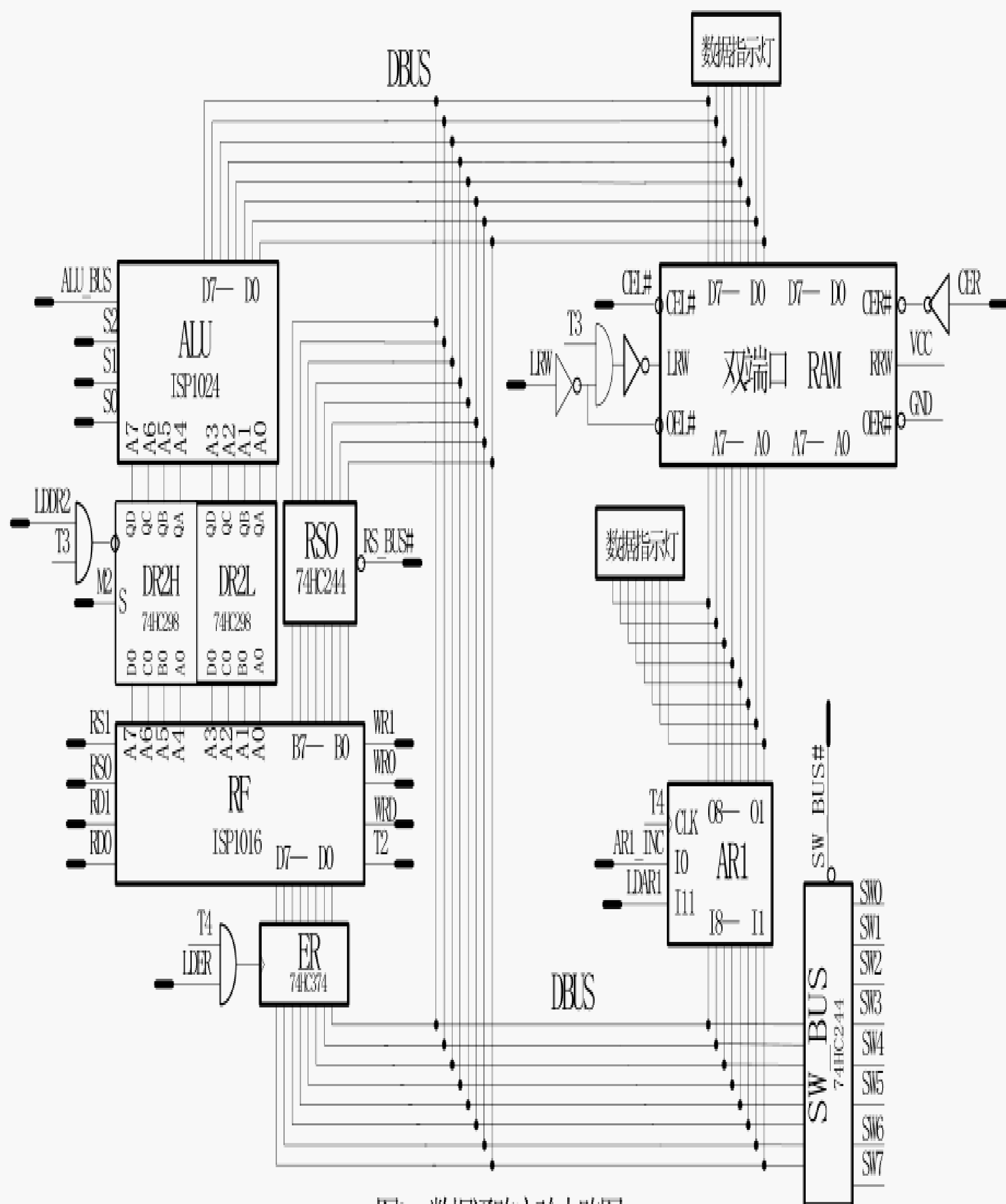


图8 数据通路实验电路图

(3) 分别将 R0 至 R3 中的数据同时读入到 DR2 寄存器中和 DBUS 上, 观察其数据是否是存入 R0 至 R3 中的数据, 并记录数据。其中 DBUS 上的数据可直接用指示灯显示, DR2 中的数据可通过运算器 ALU, 用直通方式将其送往 DBUS。

(4) 用 8 位数码开关 SW0—SW7 向 AR1 送入一个地址 0FH, 然后将 R0 中的 0FH 写入双端口 RAM。用同样的方法, 依次将 R1 至 R3 中的数据写入 RAM 中的 0F0H、55H、0AAH 单元。

(5) 分别将 RAM 中 0AAH 单元的数据写入 R0, 55H 单元的数据写入 R1, 0F0H 单元写入 R2, 0FH 单元写入 R3。然后将 R3、R2、R1、R0 中的数据读出到 DBUS 上, 通过指示灯验证读出的数据是否正确, 并记录数据。

(6) 进行 RF 并行输入输出试验。

1. 选择 RS 端口 (B 端口) 对应 R0, RD 端口 (A 端口) 对应 R1, WR 端口对应 R2, 并使 WRD = 1, 观察并行输入输出的结果。选择 RS 端口对应 R2, 验证刚才的写入是否生效。记录数据。

2. 保持 RS 端口 (B 端口) 和 WR 端口同时对应 R2, WRD = 1, 而 ER 中置入新的数据, 观察并行输入输出的结果, RS 端口输出的是旧的还是新的数据?

(7) 在数据传送过程中, 发现了什么故障? 如何克服的?

五、实验步骤与实验结果

(1) 接线

IAR_BUS#接 VCC, 禁止中断地址寄存器 IAR 向数据总线 DBUS 送数据。CER 接 GND, 禁止存储器右端口工作。AR1_INC 接 GND, 禁止 AR1 加 1。S2 接 GND, S1 接 GND, S0 接 VCC, 使运算器 ALU 处于直通方式。M2 接 GND, 使 DR2 选择寄存器堆 RF 作为数据来源。置 DP = 1, DZ = 0, DB = 0, 使实验系统开机后处于单拍状态。

K0 接 SW_BUS#, K1 接 RS_BUS#, K2 接 ALU_BUS, K3 接 CEL#, K4 接 LRW, K5 接 LDAR1, K6 接 LDDR2, K7 接 LDER, K8 接 RS0, K9 接 RS1, K10 接 RD0, K11 接 RD1, K12 接 WR0, K13 接 WR1, K14 接 WRD。

合上电源。按 CLR#按钮, 使实验系统处于初始状态。

(2) 向 RF 中的四个通用寄存器分别置入数据

令 K1 (RS_BUS#) = 1, K2 (ALU_BUS) = 0, K3 (CEL#) = 1, K4 (LRW) = 1, K5 (LDAR1) = 0, K6 (LDDR2) = 0, K8 (RS0) = 0, K9 (RS1) = 0, K10 (RD0) = 0, K11 (RD1) = 0, K12 (WR0) = 0, K13 (WR1) = 0, K14 (WRD) = 0。

a. 令 K0 (SW_BUS#) = 0, K7 (LDER) = 1。置 SW7—SW0 为 0FH, 按一次 QD 按钮, 将 0FH 写入暂存寄存器 ER。

令 K7 (LDER) = 0, K14 (WRD) = 1, K12 (WR0) = 0, K13 (WR1) = 0, 按一次 QD 按钮, 将 0FH (在 ER 中) 写入 R0 寄存器。

b. 令 K0 (SW_BUS#) = 0, K7 (LDER) = 1。置 SW7—SW0 为 F0H, 按一次 QD 按钮, 将 0F0H 写入暂存寄存器 ER。

令 K7 (LDER) = 0, K14 (WRD) = 1, K12 (WR0) = 1, K13 (WR1) = 0, 按一次 QD 按钮, 将 F0H (在 ER 中) 写入 R1 寄存器。

c. 令 K0 (SW_BUS#) = 0, K7 (LDER) = 1。置 SW7—SW0 为 55H, 按一次 QD 按钮, 将 55H 写入暂存寄存器 ER。

令 K7 (LDER) = 0, K14 (WRD) = 1, K12 (WR0) = 0, K13 (WR1) = 1, 按一次 QD 按钮, 将 55H (在 ER 中) 写入 R2 寄存器。

d. 令 K0 (SW_BUS#) = 0, K7 (LDER) = 1。置 SW7—SW0 为 0AAH, 按一次 QD 按钮, 将 0AAH 写入暂存寄存器 ER。

令 K7 (LDER) = 0, K14 (WRD) = 1, K12 (WR0) = 1, K13 (WR1) = 1, 按一次 QD 按钮, 将 0AAH (在 ER 中) 写入 R0 寄存器。

(3) 分别将 R0 至 R3 中的数据同时读入到 DR2 寄存器中和 DBUS 上, 观察其数据是否是存入 R0 至 R3 中的数据。

1. 令 K0 (SW_BUS#) = 1, K2 (ALU_BUS) = 0, K3 (CEL#) = 1, K4 (LRW) = 1, K5 (LDAR1) = 0, K6 (LDDR2) = 0, K7 (LDER) = 0, K10 (RD0) = 0, K11 (RD1) = 0, K12 (WR0) = 0, K13 (WR1) = 0, K14 (WRD) = 0。

将开关 IR/DBUS 至于 DBUS 位置。令 K1 (RS_BUS#) = 0, 使寄存器堆中的数据送 DBUS 总线。令 K8 (RS0) = 0, K9 (RS1) = 0, R0 中的数据通过 B 端口送 DBUS, 数据指示灯应显示 0FH。令 K8 (RS0) = 1, K9 (RS1) = 0, R1 中的数据通过 B 端口送 DBUS, 数据指示灯应显示 0F0H。令 K8 (RS0) = 0, K9 (RS1) = 1, R2 中的数据通过 B 端口送 DBUS, 数据指示灯应显示 55H。令 K8 (RS0) = 1, K9 (RS1) = 1, R3 中的数据通过 B 端口送 DBUS, 数据指示灯应显示 0AAH。

2. 令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 1, K3 (CEL#) = 1, K4 (LRW) = 1, K5 (LDAR1) = 0, K7 (LDER) = 0, K8 (RS0) = 0, K9 (RS1) = 0, K12 (WR0) = 0, K13 (WR1) = 0, K14 (WRD) = 0。

将开关 IR/DBUS 至于 DBUS 位置。令 K2 (ALU_BUS) = 1, 使运算器 ALU 的运算结果送 DBUS 总线。由于 S2 接 GND, S1 接 GND, S0 接 VCC, ALU 做直通运算, 因此 DBUS 数据指示灯显示的是 DR2 寄存器的值。令 K10 (RD0) = 0, K11 (RD1) = 0, K6 (LDDR2) = 1, 按一次 QD 按钮, R0 中的数据通过 A 端口送入 DR2, DBUS 数据指示灯应显示 0FH。令 K10 (RD0) = 1, K11 (RD1) = 0, K6 (LDDR2) = 1, 按一次 QD 按钮, R1 中的数据通过 A 端口送入 DR2, DBUS 数据指示灯应显示 0F0H。令 K6 (LDDR2) = 1, K10 (RD0) = 0, K11 (RD1) = 1, 按一次 QD 按钮, R2 中的数据通过 A 端口送入 DR2, DBUS 数据指示灯应显示 55H。令 K10 (RD0) = 1, K11 (RD1) = 1, K6 (LDDR2) = 1, 按一次 QD 按钮, R3 中的数据通过 A 端口送入 DR2, DBUS 数据指示灯应显示 0AAH。

(4) 将 R0、R1、R2、R3 中的数据依次送入存储器 0FH、0F0H、55H、0AAH 单元。

令 K2 (ALU_BUS) = 0, K5 (LDAR1) = 0, K6 (LDDR2) = 0, K7 (LDER) = 0, K10 (RD0) = 0, K11 (RD1) = 0, K12 (WR0) = 0, K13 (WR1) = 0, K14 (WRD) = 0。

a. 置 AR1/AR2 开关到 AR1 位置。令 K0 (SW_BUS#) = 0, K1 (RS_BUS#) = 1, K3 (CEL#) = 1, K5 (LDAR1) = 1, 置 SW7—SW0 为 0FH, 按一次 QD 按钮, 将 AR1 置为 0FH, 地址指示灯应显示 0FH。

令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 0, 禁止数据开关 SW7—SW0 送 DBUS, 允许寄存器堆送数据总线 DBUS。令 K3 (CEL#) = 0, K4 (LRW) = 0, K5 (LDAR1) = 0, K8 (RS0) = 0, K9 (RS1) = 0, 按一次 QD 按钮, 将 R0 中的数据写入存储器 0FH 单元。

b. 置 AR1/AR2 开关到 AR1 位置。令 K0 (SW_BUS#) = 0, K1 (RS_BUS#) = 1, K3 (CEL#) = 1, K5 (LDAR1) = 1, 置 SW7—SW0 为 0F0H, 按一次 QD 按钮, 将 AR1 置为 0F0H, 地址指示灯应显示 F0H。

令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 0, 禁止数据开关 SW7—SW0 送 DBUS, 允许寄存器堆送数据总线 DBUS。令 K3 (CEL#) = 0, K4 (LRW) = 0, K5 (LDAR1) = 0, K8 (RS0) = 1,

K9 (RS1) = 0, 按一次 QD 按钮, 将 R1 中的数据写入存储器 F0H 单元。

- c. 置 AR1/AR2 开关到 AR1 位置。令 K0 (SW_BUS#) = 0, K1 (RS_BUS#) = 1, K3 (CEL#) = 1, K5 (LDAR1) = 1, 置 SW7—SW0 为 55H, 按一次 QD 按钮, 将 AR1 置为 55H, 地址指示灯应显示 55H。

令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 0, 禁止数据开关 SW7—SW0 送 DBUS, 允许寄存器堆送数据总线 DBUS。令 K3 (CEL#) = 0, K4 (LRW) = 0, K5 (LDAR1) = 0, K8 (RS0) = 0, K9 (RS1) = 1, 按一次 QD 按钮, 将 R2 中的数据写入存储器 55H 单元。

- d. 置 AR1/AR2 开关到 AR1 位置。令 K0 (SW_BUS#) = 0, K1 (RS_BUS#) = 1, K3 (CEL#) = 1, K5 (LDAR1) = 1, 置 SW7—SW0 为 0AAH, 按一次 QD 按钮, 将 AR1 置为 0AAH, 地址指示灯应显示 0AAH。

令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 0, 禁止数据开关 SW7—SW0 送 DBUS, 允许寄存器堆送数据总线 DBUS。令 K3 (CEL#) = 0, K4 (LRW) = 0, K5 (LDAR1) = 0, K8 (RS0) = 1, K9 (RS1) = 1, 按一次 QD 按钮, 将 R3 中的数据写入存储器 0AAH 单元。

(5) 将 RAM 中 0AAH、55H、0F0H、0FH 单元的数据依次写入 R0、R1、R2、R3。然后将 R3、R2、R1、R0 中的数据读出到 DBUS 上, 通过指示灯验证读出的数据是否正确。

1. 令 K1 (RS_BUS#) = 1, K2 (ALU_BUS) = 0, K6 (LDDR2) = 0, K8 (RS0) = 0, K9 (RS1) = 0, K10 (RD0) = 0, K11 (RD1) = 0。

- a. 令 K0 (SW_BUS#) = 0, K5 (LDAR1) = 1, K3 (CEL#) = 1, K14 (WRD) = 0, 置 SW7—SW0 为 0AAH, 按一次 QD 按钮, 将 AR1 置为 0AAH。

令 K14 (WRD) = 0, K0 (SW_BUS#) = 1, K3 (CEL#) = 0, K4 (LRW) = 1, K5 (LDAR1) = 0, K7 (LDER) = 1, 按一次 QD 按钮, 将存储器 0AAH 单元的内容读出, 写入到暂存寄存器 ER。

令 K0 (SW_BUS#) = 1, K3 (CEL#) = 1, K14 (WRD) = 1, K5 (LDAR1) = 0, K7 (LDER) = 0, K12 (WR0) = 0, K13 (WR1) = 0, 按一次 QD 按钮, 将 ER 中的数据写入 R0。

- b. 令 K0 (SW_BUS#) = 0, K5 (LDAR1) = 1, K3 (CEL#) = 1, K14 (WRD) = 0, 置 SW7—SW0 为 55H, 按一次 QD 按钮, 将 AR1 置为 55H。令 K14 (WRD) = 0, K0 (SW_BUS#) = 1, K3 (CEL#) = 0, K4 (LRW) = 1, K5 (LDAR1) = 0, K7 (LDER) = 1, 按一次 QD 按钮, 将存储器 55H 单元的内容读出, 写入到暂存寄存器 ER。令 K0 (SW_BUS#) = 1, K5 (LDAR1) = 0, K7 (LDER) = 0, K3 (CEL#) = 1, K14 (WRD) = 1, K12 (WR0) = 1, K13 (WR1) = 0, 按一次 QD 按钮, 将 ER 中的数据写入 R1。

令 K0 (SW_BUS#) = 0, K5 (LDAR1) = 1, K3 (CEL#) = 1, K14 (WRD) = 0, 置 SW7—SW0 为 0F0H, 按一次 QD 按钮, 将 AR1 置为 0F0H。令 K14 (WRD) = 0, K0 (SW_BUS#) = 1, K3 (CEL#) = 0, K4 (LRW) = 1, K5 (LDAR1) = 0, K7 (LDER) = 1, 按一次 QD 按钮, 将存储器 0F0H 单元的内容读出, 写入到暂存寄存器 ER。令 K0 (SW_BUS#) = 1, K5 (LDAR1) = 0, K7 (LDER) = 0, K3 (CEL#) = 1, K14 (WRD) = 1, K12 (WR0) = 0, K13 (WR1) = 1, 按一次 QD 按钮, 将 ER 中的数据写入 R2。

令 K0 (SW_BUS#) = 0, K5 (LDAR1) = 1, K3 (CEL#) = 1, K14 (WRD) = 0, 置 SW7—SW0 为 0FH, 按一次 QD 按钮, 将 AR1 置为 0FH。令 K14 (WRD) = 0, K0 (SW_BUS#) = 1, K3 (CEL#) = 0, K4 (LRW) = 1, K5 (LDAR1) = 0, K7 (LDER) = 1, 按一次 QD 按钮, 将存储器 0FH 单元的内容读出, 写入到暂存寄存器 ER。令 K0 (SW_BUS#) = 1, K5 (LDAR1) = 0, K7 (LDER) = 0, K3 (CEL#) = 1, K14 (WRD) = 1, K12 (WR0) = 1, K13 (WR1) = 1, 按一次 QD 按钮, 将 ER 中的数据写入 R3。

2. 令 K0 (SW_BUS#) = 1, K2 (ALU_BUS) = 0, K3 (CEL#) = 1, K4 (LRW) = 1, K5 (LDAR1) = 0,

K6 (LDDR2) = 0, K7 (LDER) = 0, K10 (RD0) = 0, K11 (RD1) = 0, K12 (WR0) = 0, K13 (WR1) = 0, K14 (WRD) = 0。

将开关 IR/DBUS 至于 DBUS 位置。令 K1 (RS_BUS#) = 0, K8 (RS0) = 0, K9 (RS1) = 0, 数据指示灯显示 R0 的值, 应为 0AAH。令 K8 (RS0) = 1, K9 (RS1) = 0, 数据指示灯显示 R1 的值, 应为 55H。令 K8 (RS0) = 0, K9 (RS1) = 1, 数据指示灯显示 R2 的值, 应为 0F0H。令 K8 (RS0) = 1, K9 (RS1) = 1, 数据指示灯显示 R3 的值, 应为 0FH。

(6) 进行 RF 并行输入输出试验

1. 选择 RS 端口 (B 端口) 对应 R0, RD 端口 (A 端口) 对应 R1, WR 端口对应 R2, 并使 WRD = 1, 观察并行输入输出的结果。选择 RS 端口对应 R2, 验证刚才的写入是否生效。

令 K3 (CEL#) = 1, K4 (LRW) = 1, K5 (LDAR1) = 0。

将开关 IR/DBUS 至于 DBUS 位置。令 K0 (SW_BUS#) = 0, K1 (RS_BUS#) = 1, K2 (ALU_BUS) = 0, K6 (LDDR2) = 0, K7 (LDER) = 1, K14 (WRD) = 0。将 SW7—SW0 置为 35H, 按一次 QD 按钮, 将 35H 写入暂存寄存器 ER。

令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 0, K2 (ALU_BUS) = 0, K14 (WRD) = 1, K6 (LDDR2) = 1。再令 K8 (RS0) = 0, K9 (RS1) = 0, RS 端口选择 R0; K10 (RD0) = 1, K11 (RD1) = 0, RD 端口选择 R1; K12 (WR0) = 0, K13 (WR1) = 1, WR 端口选择 R2。按一次 QD 按钮, 这时 ER 中的数据 (35H) 写入了 R2, 同时 R1 中的数据 (55H) 写入了 DR2, R0 中的数据送数据总线 DBUS (其实, R0 中的数据并不受按 QD 按钮的影响)。数据指示灯应显示 R0 中的数据 0AAH, 如果令 K1 (RS_BUS#) = 1, K2 (ALU_BUS) = 1, 数据指示灯应显示 DR2 的内容 (即 R1 的内容), 应为 55H。再令 K1 (RS_BUS#) = 0, K2 (ALU_BUS) = 0, 使数据指示灯仍显示 RS 端口的内容, 同时令 K8 (RS0) = 0, K9 (RS1) = 1, RS 端口选择 R2,

数据指示灯应显示新的 R2 的值, 即 35H。

2. 保持 RS 端口 (B 端口) 和 WR 端口同时对应 R2, WRD = 1, 而 ER 中置入新的数据, 观察并行输入输出的结果, RS 端口输出的是旧的还是新的数据?

令 K2 (ALU_BUS) = 0, K3 (CEL#) = 1, K4 (LRW) = 1, K5 (LDAR1) = 0, K6 (LDDR2) = 0, K10 (RD0) = 0, K11 (RD1) = 0。

将开关 IR/DBUS 至于 DBUS 位置。令 K0 (SW_BUS#) = 0, K1 (RS_BUS#) = 1, K7 (LDER) = 1, K14 (WRD) = 0。将 SW7—SW0 置为 53H, 按一次 QD 按钮, 将 53H 写入暂存寄存器 ER。令 K0 (SW_BUS#) = 1, K1 (RS_BUS#) = 0, K8 (RS0) = 0, K9 (RS1) = 1, RS 端口选择 R2, 数据指示灯显示 R2 的值, 应为 35H。令 K14 (WRD) = 1, K12 (WR0) = 0, K13 (WR1) = 1, WR 端口选择 R2, 允许写操作。当按下 QD 按钮时, 新的值 53H 从 ER 写入 R2, 数据指示灯立即变为 53H。

第五节 微程序控制器组成实验

- 一、实验目的
- (1) 掌握时序产生器的组成原理。
 - (2) 掌握微程序控制器的组成原理。
 - (3) 掌握微指令格式的化简和归并。

二、实验电路

1. 时序发生器

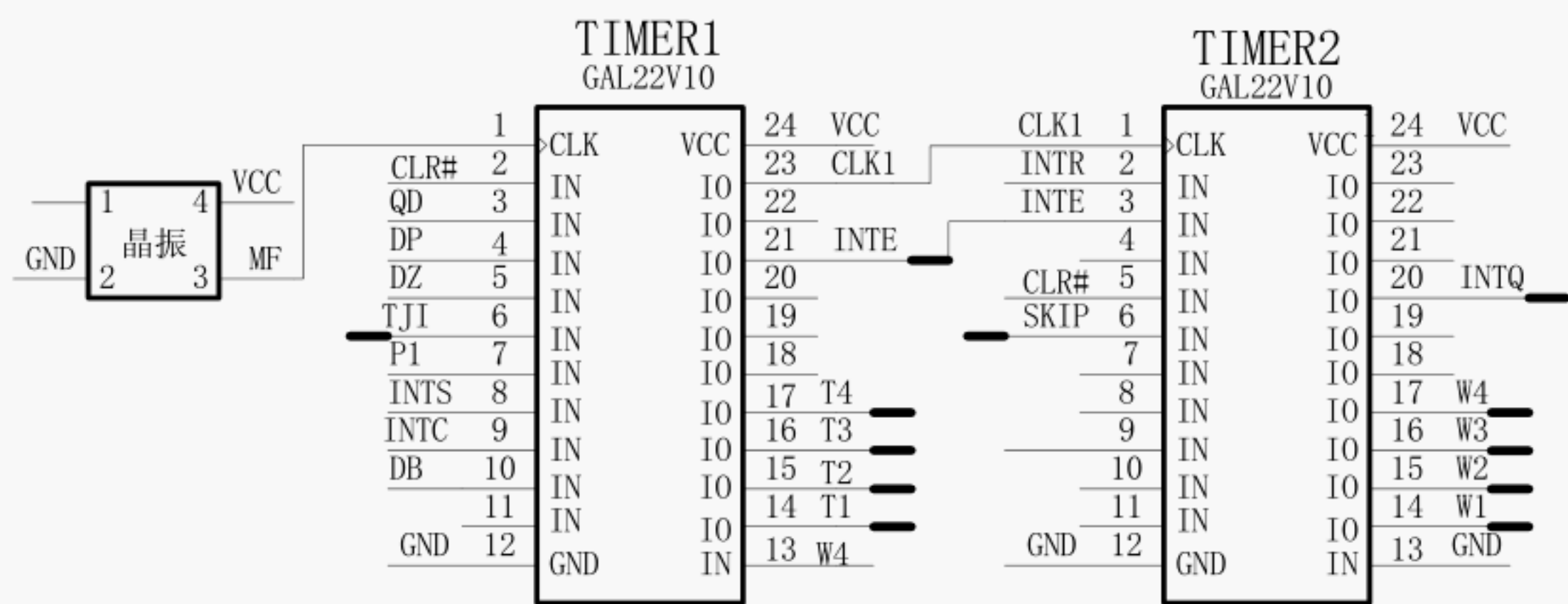


图9 时序信号发生器图

TEC—4 计算机组成原理实验系统的时序电路如图 9 所示。电路采用 2 片 GAL22V10 (U6、U7)，可产生两级等间隔时序信号 T1—T4 和 W1—W4，其中一个 W 由一轮 T1—T4 循环组成，它相当于一个微指令周期或硬布线控制器的一拍，而一轮 W1—W4 循环可供硬布线控制器执行一条机器指令。

本次实验不涉及硬布线控制器，因此时序发生器中的相关内容也可根据需要放到硬布线控制器实验中进行。

微程序控制器只使用时序信号 T1—T4，产生 T 信号的功能集成在 GAL22V10 芯片 TIMER1 (U6) 中，另外它还产生节拍信号 W1、W2、W3、W4 的控制时钟 CLK1。该芯片的逻辑功能用 ABEL 语言实现，其源程序如下：

```
MODULE TIMER1
TITLE 'CLOCK GGENERATOR T1, T2, T3, T4'
"2000, 9

"INPUT
MF, CLR, QD, DP, DZ, TJ, P1, INTS, INTC, DB PIN 1.. 10;
W4 PIN 13;

"OUTPUT
```



```

T1, T2, T3, T4, QD1, QD2, ACT, INTE, QDR  PIN 14..22;
CLK1  PIN 23;
T1, T2, T3, T4, QD1, QD2, INTE, QDR  ISTYPE 'REG' ;
ACT, CLK1  ISTYPE 'COM' ;
CLK = .C. ;
S =[T1, T2, T3, T4, QD1, QD2, INTE, QDR] ;

```

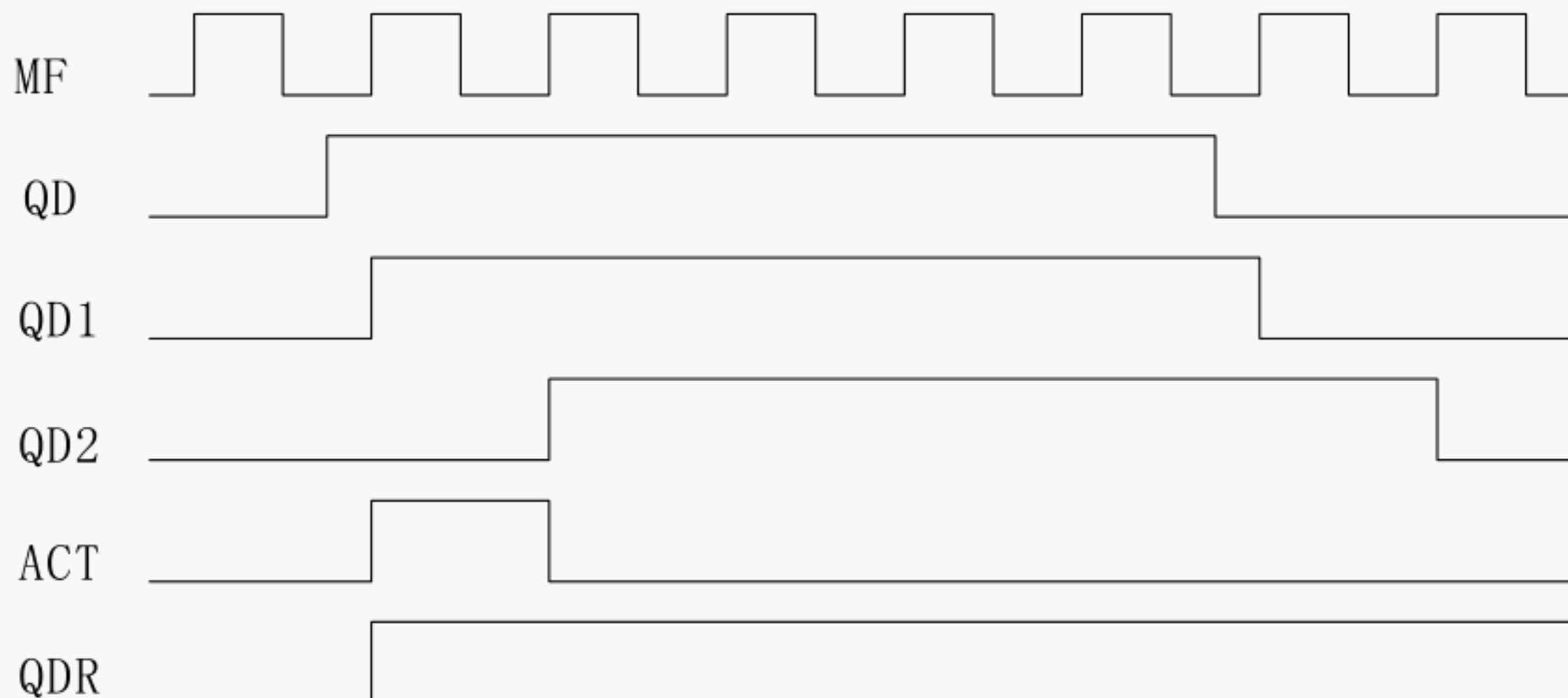
EQUATIONS

```

QD1 := QD;
QD2 := QD1;
QDR := CLR & QD # CLR & QDR;
ACT = QD1 & !QD2;
T1 := CLR & T4 & ACT # CLR & T4 & !(DP # TJ # DZ & P1 # DB & W4) & QDR;
T2 := CLR & T1;
T3 := CLR & T2;
T4 := !CLR # T3 # T4 & !ACT & (DP # TJ # DZ & P1 # DB & W4) # !QDR;
INTE := CLR & INTS # CLR & INTE & !INTC;
CLK1 = T1 # !CLR & MF;
S.CLK = MF;

```

END



硬布线控制器只使用时序信号 W1—W4，产生 W 信号的功能集成在 GAL22V10 芯片 TIMER2 (U7) 中，该芯片的逻辑功能用 ABEL 语言实现，其源程序如下：

```

MODULE TIMER2
TITLE 'CLOCK GGENERATOR W1 W2 W3 W4'
" 2000, 9

"INPUT
CLK1, INTR, IE  PIN 1..3;

```

```
CLR, SKIP PIN 5..6;
```

```
"OUTPUT
```

```
W1, W2, W3, W4, INTR1 PIN 14..18 ISTYPE 'REG' ;
```

```
INTQ PIN 20 ISTYPE 'COM' ;
```

```
CLK = .C. ;
```

```
EQUATIONS
```

```
W1 := CLR & W4 ;
```

```
W2 := CLR & W1 & !SKIP;
```

```
W3 := CLR & W2 & !SKIP;
```

```
W4 := !CLR # W3 # SKIP & W1 # SKIP & W2 ;
```

```
INTR1 := INTR;
```

```
INTQ = IE & INTR1;
```

```
[W1, W2, W3, W4, INTR1].CLK = CLK1;
```

```
END
```

TIMER1 和 TIMER2 中还集成了中断逻辑，中断逻辑的介绍见第八节。TIMER1 的输入信号中，MF 接实验台上晶体振荡器的输出，频率为 1MHz。T1 至 T4 的脉冲宽度为 100ns。CLR（**注意，实际上是控制台上的 CLR#信号，因为 ABEL 语言的书写关系改为 CLR，仍为低有效信号**）为复位信号，低有效。实验台处于任何状态下令 $CLR\# = 0$ ，都会使时序发生器和微程序控制器复位（回到初始状态）， $CLR\# = 1$ 时，则可以正常运行。复位后时序发生器停在 T4、W4 状态，微程序地址为 000000B。建议每次实验仪加电后，先用 CLR#复位一次。控制台上有一个 CLR#按钮，按一次，产生一个 CLR#负脉冲，实验台印制板上已连好控制台 CLR#到时序电路 CLR 的连线。

TJ（停机）是控制器的输出信号之一。连续运行时，如果控制信号 $TJ = 1$ ，会使机器停机，停止发送时序脉冲 T1—T4、W1—W4，时序停在 T4。在实验台上为了将时序信号发生器的输入信号 TJ 和控制存储器产生的 TJ 信号区分开来，以便于连线操作，在实验台上时序信号发生器的输入信号 TJ 命名为 TJ1，而控制存储器产生的信号 TJ 仍命名为 TJ。QD（启动）是来自启动按钮 QD 的脉冲信号，在 TIMER1 中，对 QD 用 MF 进行了同步，产生 QD1 和 QD2。ACT 表示 QD1 上升沿，表达式是 $QD1 \& !QD2$ ，脉冲宽度为 1000ns。QDR 是运行标志，QD 信号使其为 1，CLR 信号将其置 0。DP（单拍）是来自控制台的 DP 开关信号，当 $DP = 1$ 时，机器处于单拍运行状态，按一次启动按钮 QD，只发送一条微指令周期的时序信号就停机。利用单拍方式，每次只执行一条微指令，因而可以观察微指令代码和当前微指令的执行结果。DZ（单指）信号是针对微程序控制器的，接控制台开关 DZ 和 P1 信号配合使用。P1 是微指令字判断字段中的一个条件信号，从微程序控制器输出。P1 信号在微程序中每条机器指令执行结束时为 1，用于检测有无中断请求 INTQ，而时序发生器用它来实现单条机器指令停机。在 $DB = 0$ 且 $DP = 0$ 的前提下，当 $DZ = 0$ 时，机器连续运行。当 $DZ = 1$ 时，机器处于单指方式，每次只执行一条机器指令。

DB、SKIP、CLK1 信号以及 W1—W4 时序信号都是针对硬布线控制器的。W1—W4 是节拍信号，硬布线控制器执行一条机器指令需要一组 W1—W4 信号。DB（单步）信号就是每次发送一组 W 信号后停机，可见其功能与 DZ 类似。执行某些机器指令不需要完整的一组 W 信号周期，SKIP 信号就是用来跳过本指令剩余的 W 节拍信号的。中断允许标志 IE 由控制存储器的输出信号 INTS 将其置 1，由控制

存储器的输出信号 INTC 将其置 0。在 TIMER2 内部，控制台产生的中断请求被用时钟 CLK1 进行同步，产生了 INTR1。只有在 INTE = 1 时，控制台产生的中断请求脉冲 INTR 才能起作用，即产生向控制器输出的中断中断信号 INTQ， $INTQ = INTE \& INTR1$ 。

2．数据通路

微程序控制器是根据数据通路和指令系统来设计的。这里采用的数据通路是在综合前面各实验模块的基础上，又增加程序计数器 PC（U18）、地址加法器 ALU2（U17）、地址缓冲寄存器 R4（U25、U26）和中断地址寄存器 IAR（U19），详见第二节的图 4。PC 和 ALU2 各采用一片 GAL22V10，两者配合使用，可完成程序地址的存储、增 1 和加偏移量的功能。R4 由两片 74HC298 组成，带二选一输入端。IAR 是一片 74HC374，用于中断时保存断点地址。有关数据通路总体的详细说明，请参看第一节。

3．微指令格式与微程序控制器电路

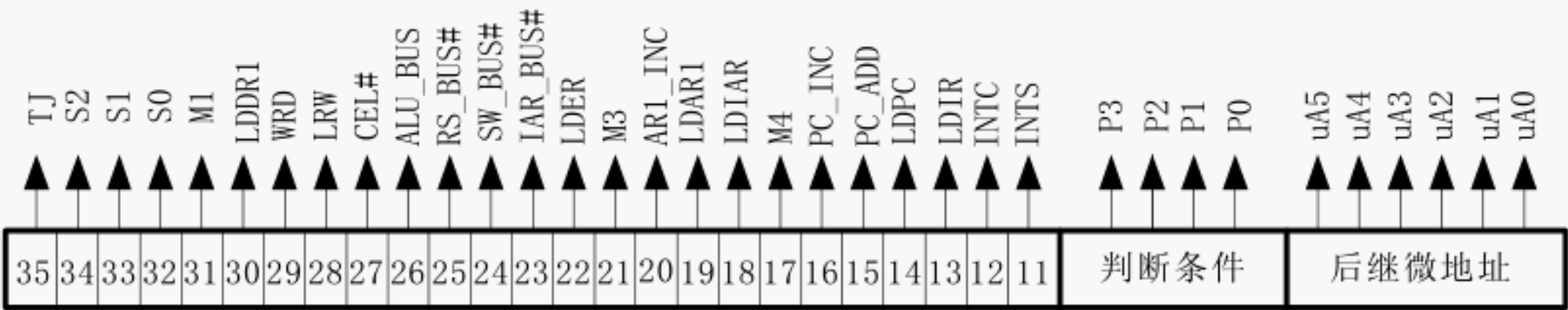


图10 微指令格式

根据给定的 12 条机器指令功能和数据通路总体图的控制信号，采用的微指令格式见图 10。微指令字长共 35 位。其中顺序控制部分 10 位：后继微地址 6 位，判别字段 4 位，操作控制字段 25 位，各位进行直接控制。微指令格式中，信号名带有后缀“#”的信号为低有效信号，不带有后缀“#”的信号为高有效信号。

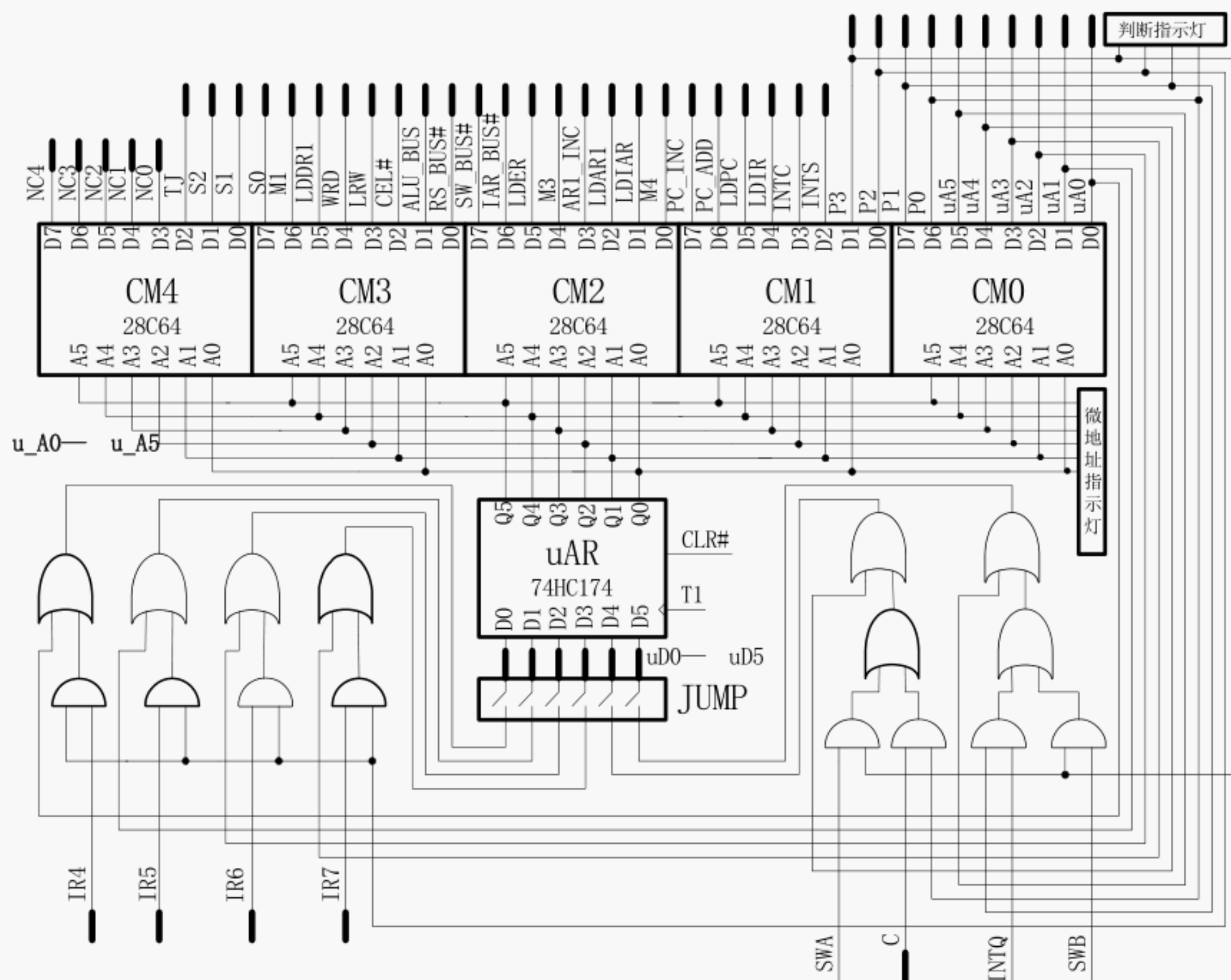


图11 微程序控制器的组成

对应微指令格式，微程序控制器的组成见图 11。控制存储器采用 5 片 EEPROM 28C64 (U8、U9、U10、U11、U12)。28C64 的输出是 D0—D7，分别与引脚 11、12、13、15、16、17、18、19 相对应，CM0 是最低字节，CM4 是最高字节。微地址寄存器 6 位，用一片 6D 触发器 74HC174 (U1) 组成，带有清零端。两级与门、或门构成微地址转移逻辑，用于产生下一微指令的地址。在每个 T1 上升沿时刻，新的微指令地址会打入微地址寄存器中，控制存储器随即输出相应的微命令代码。微地址转移逻辑生成下一地址，等下一个 T1 上升沿时打入微地址寄存器。跳转开关 JUMP (J1) 是一组 6 个跳线开关。当用短路子将它们连通时，微地址寄存器 μ AR 从本实验系统提供的微程序地址译码电路得到新的微程序地址 μ D0— μ D5。当他们被断开时，用户提供自己的新微程序地址 μ D0— μ D5。这样用户能够使用自己设计的微程序地址译码电路。5 片 EEPROM 的地址 A6 (引脚 4) 直接与控制台开关 SWC 连接，当 SWC = 1 时，微地址大于或者等于 40H，当 SWC = 0 时，微地址的范围为 00H—3FH。SWC 主要用于实现读寄存器堆的功能。

微地址转移逻辑的多个输入信号中，INTQ 是中断请求，本实验中可以不理睬。SWA、SWB 是控制台的两个二进制开关信号，实验台上线已接好接。C 是进位信号，IR7—IR4 是机器指令代码，由于本次实验不连接数据通路，这些信号都接到二进制开关 K0—K15 上。

三、机器指令与微程序

为教学中简单明了，本实验仪使用 12 条机器指令，均为单字长 (8 位) 指令。指令

功能及格式如表 5 所示。指令的高 4 位提供给微程序控制器，低 4 位提供给数据通路。

表 5 指令功能与格式

名称	助记符	功能	指令格式							
			R7	R6	R5	R4	R3	R2	R1	R0
加法	ADD Rd, Rs	$Rd+Rs \rightarrow Rd$	0	0	0	0	RS1	RS0	RD1	RD0
减法	SUB Rd, Rs	$Rd-Rs \rightarrow Rd$	0	0	0	1	RS1	RS0	RD1	RD0
乘法	MUL Rd, Rs	$Rd*Rs \rightarrow Rd$	0	0	1	0	RS1	RS0	RD1	RD0
逻辑与	AND Rd, Rs	$Rd \& Rs \rightarrow Rd$	0	0	1	1	RS1	RS0	RD1	RD0
存数	STA Rd, [Rs]	$Rd \rightarrow [Rs]$	0	1	0	0	RS1	RS0	RD1	RD0
取数	LDA Rd, [Rs]	$[Rs] \rightarrow Rd$	0	1	0	1	RS1	RS0	RD1	RD0
无条件转移	JMP [Rs]	$[Rs] \rightarrow PC$	1	0	0	0	RS1	RS0	X	X
条件转移	JC D	若 $C=1$ 则 $PC+D \rightarrow PC$	1	0	0	1	D3	D2	D1	D0
停机	STP	暂停运行	0	1	1	0	X	X	X	X
中断返回	IRET	返回断点	1	0	1	0	X	X	X	X
开中断	INTS	允许中断	1	0	1	1	X	X	X	X
关中断	INTC	禁止中断	1	1	0	0	X	X	X	X

应当指出，用以上 12 条指令来编写实际程序是不够的。好在我们的目的不是程序设计，而主要是为了教学目的，通过 CPU 执行一些最简单的程序来掌握微程序控制器的工作原理。

上述 12 条指令的微程序流程设计如图 12 所示。每条微指令可按前述的微指令格式转换成二进制代码，然后写入 5 个 28C64 中。

为了向 RAM 中装入程序和数据，检查写入是否正确，并能启动程序执行，还设计了以下五个控制台操作微程序：

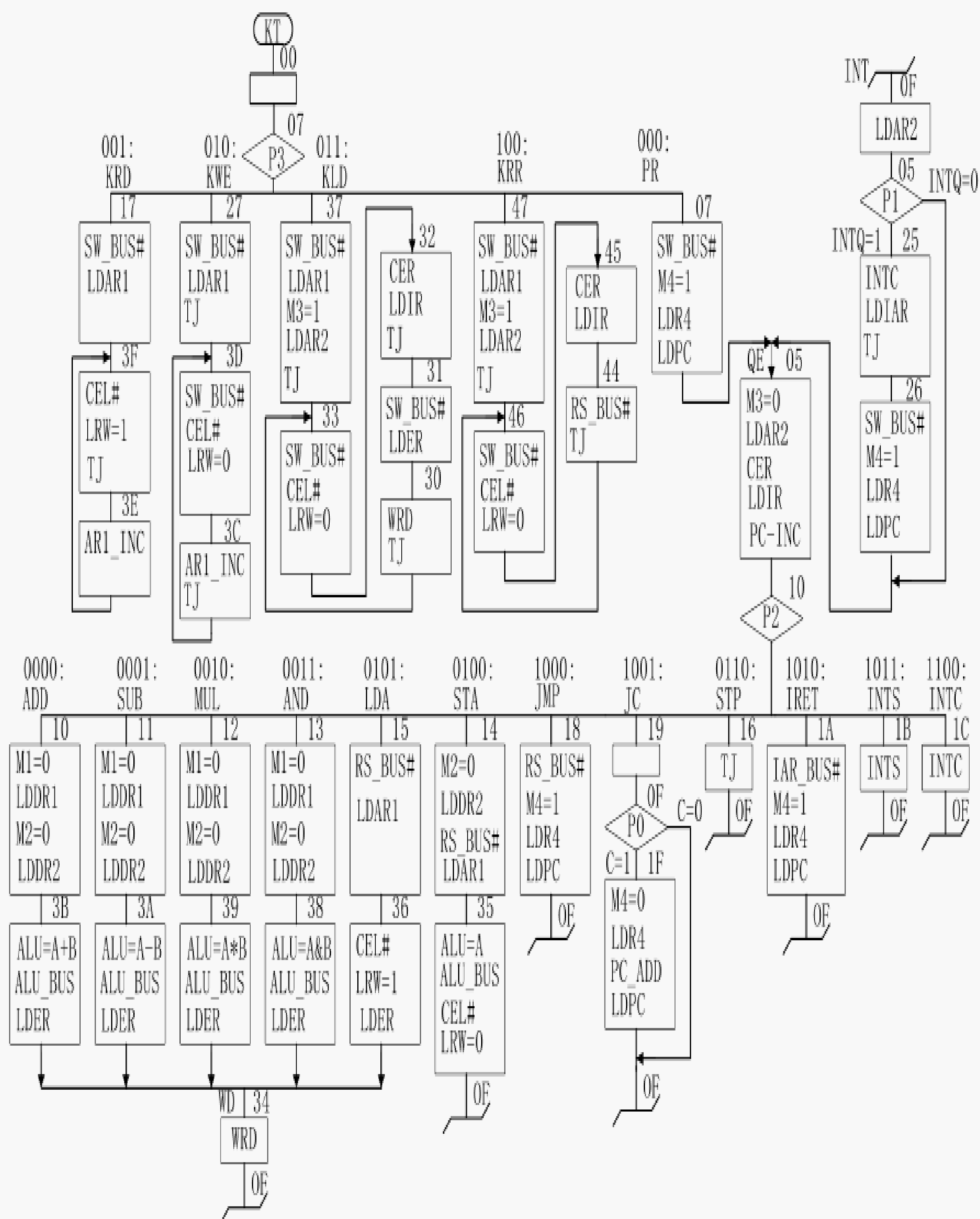


图12 微程序流程图

存储器写操作 (KWE): 按下复位按钮 CLR#后, 微地址寄存器状态为全零。此时置 $SWC = 0$ 、 $SWB = 1$, $SWA = 0$, 按启动按钮后微指令地址转入 27H, 从而可对 RAM 连续进行手动写入。

存储器读操作 (KRD): 按下复位按钮 CLR#后, 置 $SWC = 0$, $SWB = 0$, $SWA = 1$, 按启动按钮后微指令地址转入 17H, 从而可对 RAM 连续进行读操作。

写寄存器操作 (KLD): 按下复位按钮 CLR#后, 置 $SWC = 0$, $SWB = 1$, $SWA = 1$, 按启动按钮后微指令地址转入 37H, 从而可对寄存器堆中的寄存器连续进行写操作。

读寄存器操作 (KRR): 按下复位按钮 CLR#后, 置 $SWC = 1$, $SWB = 0$, $SWA = 0$, 按启动按钮后微指令地址转入 47H, 从而可对寄存器堆中的寄存器连续进行读操作。

启动程序 (PR): 按下复位按钮 CLR#后, 置 $SWC = 0$, $SWB = 0$, $SWA = A$, 用数据开关 SW7—SW0 设置内存中程序的首地址, 按启动按钮后微指令地址转入 07H, 然后转到“取指”微指令。

应当着重指出, 在微指令格式的设计过程中, 对数据通路所需的控制信号进行了归并和化简。细心的同学可能已经发现, 微程序控制器输出的控制信号远远少于数据通路所需的控制信号。这里提供的微程序流程图, 是没有经过归并和化简的。仔细研究一下微程序流程图, 就会发现有些信号的出现的位置完全一样, 这样的信号用其中一个信号就可以代表。请看信号 LDPC 和 LDR4, 这两个信号都在微程序地址 07H、1AH、1FH、26H 出现, 而在其他的微程序地址都不出现, 因此这两个信号产生的逻辑条件是完全一样的。从逻辑意义上看, 这两个信号的作用是产生新的 PC, 完全出现在相同的微指令中是很正常的, 因此用 LDPC 完全可以代替 LDR4。还有另一些信号, 例如 LDDR1 和 LDDR2, 出现的位置基本相同。LDDR2 和 LDDR1 的唯一不同是在地址 14H 的微指令中, 出现了 LDDR2 信号, 但是没有出现 LDDR1 信号。LDDR1 和 LDDR2 是否也可以归并成一个信号呢? 答案是肯定的。微程序流程图中只是指出了在微指令中必须出现的信号, 并没有指出出现其他信号行不行, 这就要根据具体情况具体分析。在地址 14H 的微指令中, 出现 LDDR1 信号行不行呢? 完全可以。在地址 14H 出现的 LDDR1 是一个**无用的**信号, 同时也是一个**无害的**信号, 它的出现完全没有副作用, 因此 LDDR1 和 LDDR2 可以归并为一个信号 LDDR1。根据以上两条原则, 我们对下列信号进行了归并和化简:

LDIR (CER)	为 1 时, 允许对 IR 加载, 此信号也可用于作为双端口存储器右端口选择 CER。
LDPC (LDR4)	为 1 时, 允许对程序计数器 PC 加载, 此信号也可用于作为 R4 的加载允许信号 LDR4。
LDAR1 (LDAR2)	为 1 时, 允许对地址寄存器 AR1 加载, 此信号也可用于作为对地址寄存器 AR2 加载。
LDDR1 (LDDR2)	为 1 时允许对操作数寄存器 DR1 加载。此信号也可用于作为对操作数寄存器 DR2 加载。
M1 (M2)	当 $M1 = 1$ 时, 操作数寄存器 DR1 从数据总线 DBUS 接收数据; 当 $M1 = 0$ 时, 操作数寄存器 DR1 从寄存器堆 RF 接收数据。此信号也可用于作为操作数寄存器 DR2 的数据来源选择信号。

在对微指令格式进行归并和化简的过程中, 我们有意保留了一些信号, 没有化简, 同学们可以充分发挥创造性, 提出更为简单的微指令格式。

还要说明的是, 为什么微指令格式可以化简, 实验台数据通路的控制信号为什么不进行化简? 最主要的原因是前面进行的各个实验的需要, 例如 LDDR1 和 LDDR2 这两个信号, 在做运算器数据通路实验时, 是不能设计成一个信号的。还有一个原因是考虑到实验时易于理解, 对某些可以归并的信号也没有予以归并。

四、实验设备

- (1) TEC—4 计算机组成原理实验系统一台
- (2) 双踪示波器一台
- (2) 直流万用表一只
- (3) 逻辑测试笔一支

五、实验任务

- (1) 按实验要求连接实验台的数码开关 K0—K15、按钮开关、时钟信号源和微程序控制器。

注意：本次实验只做微程序控制器本身的实验，故微程序控制器输出的微命令信号与执行部件（数据通路）的连线暂不连接。连线完成后应仔细检查一遍，然后才可加上电源。

- (2) 观察时序信号。

用双踪示波器观测时序产生器的输入输出信号：MF，W1—W4，T1—T4。比较相位关系，画出其波形，并标注测量所得的脉冲宽度。观察时须将 TJ1 接低电平，DB、DZ、DP 开关均置为 0 状态，然后按 QD 按钮，则连续产生 T1、T2、T3、T4、W1、W2、W3、W4。

了解启停控制信号的功能，并熟练地使用连接这些控制信号的按钮或开关。

- (3) 熟习微指令格式的定义，按此定义将控制台指令微程序的 8 条微指令按十六进制编码，列于下表。三种控制台指令的功能由 SWC、SWB、SWA 三个二进制开关的状态来指定（KRD = 001B，KWE = 010B，PR = 000B）。此表必须在预习时完成。

微指令地址	微指令编码	微指令地址	微指令编码
00H		3CH	
07H		17H	
27H		3FH	
3DH		3EH	

单拍（DP）方式执行控制台微程序，读出上述八条微指令，用 P 字段和微地址指示灯跟踪微指令执行情况，并与上表数据对照。用连续方式执行 KWE 和 KRD（将 TJ1 接地），画出 u_A0（28C64 的地址 A0，U12 的引脚 10）信号波形，作出解释。

单拍（DP）方式执行控制台微程序，读出上述八条微指令，用 P 字段和微地址指示灯跟踪微指令执行情况，并与上表数据对照。用连续方式执行 KWE 和 KRD（将 TJ1 接地），画出 u_A0（28C64 的地址 A0，U12 的引脚 10）信号波形，作出解释。

- (4) 用 P3 和 SWC、SWB、SWA 的状态组合，观察验证三种控制台指令 KRD、KWE、PR 微地址转移逻辑功能的实现。

- (5) 熟习 05H、10H 两条微指令的功能和 P2 测试的状态条件（IR4—IR7），用二进制开关设置 IR7—IR4 的不同状态，观察 ADD 至 STP 九条机器指令微地址转移逻辑功能的实现。（用逻辑笔测试有关逻辑电路的电平，分别做出测试记录，自行设计表格。）

- (6) 设置 IR7—IR4 的不同组合，用单拍方式执行 ADD 至 STP 九条机器指令微程序，用微地址和 P 字段指示灯跟踪微程序转移和执行情况。用逻辑笔测试小插座上输出的微命令信号，记录 ADD、SUB、LDA、STA 四条机器指令的微命令信号，自行设计表格。

六、实验步骤和实验结果

- (1) 接线

跳线开关 J1 用短路子短接。控制器的输入 C 接 K0, IR4 接 K1, IR5 接 K2, IR6 接 K3, IR7 接 K4, TJI 接 K5, SKIP 接 GND。

合上电源。按 CLR#按钮, 使实验系统处于初始状态。

(2) 观察时序信号波形

令 K5 (TJI) = 0, 置 DP = 0, DZ = 0, DB = 0。按 QD 按钮, 则时序部分开始不停止的运行, 直到按 CLR#按钮为止。用双踪示波器观察 MF、T1、T2、T3、T4、W1、W2、W3、W4 信号。观察的方法是同时观察两路信号, 以便比较相位。可按下述顺序进行: MF 和 T1, T1 和 T2, T2 和 T3, T3 和 T4, T1 和 W1, W1 和 W2, W2 和 W3, W3 和 W4。根据观察的结果, 可画出波形图。波形图的图形请参看第一节图 3 的基本时序图。MF 的周期为 1000 毫秒, 占空比为 50%。

(3) 控制台操作微指令编码测试和 KWRD、KRD 连续运行时的 μ A0 波形

1. 控制台操作微指令编码测试

控制台微代码在 28C64 的 D0—D7 输出, D0 是最低位, D7 是最高位, CM0 是最低字节, CM4 是最高字节。D0—D7 对应于 28C64 的引脚 11、12、13、15、16、17、18、19。

令 K5 (TJI) = 0, 置 DP = 1, DZ = 0, DB = 0, 使实验系统处于单拍状态。置 SWC = 0、SWB = 0、SWA = 1, 实验系统处于 KRD 工作模式。按 CLR#按钮, 使实验系统处于初始状态, 微地址 μ A5— μ A0 为 00H, 测得的微码是 00H 05H 0C0H 02H 07H。按一次 QD 按钮, 微地址 μ A5— μ A0 为 17H, 测得的微码是 04H 05H 44H 00H 3FH。按一次 QD 按钮, 微地址 μ A5— μ A0 为 3FH, 测得的微码是 04H 09H 0C0H 00H 3EH。按一次 QD 按钮, 微地址 μ A5— μ A0 为 3EH, 测得的微码是 00H 05H 0C8H 00H 3FH。按一次 QD 按钮, 微地址 μ A5— μ A0 为 3FH。由于微地址又返回 3FH, 停止测试。

令 K5 (TJI) = 0, 置 DP = 1, DZ = 0, DB = 0, 使实验系统处于单拍状态。置 SWC = 0、SWB = 1、SWA = 0, 实验系统处于 KWE 工作模式。按 CLR#按钮, 使实验系统处于初始状态, 微地址 μ A5— μ A0 为 00H。按一次 QD 按钮, 微地址 μ A5— μ A0 为 27H, 测得的微码是 04H 05H 44H 00H 3DH。按一次 QD 按钮, 微地址 μ A5— μ A0 为 3DH, 测得的微码是 00H 01H 40H 00H 3CH。按一次 QD 按钮, 微地址 μ A5— μ A0 为 3CH, 测得的微码是 04H 05H 0C8H 00H 3DH。按一次 QD 按钮, 微地址 μ A5— μ A0 为 3CH。由于微地址又返回 3CH, 停止测试。

令 K5 (TJI) = 0, 置 DP = 1, DZ = 0, DB = 0, 使实验系统处于单拍状态。置 SWC = 0、SWB = 0、SWA = 0, 实验系统处于 PR 工作模式。按 CLR#按钮, 使实验系统处于初始状态, 微地址 μ A5— μ A0 为 00H。按一次 QD 按钮, 微地址 μ A5— μ A0 为 07H, 测得的微码是 00H 05H 41H 20H 05H。按一次 QD 按钮, 微地址 μ A5— μ A0 为 05H, 测得的微码是 00H 05H 0C4H 91H 10H。由于以后的微码与机器指令有关, 停止测试。

2. KWE、KRD 连续运行时的 μ A0 波形

令 K5 (TJI) = 0, 置 DP = 0, DZ = 0, DB = 0, 使实验系统处于连续状态。置 SWC = 0、SWB = 1、SWA = 0, 实验系统处于 KWE 工作模式。按 CLR#按钮, 使实验系统处于初始状态。按一次 QD 按钮, 使系统连续运行。用示波器测量 μ A0 (U12 的引脚 11) 的波形, 得到波形如下:



虽然完成 KWE 控制台功能需要 27H、3DH、3CH 3 条微指令, 但是在连续运行时, 循环执行的只

是 3DH、3CH 2 条微指令，因此 μ_A0 的波形如上。

令 K5 (TJI) = 0，置 DP = 0，DZ = 0，DB = 0，使实验系统处于连续状态。置 SWC = 0、SWB = 0、SWA = 1，实验系统处于 KRD 工作模式。按 CLR#按钮，使实验系统处于初始状态。按一次 QD 按钮，使系统连续运行。用示波器测量 μ_A0 (U12 的引脚 11) 的波形，得到波形与 KWE 时的波形相同，原因也类似。

(4) 用 P3 和 SWB、SWA 的状态组合，观察验证三种控制台指令 KWE、KRD、PR 微地址转移逻辑功能的实现。

其实，第(3)步已验证了验证三种控制台指令 KWE、KRD、PR 微地址转移逻辑功能的实现。在这里，可将时序电路的输入 TJI 与控制存储器的输出 TJ 连接，置 DP = 1，DB = 0，DZ = 0。选择 SWC = 0、SWB = 0、SWA = 1，按 QD 按钮，验证 KRD 的微地址转移功能；选择 SWC = 0、SWB = 1、SWA = 0，按 QD 按钮，验证 KWE 的微地址转移功能；选择 SWC = 0、SWB = 0、SWA = 1，按 QD 按钮，验证 KRD 的微地址转移功能。这里不再详述。

(5) 熟习 05H、10H 两条微指令的功能和 P2 测试的状态条件 (IR4—IR7)，用二进制开关设置 IR7—IR4 的不同状态，观察 ADD 至 STP 九条机器指令微地址转移逻辑功能的实现。

1. 05H 微指令的功能是根据程序计数器 PC 从存储器取指令，送往指令寄存器 IR，同时进行 PC+1 的操作。05H 微指令的下一微指令地址是 10H。不过，10H 只是一个表面的下一微地址，由于该微指令中 P2 = 1，因此实际的微指令地址的低 4 位要根据 IR7—IR4 确定，实际微地址为 10H + IR7 IR6 IR5 IR4。
2. 置 DP = 1，DZ = 0，DB = 0，使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0，按 CLR#按钮，使实验系统处于初始状态，微地址是 00H。令 K4 (IR7) = 0，K3 (IR6) = 0，K2 (IR5) = 0，K1 (IR4) = 0，相当于 ADD 指令的操作码。按一次 QD 按钮，微地址变为 07H。按一次 QD 按钮，微地址变为 05H。按一次 QD 按钮，微地址变为 10H。
3. 置 DP = 1，DZ = 0，DB = 0，使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0，按 CLR#按钮，使实验系统处于初始状态，微地址是 00H。令 K4 (IR7) = 0，K3 (IR6) = 0，K2 (IR5) = 0，K1 (IR4) = 1，相当于 SUB 指令的操作码。按一次 QD 按钮，微地址变为 07H。按一次 QD 按钮，微地址变为 05H。按一次 QD 按钮，微地址变为 11H。
4. 置 DP = 1，DZ = 0，DB = 0，使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0，按 CLR#按钮，使实验系统处于初始状态，微地址是 00H。令 K4 (IR7) = 0，K3 (IR6) = 0，K2 (IR5) = 1，K1 (IR4) = 0，相当于 MUL 指令的操作码。按一次 QD 按钮，微地址变为 07H。按一次 QD 按钮，微地址变为 05H。按一次 QD 按钮，微地址变为 12H。
5. 置 DP = 1，DZ = 0，DB = 0，使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0，按 CLR#按钮，使实验系统处于初始状态，微地址是 00H。令 K4 (IR7) = 0，K3 (IR6) = 0，K2 (IR5) = 1，K1 (IR4) = 1，相当于 AND 指令的操作码。按一次 QD 按钮，微地址变为 07H。按一次 QD 按钮，微地址变为 05H。按一次 QD 按钮，微地址变为 13H。
6. 置 DP = 1，DZ = 0，DB = 0，使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0，按 CLR#按钮，使实验系统处于初始状态，微地址是 00H。令 K4 (IR7) = 0，K3 (IR6) = 1，K2 (IR5) = 0，K1 (IR4) = 0，相当于 STA 指令的操作码。按一次 QD 按钮，微地址变为 07H。按一次 QD 按钮，微地址变为 05H。按一次 QD 按钮，微地址变为 14H。
7. 置 DP = 1，DZ = 0，DB = 0，使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0，按 CLR#按钮，使实验系统处于初始状态，微地址是 00H。令 K4 (IR7) = 0，K3 (IR6) = 1，K2 (IR5)

= 0, K1 (IR4) = 1, 相当于 LDA 指令的操作码。按一次 QD 按钮, 微地址变为 07H。按一次 QD 按钮, 微地址变为 05H。按一次 QD 按钮, 微地址变为 15H。

8. 置 DP = 1, DZ = 0, DB = 0, 使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0, 按 CLR#按钮, 使实验系统处于初始状态, 微地址是 00H。令 K4 (IR7) = 0, K3 (IR6) = 1, K2 (IR5) = 1, K1 (IR4) = 0, 相当于 STP 指令的操作码。按一次 QD 按钮, 微地址变为 07H。按一次 QD 按钮, 微地址变为 05H。按一次 QD 按钮, 微地址变为 16H。
9. 置 DP = 1, DZ = 0, DB = 0, 使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0, 按 CLR#按钮, 使实验系统处于初始状态, 微地址是 00H。令 K4 (IR7) = 1, K3 (IR6) = 0, K2 (IR5) = 0, K1 (IR4) = 0, 相当于 JMP 指令的操作码。按一次 QD 按钮, 微地址变为 07H。按一次 QD 按钮, 微地址变为 05H。按一次 QD 按钮, 微地址变为 18H。
10. 置 DP = 1, DZ = 0, DB = 0, 使实验系统处于单拍状态。选择 SWC = 0、SWB = 0、SWA = 0, 按 CLR#按钮, 使实验系统处于初始状态, 微地址是 00H。令 K4 (IR7) = 1, K3 (IR6) = 0, K2 (IR5) = 0, K1 (IR4) = 1, 相当于 JC 指令的操作码。按一次 QD 按钮, 微地址变为 07H。按一次 QD 按钮, 微地址变为 05H。按一次 QD 按钮, 微地址变为 19H。

(6) 设置 IR7—IR4 的不同组合, 用单拍方式执行 ADD 至 STP 九条机器指令微程序, 记录 ADD、SUB、LDA、STA 四条机器指令的微命令信号。

操作的方法与 (5) 类似, 只是需纪录 ADD、SUB、LDA、STA 四条机器指令的微命令信号。不再详述。

(7) 提出自己的微程序格式方案

本内容可由学生自己提出。这里仅对信号的进一步归并提一点参考意见。

1. 从微程序格式中去掉 M1 (M2) 信号。在微程序流程图中, 只有 M1 (M2) = 0, 没有 M1 (M2) = 1, 因此只要使 M1 固定接 GND, M2 固定接 GND, 就可以从微程序格式中去掉 M1 (M2) 信号。
2. 从微程序格式中去掉 LDER 信号。暂存寄存器 ER 的唯一作用是作为寄存器堆的数据来源。因此, 可将 LDER 固定接 VCC。虽然每条微指令中都有 LDER 出现, 有可能改变 ER 的值, 但是只有在下一条微指令中含有 WRD 信号时 LDER 信号才起作用。观察微程序流程图可以发现, 每条含有 WRD 信号的微指令前面的微指令中均含有 LDER 微指令, 因此可以从微程序格式中去掉 LDER 信号。
3. 从微程序格式中去掉 PC_INC 信号, 将它和 CER、LDIR 归并为一个 LDIR 信号。

第六节 CPU 组成与机器指令执行实验

一、实验目的

- (1) 将微程序控制器同执行部件（整个数据通路）联机，组成一台模型计算机；
- (2) 用微程序控制器控制模型机数据通路；
- (3) 通过 CPU 运行九条机器指令（排除中断指令）组成的简单程序，掌握机器指令与微指令的关系，牢固建立计算机的整机概念。

二、实验电路

本次实验用到前面四个实验中的所有电路，包括运算器、存储器、通用寄存器堆、程序计数器、指令寄存器、微程序控制器等，将几个模块组合成为一台简单计算机。因此，在基本实验中，这是最复杂的一个实验，也是最能得到收获的一个实验。

在前面的实验中，实验者本身作为“控制器”，完成数据通路的控制。而在本次实验中，数据通路的控制将由微程序控制器来完成。CPU 从内存取出一条机器指令到执行指令结束的一个机器指令周期，是由微指令组成的序列来完成的，即一条机器指令对应一个微程序。

三、实验设备

- (1) TEC—4 计算机组成原理实验系统一台
- (2) 双踪示波器一台
- (3) 直流万用表一只
- (4) 逻辑测试笔一支

四、实验任务

- (1) 对机器指令系统组成的简单程序进行译码。

将下表的程序按指令格式手工汇编成十六进制机器代码，此项任务应在预习时完成。

地址	指令	机器代码
00H	LDA R0, [R2]	
01H	LDA R1, [R3]	
02H	ADD R0, R1	
03H	JC +5	
04H	AND R2, R3	
05H	SUB R3, R2	
06H	STA R3, [R2]	
07H	MUL R0, R1	
08H	STP	
09H	JMP [R1]	

- (2) 按照下面框图，参考前面实验的电路图完成连线，控制器是控制部件，数据通路（包括上面各模块）是执行部件，时序产生器是时序部件。连线包括控制台、时序部分、数据通路和微程序控制器之间的连接。其中，为把操作数传送给通用寄存器组 RF，数据通路上的 RS1、RS0、RD1、RD0

应分别与 IR3 至 IR0 连接，WR1、WR0 也应接到 IR1、IR0 上。

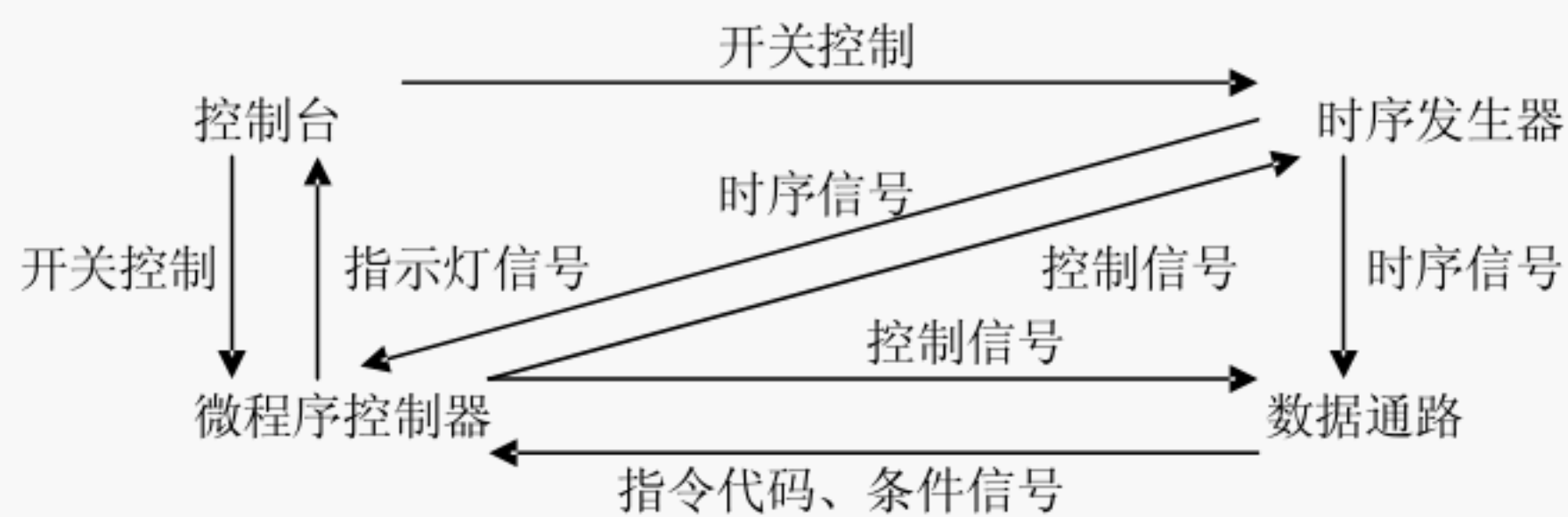


图 13 模型计算机连线示意图

(3) 将上述任务 (1) 中的程序机器代码用控制台操作存入内存中，并根据程序的需要，用数码开关 SW7—SW0 设置通用寄存器 R2、R3 及内存相关单元的数据。**注意：由于设置通用寄存器时会破坏内存单元的数据，因此一般应先设置寄存器的数据，再设置内存数据。**

(4) 用单拍 (DP) 方式执行一遍程序，列表记录通用寄存器堆 RF 中四个寄存器的数据，以及由 STA 指令存入 RAM 中的数据 (程序结束后从 RAM 的相应单元中读出)，与理论分析值作对比。单拍方式执行时注意观察微地址指示灯、IR/DBUS 指示灯、AR2 / AR1 指示灯和判断字段指示灯的值，以跟踪程序中取指令和执行指令的详细过程 (可观察到每一条微指令)。

(5) 以单指 (DZ) 方式重新执行程序一遍，注意观察 IR/DBUS 指示灯、AR2/AR1 指示灯的值 (可观察到每一条机器指令)。执行结束后，记录 RF 中四个寄存器的数据，以及由 STA 指令存入 RAM 中的数据，与理论分析值作对比。**注意：单指方式执行程序时，四个通用寄存器和 RAM 中的原始数据与第一遍执行程序的结果有关。**

(6) 以连续方式 (DB、DP、DZ 都设为 0) 再次执行程序。这种情况相当于计算机正常运行程序。由于程序中有停机指令 STP，程序执行到该指令时自动停机。执行结束后，记录 RF 中四个寄存器的数据，以及由 STA 指令存入 RAM 中的数据，与理论分析值作对比。同理，程序执行前的原始数据与第二遍执行结果有关。

五、实验步骤和实验结果

(1) 对机器指令系统组成的简单程序进行译码。

地址	指令	机器代码
00H	LDA R0, [R2]	58H
01H	LDA R1, [R3]	5DH
02H	ADD R0, R1	04H
03H	JC +5	95H
04H	AND R2, R3	3EH
05H	SUB R3, R2	1BH
06H	STA R3, [R2]	4BH
07H	MUL R0, R1	24H
08H	STP	60H
09H	JMP [R1]	84H

(2) 接线

本实验的接线比较多，需仔细。

1. 将跳线开关 J1 用短路子短接。时序发生器的输入 TJI 接控制存储器的输出 TJ。控制器的输入 C 接运算器 ALU 的 C。控制器的输入 IR7、IR6、IR5、IR4 依次指令寄存器 IR 的输出 IR7、IR6、IR5、IR4。共 6 条线。
2. 控制器的输出 LDIR (CER)、LDPC (LDR4)、PC_ADD、PC_INC、M4、LDIAR、LDAR1 (LDAR2)、AR1_INC、M3、LDER、IAR_BUS#、SW_BUS#、RS_BUS#、ALU_BUS、CEL#、LRW、WRD、LDDR1 (LDDR2)、M1 (M2)、S2、S1、S0 依次与数据通路的对应信号连接。共 27 条线。
3. 指令寄存器 IR 的输出 IR0 接双端口寄存器堆的 RD0、WR0，IR1 接 RD1、WR1，IR2 接 RS0，IR3 接 RS1。共 6 条线。

合上电源。按 CLR#按钮，使实验系统处于初始状态。

(3) 存程序机器代码，设置通用寄存器 R2、R3 及内存相关单元的数据。

● 设置通用寄存器 R2、R3 的值。

在本操作中，我们打算使 R2 = 60H，R3 = 61H。

1. 令 DP = 0，DB = 0，DZ = 0，使实验系统处于连续运行状态。令 SWC = 0、SWB = 1、SWA = 1，使实验系统处于寄存器加载工作方式 KLD。按 CLR#按钮，使实验系统处于初始状态。
2. 在 SW7—SW0 上设置一个存储器地址，该存储器地址供设置通用寄存器使用。该存储器地址最好是不常用的一个地址，以免设置通用寄存器操作破坏重要的存储器单元的内容。例如可将该地址设置为 0FFH。按一次 QD 按钮，将 0FFH 写入 AR0 和 AR1。
3. 在 SW7—SW0 上设置 02H，作为通用寄存器 R2 的寄存器号。按一次 QD 按钮，则将 02H 写入 IR。
4. 在 SW7—SW0 设置 60H，作为 R2 的值。按一次 QD 按钮，将 60H 写入 IR 指定的 R2 寄存器。
5. 在 SW7—SW0 上设置 03H，作为通用寄存器 R3 的寄存器号。按一次 QD 按钮，将 03H 写入 IR。
6. 在 SW7—SW0 设置 61H，作为 R3 的值。按一次 QD 按钮，将 61H 写入 R3。
7. 设置 R2、R3 结束，按 CLR#按钮，使实验系统恢复到初始状态。

● 存程序机器代码。

本操作中，我们从 00 地址开始存 10 个机器代码：58H，5DH，04H，95H，3EH，1BH，4BH，24H，60H，84H。在 60H 存入 24H，用于给 R0 置初值；在 61H 存入 83H，用于给 R0 置初值。

1. 令 DP = 0，DB = 0，DZ = 0，使实验系统处于连续运行状态。令 SWC = 0、SWB = 1、SWA = 0，使实验系统处于写双端口存储器工作方式 KWE。按 CLR#按钮，使实验系统处于初始状态。
2. 置 SW7—SW0 为 00H，按 QD 按钮，将 00H 写入 AR1。
3. 置 SW7—SW0 为 58H，按 QD 按钮，将 58H 写入存储器 00H 单元。AR1 自动加 1，变为 01H。
4. 置 SW7—SW0 为 5DH，按 QD 按钮，将 5DH 写入存储器 01H 单元。AR1 自动加 1，变为 02H。
5. 按 QD 按钮，使 AR1+1。AR1 此时为 02H。
6. 重复进行下去，一直到将 84H 写入存储器 09H 单元。按 CLR#按钮，使实验系统恢复到初始状态。
7. 置 SW7—SW0 为 60H，按 QD 按钮，将 60H 写入 AR1。
8. 置 SW7—SW0 为 24H，按 QD 按钮，将 24H 写入存储器 60H 单元。AR1 自动加 1，变为 61H。
9. 置 SW7—SW0 83H，按 QD 按钮，将 83H 写入存储器 61H 单元。按 CLR#按钮，使实验系统恢复到初始状态。

(4) 用单拍 (DP) 方式执行一遍程序。

在单拍执行过程中, 首先要随时监测 AR2 的值和 IR 的值, 以判定程序执行到何处, 正在执行哪条指令。监测微地址指示灯和判断字段指示灯, 对照微程序流程图, 可以判断出微指令的地址和正在进行的微操作。程序执行的结果如下:

初值: R0 未定, R1 未定, R2 = 60H, R3 = 61H。存储器 60H 单元的内容是 24H, 61H 单元的内容是 83H。

1. LDA R0, [R2]

执行结果 R2 = 60H, R0 = 24H。

2. LDA R1, [R3]

执行结果 R3 = 61H, R1 = 83H。

3. ADD R0, R1

执行结果 R0 = 0A7H, R1 = 83H, C = 0。

4. JC +5

执行结果转移到 03H, 因为 C = 0。

5. AND R2, R3

执行结果 R2 = 60H, R3 = 61H。

6. SUB R3, R2

执行结果 R2 = 60H, R3 = 01H

7. STA R3, [R2]

执行结果 R2 = 60H, R3 = 01H, 存储器 60 单元的内容为 01H。

8. MUL R0, R1

执行结果 R0 = 15H, R1 = 83H

9. STP

执行结果: 无变化

10. JMP [R1]

执行结果 转移到 83H。

第一遍执行结束。执行结果是 R0 = 15H, R1 = 83H, R2 = 60H, R3 = 01H, 存储器 60H 单元的内容是 01H, 61H 单元的内容是 83H。

(5) 用单指 (DZ) 方式执行一遍程序。

初值: R0 = 15H, R1 = 83H, R2 = 60H, R3 = 01H, 存储器 60H 单元的内容是 01H, 61H 单元的内容是 83H。

1. LDA R0, [R2]

执行结果 R2 = 60H, R0 = 01H。

2. LDA R1, [R3]

执行结果 R3 = 01H, R1 = 5DH。

3. ADD R0, R1

执行结果 R0 = 5EH, R1 = 5DH, C = 0

4. JC +5

执行结果转移到 03H, 因为 C = 0。

5. AND R2, R3

执行结果 R2 = 00H, R3 = 01H。

6. SUB R3, R2

执行结果 R2 = 00H, R3 = 01H

7. STA R3, [R2]

执行结果 R2 = 00H, R3 = 01H, 存储器 00 单元的内容为 01H。

8. MUL R0, R1

执行结果 R0 = 0B6H, R1 = 5DH

9. STP

执行结果：无变化

10. JMP [R1]

执行结果 转移到 5DH

第二遍执行结束。执行结果是 R0 = B6H, R1 = 5DH, R2 = 00H, R3 = 01H, 存储器 60H 单元的内容是 01H, 61H 单元的内容是 83H, 00H 单元的内容为 01H。

(6) 用连续方式执行一遍程序

由于 00 单元的内容已被修改, 因此在执行前应首先恢复 00H 单元的内容 58H。

初值: R0 = 0B6H, R1 = 5DH, R2 = 00H, R3 = 01H, 存储器 60H 单元的内容是 01H, 61H 单元的内容是 83H, 00H 单元的内容为 58H。

1. LDA R0, [R2]

执行结果 R2 = 00H, R0 = 58H。

2. LDA R1, [R3]

执行结果 R3 = 01H, R1 = 5DH。

3. ADD R0, R1

执行结果 R0 = 0B5H, R1 = 5DH, C = 0

4. JC +5

执行结果转移到 03H, 因为 C = 0。

5. AND R2, R3

执行结果 R2 = 00H, R3 = 01H。

6. SUB R3, R2

执行结果 R2 = 00H, R3 = 01H

7. STA R3, [R2]

执行结果 R2 = 00H, R3 = 01H, 存储器 00 单元的内容为 01H。

8. MUL R0, R1

执行结果 R0 = 41H, R1 = 5DH

9. STP

执行结果：无变化

第七节 中断原理实验

一、实验目的

- (1) 从硬件、软件结合的角度，模拟单级中断和中断返回的过程；
- (2) 通过简单的中断系统，掌握中断控制器、中断向量、中断屏蔽等概念；
- (3) 了解微程序控制器与中断控制器协调工作的基本原理。

二、实验电路

仍然使用前面的 CPU 组成与机器指令执行实验的电路图，但本次实验加入中断系统。这是一个简单的中断系统模型，只支持单级中断、单个中断请求，有中断屏蔽功能，旨在说明最基本的原理。

中断屏蔽控制逻辑分别集成在 2 片 GAL22V10 (TIMER1 和 TIMER2) 中。其 ABEL 语言表达式如下：

```
INTR1 := INTR;
INTR1.CLK = CLK1;
IE := CLR & INTS # CLR & IE & !INTC;
IE.CLK = MF;
INTQ = IE & INTR1;
```

其中，CLK1 是 TIMER1 产生的时钟信号，它主要是作为 W1—W4 的时钟脉冲，这里作为 INTR1 的时钟信号，INTE 的时钟信号是晶振产生的 MF。INTS 微指令位是 INTS 机器指令执行过程中从控制存储器读出的，INTC 微指令位是 INTC 机器指令执行过程中从控制存储器读出的。INTE 是中断允许标志，控制台有一个指示灯 IE 显示其状态，它为 1 时，允许中断，为 0 时，禁止中断。当 INTS = 1 时，在下一个 MF 的上升沿 IE 变 1，当 INTC = 1 时，在下一个 MF 的上升沿 IE 变 0。CLR 信号实际是控制台产生的复位信号 CLR#。当 CLR = 0 时，在下一个 CLK1 的上升沿 IE 变 0。当 CLR=1 且 INTS = 0 且 INTC = 0 时，IE 保持不变。

INTR 是外部中断源，接控制台按钮 INTR。按一次 INTR 按钮，产生一个中断请求正脉冲 INTR。INTR1 是 INTR 经时钟 CLK1 同步后产生的，目的是保持 INTR1 与实验台的时序信号同步。INTR 脉冲信号的上升沿代表有外部中断请求到达中断控制器。INTQ 是中断屏蔽控制逻辑传递给 CPU 的中断信号，接到微程序控制器上。当收到 INTR 脉冲信号时，若中断允许位 INTE = 0，则中断被屏蔽，INTQ 仍然为 0；若 INTE = 1，则 INTQ = 1。

为保存中断的断点地址，以便中断返回，设置了一个中断地址寄存器 IAR。第二节图 4 中的 IAR (U19) 就是这个中断地址寄存器，它是一片 74HC374，有 LDIAR 和 IAR_BUS# 两个信号输入端，均连接至微程序控制器。LDIAR 信号的上升沿到达时，来自程序计数器 PC 的地址会置入 IAR 中。IAR_BUS# 为 0 时，保存在 IAR 中的断点地址会输出到数据总线 DBUS 上。由于本实验系统只有一个断点寄存器而无堆栈，因此仅支持一级中断而不支持多级中断。

中断向量即中断服务程序的入口地址，在本实验仪中由 8 位数码开关 SW7—SW0 提供。

三、中断的检测、执行和返回过程

微程序控制器每执行一条机器指令之后，执行下一条机器指令之前，先转到微地址 0F 处（见图 12 微程序流程图），在条件位 P1=1 时判断是否有中断请求 INTQ。如果没有 INTQ，则继续正常的机

器指令执行。若检测到中断请求 INTQ，首先发出关中断信号 INTC、保存断点信号 LDIAR，并且发出停机信号 TJ，等待手动设置中断向量。设置好 SW7—SW0 后，按 QD 按钮启动，机器将中断向量读入程序计数器 PC 中，从而转到中断服务子程序去执行。

执行一条机器指令 IRET，从中断服务子程序返回时，发出 IAR_BUS#信号，从中断地址寄存器 IAR 向数据总线 DBUS 输出断点地址，再从 DBUS 依次写入到 R4、PC 中，恢复执行被中断的程序。

发生中断时，关中断由硬件负责，而中断现场(寄存器堆中的寄存器，进位标志 C)的保存和恢复由中断服务程序来处理。

四、实验设备

- (1) TEC—4 计算机组成原理实验系统一台
- (2) 双踪示波器一台
- (3) 直流万用表一只
- (4) 逻辑测试笔一支

五、实验任务

(1) 了解中断系统中每个信号的意义和变化条件，并将下面的主程序和中断服务程序手工汇编成十六进制机器代码，此项任务应在预习时完成。

主程序：

地址	指令	机器代码
20H	INTS	
21H	LDA R0, [R2]	
22H	ADD R0, R0	
23H	ADD R0, R0	
24H	ADD R0, R0	
25H	ADD R0, R0	
26H	ADD R0, R0	
27H	ADD R0, R0	
28H	ADD R0, R0	
29H	JMP [R1]	

中断服务程序：

地址	指令	机器代码
0A0H	AND R0, R0	
0A1H	IRET	

(2) 参考 CPU 组成与机器指令执行实验，再加上中断系统，完成本次实验的线路连接。接通电源之前应仔细检查接线，确认无误。

(3) 将上述任务 (1) 的程序代码存入内存中，并根据需要设置通用寄存器组和内存相关单元的数据。其中，寄存器 R1 的值应置为 21H，以便程序循环执行。

(4) 从地址 20H 执行程序，在程序运行中，按一次控制台的 INTR。进入中断后，用单拍 (DP) 方式执行，直到返回主程序为止。列表记录中断系统中有关信号的变化情况，特别要纪录好断点地址和 R0 的值。

(5) 重复执行 (4) 两次。(一共执行 3 次)

(6) 将 RAM 中 20H 单元的内容由指令 INTS 改为 INTC，重作 (4)，记录发生的现象。

六、实验步骤和实验结果

(1) 实验程序

主程序：

地址	指令	机器代码
20H	INTS	0B0H
21H	LDA R0, [R2]	58H
22H	ADD R0, R0	00H
23H	ADD R0, R0	00H
24H	ADD R0, R0	00H
25H	ADD R0, R0	00H
26H	ADD R0, R0	00H
27H	ADD R0, R0	00H
28H	ADD R0, R0	00H
29H	JMP [R1]	84H

中断服务程序：

地址	指令	机器代码
0A0H	AND R0, R0	30H
0A1H	IRET	0A0H

(2) 接线

本实验的接线和第六节相同，需仔细。

- 将跳线开关 J1 用短路子短接。时序发生器的输入 TJI 接控制存储器的输出 TJ。控制器的输入 C 接运算器 ALU 的 C。控制器的输入 IR7、IR6、IR5、IR4 依次指令寄存器 IR 的输出 IR7、IR6、IR5、IR4。共 6 条线。
- 控制器的输出 LDIR (CER)、LDPC (LDR4)、PC_ADD、PC_INC、M4、LDIAR、LDAR1 (LDAR2)、AR1_INC、M3、LDER、IAR_BUS#、SW_BUS#、RS_BUS#、ALU_BUS、CEL#、LRW、WRD、LDDR1 (LDDR2)、M1 (M2)、S2、S1、S0 依次与数据通路的对应信号连接。共 32 条线。
- 指令寄存器 IR 的输出 IR0 接双端口寄存器堆的 RD0、WR0，IR1 接 RD1、WR1，IR2 接 RS0，IR3 接 RS1。共 6 条线。

合上电源。按 CLR#按钮，使实验系统处于初始状态。

(3) 存程序机器代码，设置通用寄存器 R1、R2 及内存相关单元的数据。

● 设置寄存器 R1、R2 的值

根据要求，设置 R1 = 21H，R2 的值由实验者自定，假定为 10H。

- 令 DP = 0，DB = 0，DZ = 0，使实验系统处于连续运行状态。令 SWB = 1，SWA = 1，使实验系统处于寄存器加载工作方式 KLD。按 CLR#按钮，使实验系统处于初始状态。
- 在 SW7—SW0 上设置一个存储器地址，该存储器地址供设置通用寄存器使用。该存储器地址最好是不常用的一个地址，以免设置通用寄存器操作破坏重要的存储器单元的内容。例如可将该地

址设置为 0FFH。按一次 QD 按钮，将 0FFH 写入 AR1 和 AR2。

9. 在 SW7—SW0 上设置 01H，作为通用寄存器 R1 的寄存器号。按一次 QD 按钮，将 01H 写入 IR。
10. 在 SW7—SW0 设置 21H，按一次 QD 按钮，将 21H 写入 IR 指定的 R1 寄存器。
11. 在 SW7—SW0 上设置 02H，作为通用寄存器 R2 的寄存器号。按一次 QD 按钮，将 02H 写入 IR。
12. 在 SW7—SW0 设置 10H，作为 R2 的值。按一次 QD 按钮，将 10H 写入 IR 指定的 R2 寄存器。
13. 设置 R1、R2 结束，按 CLR#按钮，使实验系统恢复到初始状态。

● 存程序机器代码。

本操作中，我们在 10H 单元存入 01H(也可以是其他值)，从 20 地址开始存 10 个机器代码：0B0H，58H，00H，00H，00H，00H，00H，00H，00H，84H。从 0A0H 存入 2 个机器代码：30H，0A0H。

10. 令 DP = 0，DB = 0，DZ = 0，使实验系统处于连续运行状态。令 SWB = 1，SWA = 0，使实验系统处于写双端口存储器工作方式 KWRD。按 CLR#按钮，使实验系统处于初始状态。
11. 置 SW7—SW0 为 10H，按 QD 按钮，将 10H 写入 AR1。
12. 置 SW7—SW0 为 01H，按 QD 按钮，将 01H 写入存储器 10H 单元。
13. 按 CLR#按钮，使实验系统恢复初始状态。
14. 置 SW7—SW0 为 20H，按 QD 按钮，将 20H 写入 AR1。
15. 置 SW7—SW0 为 0B0H，按 QD 按钮，将 0B0H 写入存储器 20H 单元。AR1 自动加 1，变为 21H。
16. 置 SW7—SW0 为 58H，按 QD 按钮，将 58H 写入存储器 21H 单元。AR1 自动加 1，变为 22H。
8. 重复进行下去，一直到将 84H 写入存储器 29H 单元。按 CLR#按钮，使实验系统恢复到初始状态。
9. 置 SW7—SW0 为 0A0H，按 QD 按钮，将 0A0H 写入 AR1。
10. 置 SW7—SW0 为 30H，按 QD 按钮，将 30H 写入存储器 0A0H 单元。
11. 置 SW7—SW0 0A0H，按 QD 按钮，将 0A0H 写入存储器 0A1H 单元。
12. 按 CLR#按钮，使实验系统恢复到初始状态。

(4) 从地址 20H 执行程序，在程序运行中，按一次控制台的 INTR。进入中断后，用单拍 (DP) 方式执行，直到返回主程序为止。列表记录中断系统中有关信号的变化情况，特别要纪录好断点地址和 R0 的值。

1. 令 DP = 0，DZ = 0，DB = 0，使实验系统处于连续运行状态。
2. 置 SW7—SW0 为 20H，作为程序的起始地址。按 QD 按钮，启动程序从 20H 地址运行。
3. 按 INTR 按钮，发出一个 INTR 中断脉冲，请求中断。中断后硬件自动将中断地址存入中断地址寄存器 IAR。微程序地址应为 25H。
4. 置 SW7—SW0 为 0A0H，这是中断程序的入口地址。将 DP 由置 0 改为置 1。按一次 QD 按钮，将 0A0H 送入程序计数器 PC。微程序地址应为 26H。
5. 按一次 QD 按钮，进行取指微操作。微程序地址应为 05H。
6. 按一次 QD 按钮，进行置数微操作。微程序地址应为 13H。
7. 按一次 QD 按钮，进行 R0 & R0 操作，这时微程序地址应为 38H。观察 DBUS 总线的值，即为 R0 的值。
8. 按一次 QD 按钮，进行写回微操作。微程序地址应为 34H。
9. 按一次 QD 按钮，进行判定有无中断请求 INTQ 微操作。微程序地址应为 0FH。
10. 按一次 QD 按钮，进行取指微操作。微程序地址应为 05H。
11. 按一次 QD 按钮，进行从中断程序返回主程序微操作。微程序地址应为 1AH。这时观察 DBUS 的值，即中断地址。

12. 按一次 QD 按钮，进行判有无新的中断操作。微程序地址为 0FH。

13. 按一次 QD 按钮，进行取指微操作。微程序地址为 05H。此刻，程序已返回主程序。

(5) 操作方式同 (4)。三次纪录到的中断地址和 R0 值一般不会相同。由于按下 INTR 按钮的时间对主程序而言，是随机的，具有不确定性，因此各次中断地址、R0 会具有不同的值。

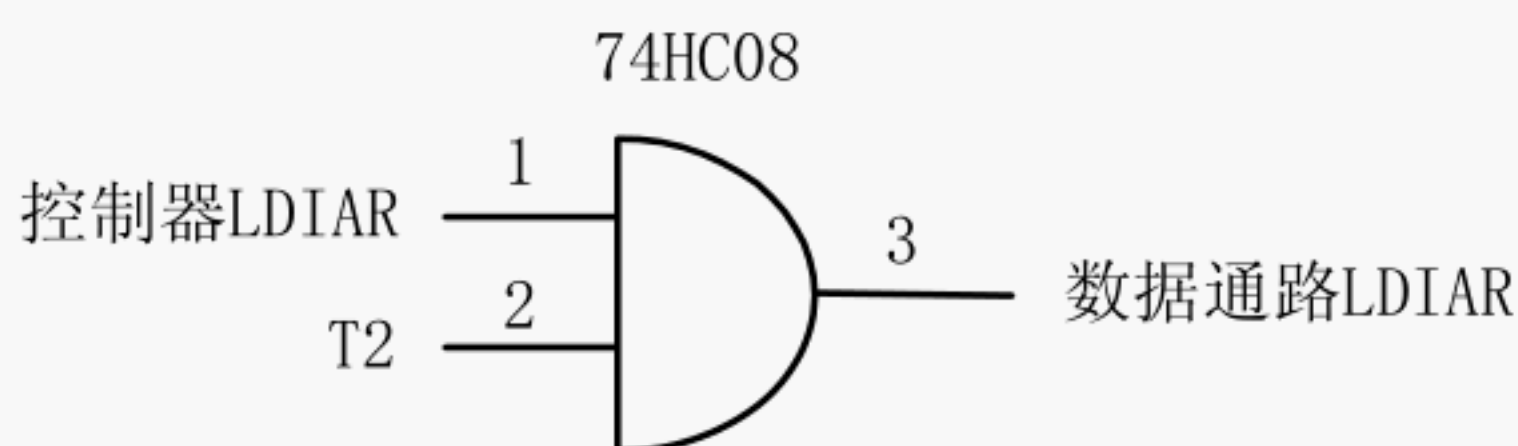
(6) 将 RAM 中 20H 单元的内容由指令 INTS 改为 INTC，重作 (4)，记录发生的现象。

由于 INTC 是关中断指令，因此将不会发生中断。

关于中断原理实验的接线修改说明

中断原理实验的接线在教师实验指导书中接线说明的基础上，需进行如下修改：

1. 控制器的输出 LDIAR 不直接接数据通路的 LDIAR，改为按下图接线。



即控制器的 LDIAR 接芯片 74HC08 的引脚 1，时钟信号 T2 接 74HC08 的引脚 2，74HC08 的引脚 3（即 $LDIAR * T2$ ）接数据通路的 LDIAR。芯片 74HC08 插在用户自选器件实验区中的一个 14 芯双列直插插座中。74HC08 的引脚 7 接地，引脚 14 接+5V。

2. 上述改动的目的是改进实验的可靠性。未改动之前，中断地址寄存器（U29 74HC374）的时钟信号为控存输出的电平信号 LDIAR，对 LDIAR 的上升沿要求高。改动后，U29 的时钟信号变成 $LDIAR * T2$ ，时钟信号的上升沿取决于 T2 的上升沿，降低了对空存输出的 LDIAR 上升沿的要求。

3. 中断原理实验的其余接线参照教师实验指导书。

清华同方股份有限公司
教学仪器设备公司
2002 年 6 月

课程设计一 硬布线控制器的设计与调试

一、教学目的、任务与实验设备

（一）教学目的

（1）融会贯通计算机组成原理课程和计算机系统结构课程的内容，通过知识的综合运用，加深对计算机系统各模块的工作原理及相互联系的认识，特别是对硬布线控制器的认识。

（2）学习运用 ISP（在系统编程）技术进行设计和调试的基本步骤和方法，熟悉集成开发软件中设计、模拟调试工具的使用，体会 ISP 技术相对于传统开发技术的优点。

（3）培养科学研究的独立工作能力，取得工程设计与组装调试的实践经验。

（二）设计与调试任务

（1）按给定的数据格式和指令系统，在所提供的器件范围内，设计一台硬布线控制器控制的模型计算机。

（2）根据设计图纸，在通用实验台上进行组装，或对微程序控制方案的模型机进行改装，并调试成功。

（3）在组装调试成功的基础上，整理出设计图纸和其他文件，包括：

1. 总框图（数据通路图）；
2. 硬布线控制器逻辑模块图；
3. 模块 ABEL 语言源程序（如果有的话）；
4. 硬布线控制流程图；
5. 模拟向量测试方程（可以和 ABEL 源程序合并）；
6. 元件排列图；
7. 设计说明书；
8. 调试小结。

如果没有第 3 点的 ABEL 源程序，则文件中必须包含控制信号的译码函数方程组，用逻辑表达式表示出每个控制信号。

（三）实验设备

- （1）TEC—4 计算机组成原理实验系统一台
- （2）双踪示波器一台
- （3）直流万用表一只
- （4）逻辑测试笔一支
- （5）集成电路若干片，取决于设计方案。

集成电路建议使用 ISP 芯片（一片 ispLSI1032）。采用 ISP 器件，则需要一台 PC 机运行设计自动化软件（例如 ispEXPERT）作设计、编程和下载使用。

二、数据格式和指令系统

采用与模型计算机相同的指令系统，即 12 条机器指令。同样考虑到复杂度和时间问题，一般应降低难度，采用该指令系统的子集：去掉中断指令后的 3 条机器指令，只保留 9 条指令。

三、总体设计

采用的数据通路和微程序控制器方案相同，可参阅前面的介绍，不再赘述。

硬布线控制器模型机所使用的时序信号比微程序控制器的要多一些，除了原有的时序信号 T1 至 T4 外，还需要节拍信号，一拍等于一个 T1 至 T4 循环。实验仪提供的节拍信号有四个：W1 至 W4。如果实验台提供的节拍信号不能满足需要（例如需要八拍），可以用寄存器或 GAL 等自行构造，再从时序发生器引入 T1 至 T4 信号，也可以直接将节拍发生器设计到 ISP 芯片内部。

四、硬布线控制器的设计特点

本次实验的重点就在于硬布线控制器的设计，因为硬布线控制器大部分线路连接都在 ispLSI1032 芯片内部，因此设计方案的优劣，不取决于使用器件的多少，主要取决于设计的思路和质量。

（1）控制器的设计思路

硬布线控制器能够实现控制功能，关键在于它的组合逻辑译码电路。译码电路的任务就是将一系列有关指令、时序等的输入信号，转化为一个个控制信号，输出到各执行部件中。

根据硬布线控制器的基本原理，针对每个控制信号 S，可以列出它的译码函数

$$S = f(I_m, M_i, T_k, B_j)$$

其中 I_m 是机器指令操作码译码器的输出信号， M_i 是节拍信号发生器的节拍信号， T_k 是时序信号发生器的时序信号， B_j 是状态条件判断信号。

在 TEC—4 计算机组成原理实验系统中，因为时序信号 T_k (T1—T4) 已经直接输送给数据通路，所以译码电路不需 T_k 作为输入。又因为机器指令系统比较简单，操作码只有 4 位，不需要专门的操作码译码器，因此 I_m 直接就是操作码，即指令寄存器的 IR4—IR7 信号。 M_i 的来源就是时序模块的节拍信号，例如 W4—W1。 B_j 的信号包括：

- 1、来自数据通路中运算器 ALU 的进位信号 C；
- 2、来自控制台的开关信号 SWC、SWB、SWA；
- 3、其他信号。

其中 C、SWC、SWA 和 SWB 信号在微程序控制器中同样存在，不用加以解释。由于硬布线控制器设计和微程序控制器设计的需求和特点以及控制器的设计方案的不同，可能需要其他信号，也可能不需要其他信号，根据设计方案而定。

每个控制信号的函数式都是上述输入信号的逻辑表达式，因此可以用各种组合逻辑构造电路网络，实现这些表达式的逻辑功能。理论上，只要对所有控制信号都设计出译码函数，这个硬布线控制器的方案也就得到了。

（2）控制流程设计

设计硬布线控制器的控制流程，也就是解决 M_i 、 I_m 、 B_j 如何起作用的问题。设计微程序控制器时可以使用流程图，设计硬布线控制器同样可以使用流程图。微程序控制器的控制信号以微指令周期为时间单位，硬布线控制器以节拍为时间单位，两者本质上是一样的，1 拍和 1 个微指令周期都是从时序 T1 的上升沿到 T4 的下降沿的一段时间。在微程序控制流程图中，1 个执行框代表 1 个微指令周期，而在硬布线控制流程图中，1 个执行框就代表 1 拍。

不过，在微程序控制器中，控制流程可以直接转化为微指令程序，存放于控制存储器内，受到的限制较少；而设计硬布线控制器的控制流程，要考虑的因素就多一些。

1、执行一条机器指令的节拍数

微程序控制器中, 执行一条机器指令所需要的微指令数目可根据需要而定, 不同的机器指令需要的微指令数目可以差别很大, 有的指令使用的微指令数目较多, 只要控制存储器中能安排得下就行。执行一条机器指令所需的微指令数目, 在硬布线控制器中相当于机器指令所需的节拍数。决定执行一条指令需要的节拍数, 要根据所有指令而定。既不能只考虑某些需要最多节拍的指令, 也不能只考虑节拍数最少的指令, 一般要根据大多数机器指令所需的节拍数而定, 设计才比较合理。在本实验中, 由于选用 4 拍对大多数指令就够用, 所以节拍发生器产生 4 个节拍信号 (W1—W4)。

统一用 4 拍执行 1 条机器指令后, 对于所需节拍较少的指令, 例如 JMP 指令只用 2 拍 (忽略中断), 剩下 2 拍就无事可做了。这在可行性上当然没有问题, 但在性能上就打了折扣, 因为多余的节拍都浪费掉了。为减少浪费, 在时序电路中加入了一个控制信号 SKIP 的输入, 该信号的作用是使节拍发生器在任意状态下直接跳到最后 1 拍 (W4)。这样, 设计控制流程时, 在所需节拍较少的指令流程的适当位置使 SKIP 控制信号有效, 多余的节拍就可以跳过, 从而提高了性能。

机器指令选用四拍以后, 对于所需节拍较多的指令如何处理? 采用的方法大致有两种, 一种是修改时序电路, 采用变节拍的方式实现, 即对于所需节拍较多的指令, 时序电路产生较多的节拍, 许多计算机系统就是这样做的。另一种方法是将一条机器指令的执行化为占用两条 (或者更多) 机器指令的节拍, 例如执行一条指令可以占用 W1、W2、W3、W4、W1、W2、W3、W4。为了区分一条指令的两个不同阶段, 可用某些特殊的寄存器标志将其区分, 例如, FLAG = 0 时, 表示该指令执行第一个 W1、W2、W3、W4; FLAG = 1 时, 表示该指令执行第二个 W1、W2、W3、W4。上文中提到的 Bj 包括其他信号, FLAG 就可以认为是一个其他信号。

2、控制台操作的问题

机器指令的执行流程设计出来后, 还要有微程序控制器那样的控制台操作。这部分不属于机器指令系统, 而又要象完成机器指令那样以不超过 4 拍的时间来执行, 以便与机器指令的执行方式统一起来。微程序控制器中有微地址寄存器, 通过微程序地址可以记住是处于控制台状态还是程序运行状态。在硬布线控制器中, 就要设置一个或者几个标志来记录当前的状态。根据这个 (些) 标志, 就可以区分控制台初始状态、控制台读内存、控制台写内存、程序运行状态。

(3) 硬布线控制器

使用 ISP 技术, 则控制器的电路设计完全是在开发软件上进行的。只要在计算机上画出电路的原理图, 编写好必要的源程序, 软件会自动完成控制器内部的线路连接, 无须自己去接线, 因此, 本次实验的工作量可能会比微程序控制器要小一些。

设计出了硬布线控制流程图后, 就可以据此设计出译码逻辑电路。先根据流程图列出译码表, 作为逻辑设计的依据。与微程序表的设计相似, 译码表的内容也包括横向设计和纵向设计。流程图中横向为一拍 (W1、W2、W3、W4 等), 纵向为一条指令, 而译码逻辑是针对每一个控制信号的, 因此在译码表中, 横向变成了一个信号。下表是译码表的一般格式, 单元格中的内容表示该控制信号在该指令中有效的条件 (主要是节拍信号条件)。

表 6 组合逻辑译码表的一般格式

状态 ST	KRD	PR			
指令 IR		ADD	SUB	MUL	
LDDR1		W2			
LDDR2		W2			

CEL#	W1				
.....					

根据译码表可以写出每个控制信号的逻辑表达式,这个表达式就是它所在的行各乘积项相加(逻辑或)。如果使用可编程逻辑电路(GAL、ISP等),只要将表达式直接写成ABEL源程序,编译软件会自动完成电路优化的工作。这样做除了可以减少出错的几率外,未经简化的表达式也便于阅读和理解。在编写译码表的过程中,要注意综合和化简。例如,从译码表中,可能发现LDR4和LDPC的逻辑表达式完全一致,那末LDR4和LDPC可用一个统一的信号代替。有的信号逻辑表达式虽不完全一致,但十分类似,例如LDDR1和LDDR2,这时要考虑一下,在译码表中是否能增加冗余项,使得两个信号的逻辑表达式完全一致。如果能够找到这样的冗余项,则这两个信号可以合并为一个信号。所谓冗余项是指该项虽不是必要的,但增加了它,只是做了多余的动作,并没有导致指令工作不正确。

五、组装与调试

数据通路、时序发生器的组装和分调试可参阅第七节中的介绍,下面是硬布线控制器的分调试和最后的总调试。

由于使用在系统可编程逻辑电路,集成度高,灵活性强,编程、下载方便,用于硬布线控制器将使分调变得相当简单。控制器内部连线集中在芯片内部,由程序自动完成,其速度、准确率和可靠性都是人工接线所不可比拟的。

ISP技术设计的硬布线控制器,其分调试完全是软件模拟的向量测试。但应注意,向量测试方程的设计应全面,尽量覆盖所有的可能性,避免将错漏带到总调试中。另外要注意两个细节问题:

- (1) 测试软件要求测试状态连续,即上一方程的终结状态作为下一方程的初始状态,不能中断;
- (2) 如果方程的输入向量组中有某些影响输出向量的项缺失,测试仍能进行,但缺失项将会以随机值影响输出。

分调正常后,可进行将控制器与数据通路等模块连接,进行全机总调试。

与微程序控制器相类似,总调的第一步是检查全部硬布线控制流程,以单拍(DP)方式执行指令。进行的顺序也是先执行控制台指令,然后执行机器指令。当全部控制流程图检查完毕后,如果数据通路的执行部件(运算器、存储器等)功能正确,就算总调第一步完成。

第二步是在内存中装入包括有全部指令系统的一段程序和有关数据,进一步可采用单步(DB)方式或连续方式执行,以验证机器执行指令的正确性。

第三步是编写一段表演程序,令机器运行。

第四步是运行指导老师给出的验收程序。如果通过,则设计和调试完成。

六、参考设计方案

在上文中对设计硬布线控制器的要点和注意事项已进行了扼要的说明,本部分提供一个设计方案,作为参考。设计方案可以有多种,不存在标准答案。一个设计方案,只要能够实现指令系统的功能,就是正确的。当然,各个设计方案,可能有优劣之分,主要取决于朝什么方向优化,以及优化的程度。一个好的计算机设计方案,应从指令系统开始设计,包括指令的功能和格式,然后设计数据通路和控制器。设计数据通路和控制器的过程中,可能又要对指令系统进行修改,几经反复,才可能确定总体方案。当指令系统和数据通路已经确定之后,只要再确定了指令的节拍,控制器就容易设计了。

本参考方案采用一条指令最多用4拍完成,即W1、W2、W3、W4。对于控制台指令,不能用4拍

完成, 则用控制台标志 ST0 加以区分。按复位按钮 CLR#后, 使 ST0 复位为 0, 当执行完一组 W1、W2、W3、W4 后, 使 ST0 为 1。T1、T2、T3、T4 仍用实验系统的连接。

(一) 硬布线控制器流程图

硬布线控制器流程图如图 14A 和图 14B 所示。它是对微程序控制器流程图稍加改造产生的。此图是没有经过归并和化简的流程图, 目的是使控制流程中的信号看得更清楚。

(二) ABEL 语言源程序

对于硬布线控制器的 ABEL 语言源程序, 在实现的过程中, 我们对下列信号进行了归并。

LDAR1 和 LDAR2	归并为 LDAR2。
LDDR1 和 LDDR2	归并为 LDDR1。
M1 和 M2	由于在流程图内没有 1 的值出现, 只有 0 的值, 因此 ABEL 语言源程序中不出现这两个信号。
CER、LDIR 和 PC_INC	归并为 LDIR。
LDPC 和 LDR4	归并为 LDPC。

在用 ABEL 语言实现硬布线控制器用 ABEL 语言时, 我们增加了 1 个标志位 RUN。由于按 CLR#按钮复位后, 实验系统的时序停止在 T4、W4, ST0 的值为 0, 这样 $SST0 = !ST0 \& W4$ 的值为 1。按 QD 启动按钮后, 由于立即产生 T1 信号, 在 T1 的上升沿使 ST0 置 1, 在第一组 W1、W2、W3、W4 时, ST0 的值为 1, 这是不希望看到的。增加了标志位 RUN 后, 按 CLR#按钮复位, 使 RUN 为 0。由于 $SST0 = !ST0 * W4 * RUN$, 因此复位后的 $SST0 = 0$ 。按 QD 启动按钮, 在 T1 的上升沿, 使 $RUN = 1$ 。根据 SST0 的布尔表达式, 在 W1、W2、W3 时, $SST0 = 0$, 直到 W4 时, 才使 $SST0 = 1$ 。由于 $ST0 := CLR\# * SST0 \# CLR * ST0$, 在 W4 过后的下一个 T1 的上升沿, 才使 ST0 置 1, 从而将控制台操作的两种状态区分开来。

在下面的 ABEL 语言源程序中, 为了阅读方便, 没有将逻辑表达式化简, 编译时 ispEXPERT 会自动将其化简。

```
MODULE CTL_1
```

```
TITLE 'hard-wired control unit'
```

```
"INPUT
```

```
CLR PIN 3;                                "CLR=CLR#
MF, T1, W1, W2, W3, W4 PIN 5..10;
IR4, IR5, IR6, IR7 PIN 12..15;
SWC, SWB, SWA, C PIN 25..28 ;
```

```
"OUTPUT
```

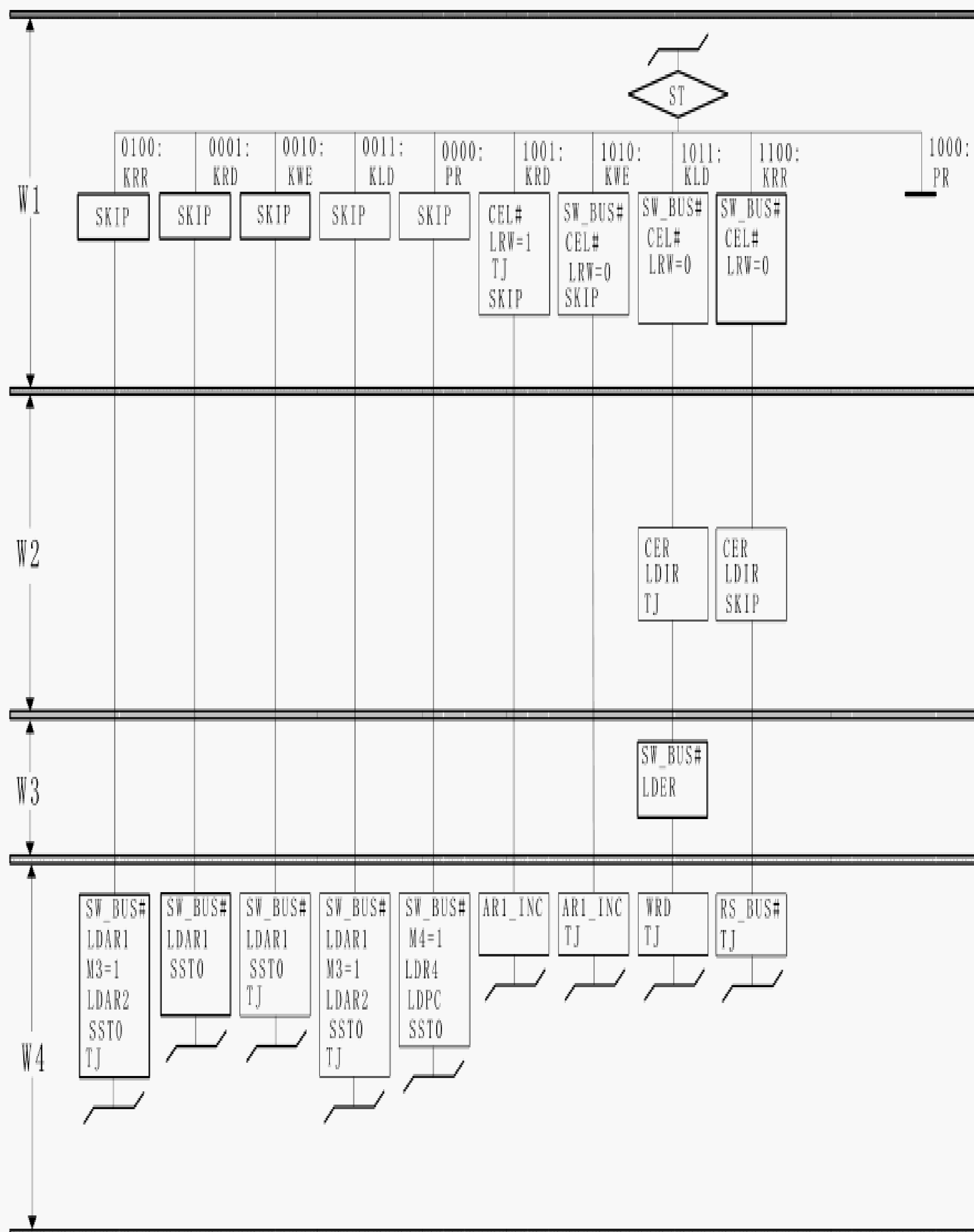


图14A 硬布线控制器流程图

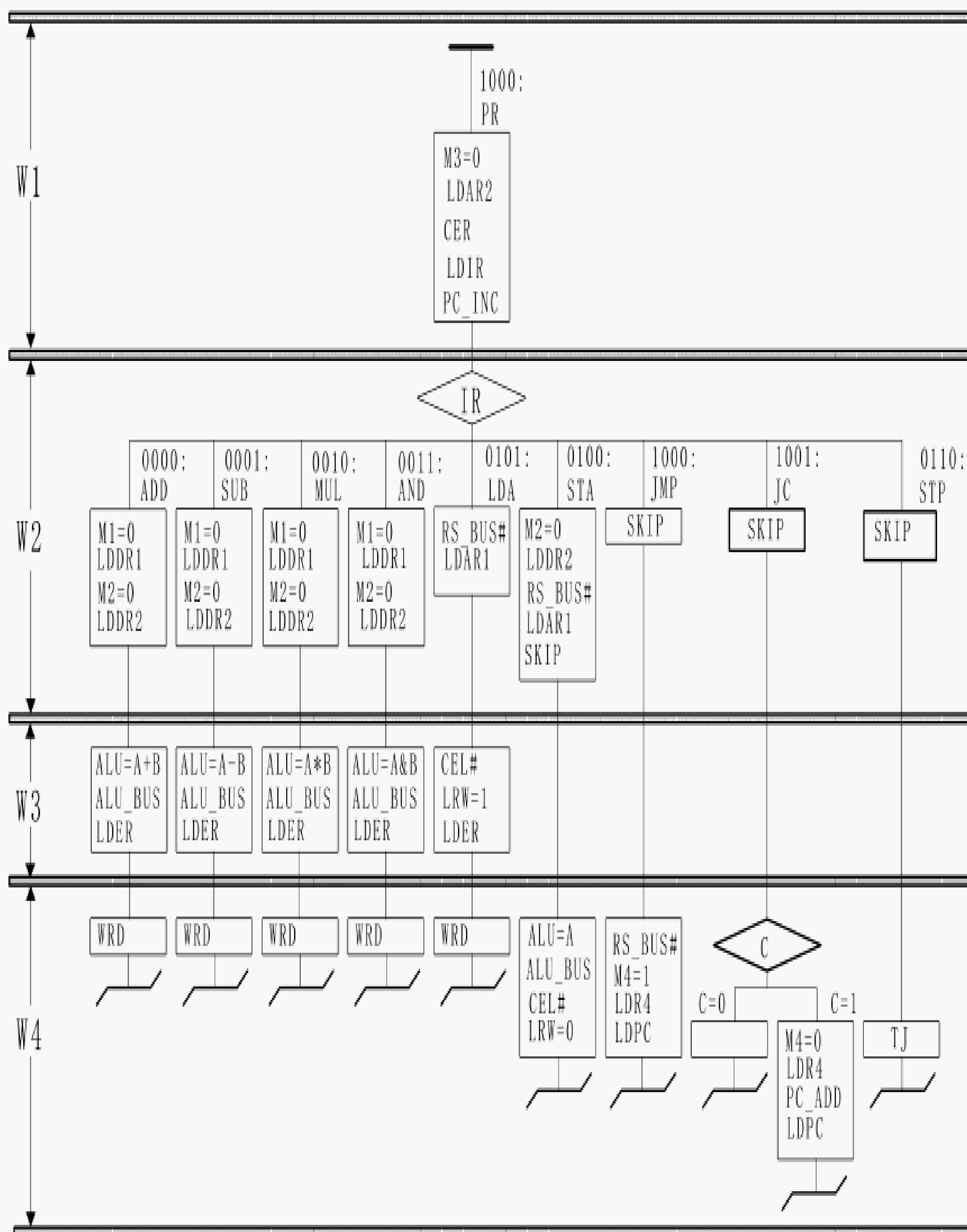


图14B 硬布线控制器流程图（续）


```

MF1, SST0 NODE ISTYPE 'COM' ;
RUN, ST0 NODE ISTYPE 'REG' ;
LDIR, LDPC, PC_ADD, M4 PIN 30..33 ISTYPE 'COM' ;
LDAR1, AR1_INC, M3, CEL, LRW PIN 35..39 ISTYPE 'COM' ;
LDER, SW_BUS, RS_BUS, WRD PIN 45..48 ISTYPE 'COM' ;
"RS_BUS=RS_BUS#, SW_BUS=SW_BUS#, CEL=CEL#
LDDR1, S2, S1, S0, ALU_BUS PIN 52..56 ISTYPE 'COM' ;
TJ, SKIP PIN 69, 70 ISTYPE 'COM' ;
CLK = .C. ;

```

EQUATIONS

```

MF1 = !CLR & MF # T1;
RUN := CLR & !ST0 # CLR & RUN;
RUN.CLK = MF1;
SST0 = !ST0 & RUN & W4;
ST0 := CLR & SST0 # CLR & ST0;
ST0.CLK = MF1;

LDIR = ST0 & !SWC & !SWB & !SWA & W1
      # ST0 & !SWC & SWB & SWA & W2
      # ST0 & SWC & !SWB & !SWA & W2;
LDPC = !ST0 & !SWC & !SWB & !SWA & W4
      # ST0 & !SWC & !SWB & !SWA & W4 & IR7 & !IR6 & !IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & IR7 & !IR6 & !IR5 & IR4 & C;
PC_ADD = ST0 & !SWC & !SWB & !SWA & W4 & IR7 & !IR6 & !IR5 & IR4 & C;
M4 = !ST0 & !SWC & !SWB & !SWA & W4
     # ST0 & !SWC & !SWB & !SWA & IR7 & !IR6 & !IR5 & !IR4 & W4;
LDAR1 = !ST0 & !SWC & !SWB & SWA & W4
        # !ST0 & !SWC & SWB & !SWA & W4
        # !ST0 & !SWC & SWB & SWA & W4
        # !ST0 & SWC & !SWB & !SWA & W4
        # ST0 & !SWC & !SWB & !SWA & W1
        # ST0 & !SWC & !SWB & !SWA & W4
        # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & IR4 & W2
        # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & !IR4 & W2;
AR1_INC = ST0 & !SWC & !SWB & SWA & W4
          # ST0 & !SWC & SWB & !SWA & W4;
M3 = !ST0 & !SWC & SWB & SWA & W4
     # !ST0 & SWC & !SWB & !SWA & W4;
LDER = ST0 & !SWC & SWB & SWA & W3
      # ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & IR4

```

```

# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & IR6 & !IR5 & IR4;
WRD = ST0 & !SWC & SWB & SWA & W4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & IR4;
SW_BUS = !( !ST0 & W4 & !SWC & !SWB & !SWA
# !ST0 & W4 & !SWC & !SWB & SWA
# !ST0 & W4 & !SWC & SWB & !SWA
# !ST0 & W4 & !SWC & SWB & SWA
# !ST0 & W4 & SWC & !SWB & !SWA
# ST0 & W1 & !SWC & SWB & !SWA
# ST0 & W1 & !SWC & SWB & SWA
# ST0 & W1 & SWC & !SWB & !SWA
# ST0 & W3 & !SWC & SWB & SWA);
RS_BUS = !(ST0 & SWC & !SWB & !SWA & W4
# ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & IR6 & !IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W4 & IR7 & !IR6 & !IR5 & !IR4);
ALU_BUS = ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & !IR4;
CEL = !(ST0 & !SWC & !SWB & SWA & W1
# ST0 & !SWC & SWB & !SWA & W1
# ST0 & !SWC & SWB & SWA & W1
# ST0 & SWC & !SWB & !SWA & W1
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & !IR4);
LRW = ST0 & !SWC & !SWB & SWA & W1
# ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & IR6 & !IR5 & IR4;
LDDR1 = ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & !IR6 & !IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & !IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & !IR6 & IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & !IR6 & IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W2 & !IR7 & IR6 & !IR5 & !IR4;
S2 = ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & IR5 & !IR4;
S1 = ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & !IR4

```

```

      # ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & IR4;
S0 = ST0 & !SWC & !SWB & !SWA & W3 & !IR7 & !IR6 & !IR5 & IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & !IR4;
TJ = !ST0 & !SWC & SWB & !SWA & W4
      # !ST0 & !SWC & SWB & SWA & W4
      # !ST0 & SWC & !SWB & !SWA & W4
      # ST0 & !SWC & !SWB & SWA & W1
      # ST0 & !SWC & SWB & SWA & W2
      # ST0 & !SWC & SWB & !SWA & W4
      # ST0 & !SWC & SWB & SWA & W4
      # ST0 & SWC & !SWB & !SWA & W4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & IR5 & !IR4;
SKIP = !ST0 & !SWC & !SWB & !SWA & W1
      # !ST0 & !SWC & !SWB & SWA & W1
      # !ST0 & !SWC & SWB & !SWA & W1
      # !ST0 & !SWC & SWB & SWA & W1
      # !ST0 & SWC & !SWB & !SWA & W1
      # ST0 & !SWC & !SWB & SWA & W1
      # ST0 & !SWC & SWB & !SWA & W1
      # ST0 & SWC & !SWB & !SWA & W2
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & !IR4 & W2
      # ST0 & !SWC & !SWB & !SWA & IR7 & !IR6 & !IR5 & !IR4 & W2
      # ST0 & !SWC & !SWB & !SWA & IR7 & !IR6 & !IR5 & IR4 & W2
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & IR5 & !IR4 & W2;
END

```

（三）编译、下载和接线

（1）编译和下载

首先，在 PC 机上运行 ispEXPERT 软件，建立一个新项目，在该项目下，输入 ABEL 语言源文件，经过编译和适配，生成 JED 文件。用出厂时提供的下载电缆将 PC 机的打印机接口和实验台上的下载插座(编程插座)相连。打开实验系统电源，即可在 ispEXPERT 软件内向 ISP1032 下载。有关 ispEXPERT 的使用，可参考有关资料，这里不再叙述。

（2）接线

1. 接固定连线和数据通路内部连线

M2 接 M1，M1 接 GND。CER 接 PC_INC，PC_INC 接 LDIR。LDAR2 接 LDAR1。LDDR2 接 LDDR2。LDPC 接 LDR4。IR0 接 RD0，IR1 接 RD1，WR0 接 RD0，WR1 接 RD1，IR2 接 RS0，IR3 接 RS1。IAR_BUS 接 VCC。共 14 条线。

2. 控制台和 isp1032 的连线

CLR#接 PIN3 (CLR)，SWC 接 PIN25 (SWC)，SWB 接 PIN26 (SWB)，SWA 接 PIN27 (SWA)。共 4 条线。

3. 时序电路和 isp1032 的连线

MF 接 PIN5 (MF)，T1 接 PIN6 (T1)，TJI 接 PIN69 (TJ)，SKIP 接 PIN70 (SKIP)。W1 接 PIN7 (W1)，

W2 接 PIN8 (W2), W3 接 PIN9 (W3), W4 接 PIN10。共 8 条线。

4. 数据通路和 isp1032 的连线

IR4 接 PIN12 (IR4), IR5 接 PIN13 (IR5), IR6 接 PIN14 (IR6), IR7 接 PIN15 (IR7)。C 接 PIN28 (C)。LDIR 接 PIN30 (LDIR)。LDPC 接 PIN31 (LDPC), PC_ADD 接 PIN32 (PC_ADD), M4 接 PIN33 (M4), M3 接 PIN37 (M3)。LDAR1 接 PIN35 (LDAR1), AR1_INC 接 PIN36 (AR1_INC), CEL#接 PIN38 (CEL), LRW 接 PIN39 (LRW)。LDER 接 PIN45 (LDER), SW_BUS#接 PIN46 (SW_BUS), RS_BUS#接 PIN47 (RS_BUS), WRD 接 PIN48 (WRD)。LDDR1 接 PIN52 (LDDR1), S2 接 PIN53 (S2), S1 接 PIN54 (S1), S0 接 PIN55 (S0), ALU_BUS 接 PIN56 (ALU_BUS)。共 23 条线。

课程设计二 流水微程序控制器的设计与调试

一、教学目的、任务与实验设备

（一）教学目的

（1）通过知识的综合运用，加深对计算机系统各模块协同工作的认识，特别是对并行、流水的认识，加深计算机工作中“时间—空间”概念的理解，从而建立基本的流水计算机的概念。

（2）学习通过“固件升级”即改写微程序的方法来提高计算机系统性能的方法，体会设计方案优劣对性能发挥的重要性。

（3）培养科学研究的独立工作和创新能力，取得设计与调试的实践经验。

（二）设计与调试任务

（1）在微程序控制器实验的基础上，设计一台微程序控制的指令级标量流水模型计算机。

（2）根据设计图纸，在通用实验台上进行组装，并调试成功。

（3）在组装调试成功的基础上，整理出设计图纸和其他文件。包括的文件是

1. 总框图（数据通路图）；
2. 微程序控制器逻辑图；
3. 微程序流程图；
4. 微程序代码表；
5. 元件排列图；
6. 设计说明书；
7. 调试小结。

（三）实验设备

- （1）TEC—4 计算机组成原理实验仪一台
- （2）双踪示波器一台
- （3）直流万用表一只
- （4）逻辑测试笔一支
- （5）集成电路若干片，取决于设计方案。

二、指令系统

采用与模型机相同的指令系统，即 12 条机器指令。考虑到复杂度和时间问题，一般应降低难度，采用该指令系统的子集：去掉中断指令后的 3 条机器指令，只保留前面的 9 条指令。

三、设计要求

设计流水方案时，牢记设计的目的是提高系统性能。没有性能改善、为流水而流水的方案是毫无意义的。

实验系统的时序发生器将一个微指令周期分为 T1 至 T4 四段，原则上，本次实验只利用四段划分，即仍使用 T1—T4 作为时序脉冲。

设计时，应充分考虑控制信号的综合和化简，出厂时的模型机提供了这方面的某些化简实例，但还可以进一步化简。

四、总体设计

指令级标量流水，是指把机器指令的解析过程分解为取指、译码、访存、执行、写回等子过程，各子过程以流水方式运行。考虑到复杂度，在实际设计中，可以分解得不那么细。例如可只分为取指、执行、写回三个子过程。

要实现流水，至少需要具备两个条件：

(1) 数据通路（包括其上的执行器件）要支持流水。

设计这种支持流水的数据通路时，需要解决几个主要问题：

- 1、流水线各段争用总线的问题；
- 2、各段之间互通信息、互相等待的问题；
- 3、各段工作时序协调一致的问题；
- 4、中断、程序转移的处理问题。

本实验仪采用了不少具有并行操作功能的器件，如存储器、寄存器堆等，对支持流水有一定的帮助，但还需要使它们组成流水线。从广义上理解并行，则流水是并行的一个特例，只有具备先后、因果关系的并行，才能称为流水。例如，图 12 中，DR1 和 DR2 可以并行操作，但它们的关系不是流水。

实际上，图 12 的数据通路已经基本布置好了流水线，它可以划分为如下功能部件：

- 1、取指段：包括 RAM 的右端口、AR2、MUX3、PC、IR。
- 2、执行段：包括 ALU、DR1、MUX1、DR2、MUX2、RF、ER、RAM 的左端口、AR1、IR、R4、MUX4、ALU2、PC。
- 3、写回段：包括 ER、RF。

注意：在数据通路中可能需要对写回寄存器选择信息进行缓冲，为此要增加如图 15 的缓冲寄存器（否则，WR1、WR0 的信息会在写回操作之前改变）。设计的方案不同，可能需要的缓冲寄存器也会有所不同。

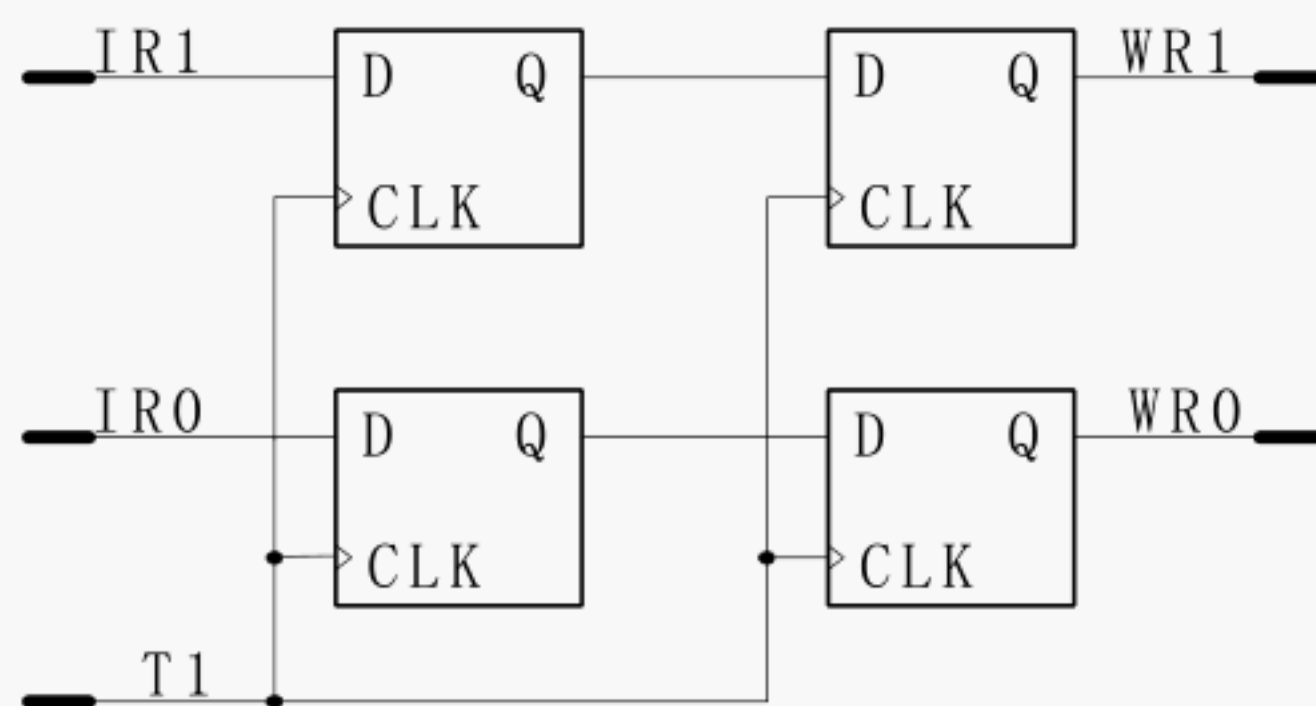


图 15 操作数缓冲寄存器

也可采用别的划分，上述划分方法仅供参考。

(2) 控制器要有能力驱动流水线。

在模型机中，使用的是常规的微程序控制器，虽然数据通路相同，但没有利用其中的流水功能，因此得到的整体系统仍然是常规模型计算机。本实验中，可能要对原有的微程序控制器进行必要的改进，使之成为流水微程序控制器。

五、流水微程序控制器

由于数据通路基本仍然沿用第二节图 4，无须重新设计，因此本次实验的重点就集中在控制器的设计上。这里使用指令系统的子集，只有 9 条指令系统。由于流水控制中，对中断断点的处理会增加流程的复杂度，增大理解的难度，因此不包括中断部分。相信善于思考的同学是可以设计出支持中断的方案。

（1）微指令格式

在本实验中仍采用水平型微指令格式，这是由于采用垂直型微指令来控制并行度较高的流水线将会困难重重。微命令编码仍然可以用直接表示法，后继地址用断定方式。

严格地说，没有中断控制，相应的控制信号可以省略，因此微指令字长可以缩短。为了兼容模型机方案，仍需保留这些控制信号，以减少接线的困难。

（2）微程序控制器

支持流水并未对控制器的硬件结构提出更高的要求。微程序控制器的逻辑结构与模型机的差异的只是顺序控制部分，即微程序转移逻辑电路，原因是微程序需要重新设计，重新分配微地址。新设计的微程序转移逻辑电路，可以用一片 ispLSI1032 实现，也可用中小规模的数字器件实现。

（3）时空图

如果是要从头设计一套流水系统，时空图应在设计数据通路之前确定，因为具体的流水线是跟时空图紧密相联的。本实验是在已有的数据通路的基础上（已知该数据通路支持流水），通过改进控制来提升系统性能，因此将时空图放在控制器部分进行设计。下面是 3 个时空图：

周期	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
取指	1		2		3		4		5		6		7		8			
执行		1	1	2	2	3	3	4	4	5	5	6	6	7	7	8	8	
写回				1		2		3		4		5		6		7		8

图 15A 参考时空图 1

周期	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
功能段 1	1	1	2	2	3	3	4	4	5	5	6	6	7	7	8	8	
功能段 2		1	1	2	2	3	3	4	4	5	5	6	6	7	7	8	8

图 15B 参考时空图 2

周期	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
功能段 1	1	2	3	4	5	6	7	8	9	10	11	12			
功能段 2		1	2	3	4	5	6	7	8	9	10	11	12		
功能段 3			1	2	3	4	5	6	7	8	9	10	11	12	
功能段 4				1	2	3	4	5	6	7	8	9	10	11	12

图 15C 参考时空图 3

选用的时空图只要设计可以实现即可。不同的时空图直接影响系统的性能。

（4）微程序设计

控制器的强大与否，对机器性能发挥出多少，就取决于如何设计控制流程，并转化为微程序。微程序设计包括了横向设计和纵向设计。要实现流水控制，纵向设计仍然重要，而横向设计也同样需要仔细考虑。因为流水线中要求尽量多的并行操作，以便充分利用硬件资源，减少闲置。横向设计中就应把不冲突的、可以同时实现的控制，放在同一微指令中。纵向设计中，考虑控制顺序要连同并发控制一起考虑，尤其是出现冲突时，需要前后错开。此外就是注意在机器指令发生转移（无条件、有条件或中断）的地方，要丢弃已取的指令，重新做一次取指操作。

设计流水控制的微程序与设计常规的微程序相比，难度要大一些，主要是因为常规流程是单线索的，每一微指令周期要做什么，思路很清晰，而流水控制的流程则同时包含多重线索，每个微指令周期都可能含有两个以上不相干的操作，比较复杂，需要用并发的思维去考虑。由于可能出现的情况大大增多，设计时容易出错，因此一定要细心，规划周全。如果先设计出常规的微程序控制流程图，然后进行微指令合并，出错的可能性会比直接设计流水控制的微程序要小一些。合并方法是：观察每个微指令周期和上一周期的操作是否冲突，不冲突则将本周期的操作叠加至上一周期。流水控制的微程序流程必须涵盖所有可能的情况，为此可以用地址不同、操作类似的微指令来实现不同的分支。

前面所叙述的课程设计说明，是对学习“计算机系统结构”课程的计算机系所有学生的基本教学要求：按给定的指令系统和指令格式，完成一台微程序控制的模型流水计算机的设计和调试。

六、参考流水控制器设计方案

(1) 流水微程序控制器流程图

图 17 是参考流水微控制器流程图。它是在图 12 常规微程序控制器流程图的基础上经过归并产生的。由于控制台操作并不需要流水操作，控制台部分和常规微程序流程图基本相同，只是在加载通用寄存器控制台操作中加了一个空操作（微地址为 2FH）。这是由于 IR1、IR0 经过两级缓存才变为 WR1、WR0，如果没有这个空操作，会造成加载的寄存器号码不对。

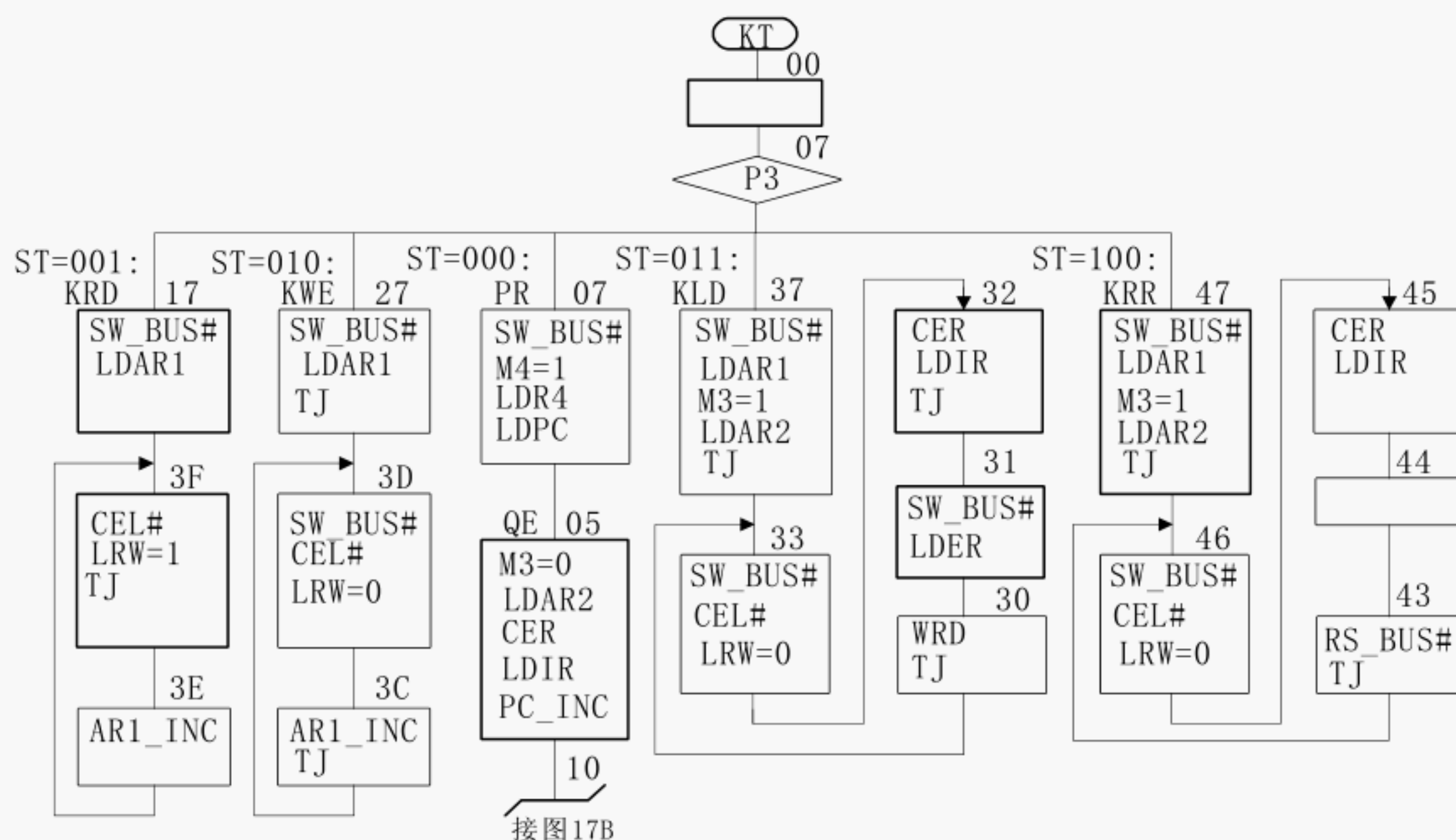


图17A 流水微程序控制器流程图

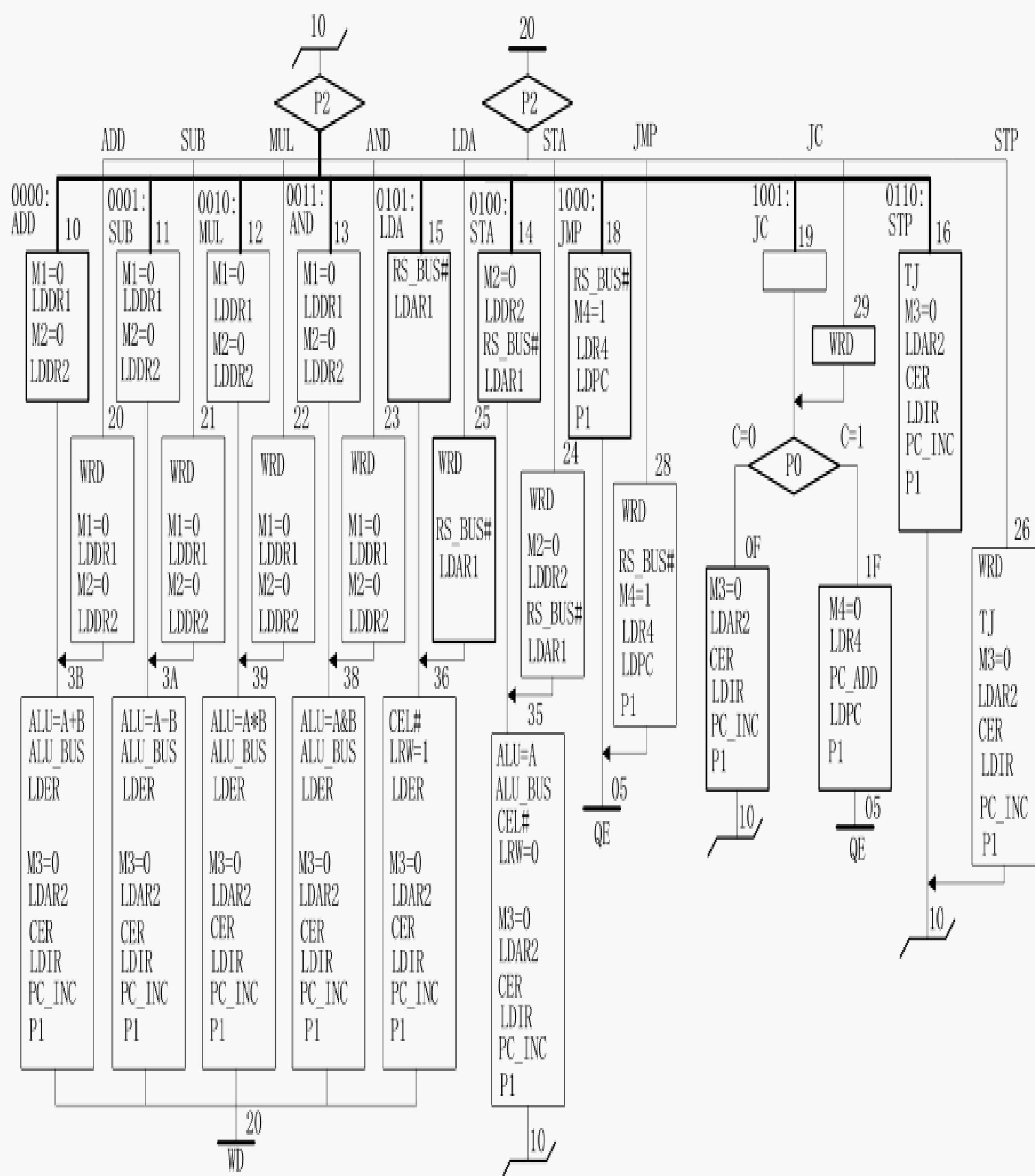


图17B 流水微程序控制器流程图（续）

在图 17B 中，并行操作主要体现在两点，取指微操作和运算微操作并行，回写微操作和置数微操作并行。

(2) 微指令信号表

直接根据流水微程序控制器流程图计算出微程序代码比较困难。为了计算微程序代码，需要一个各微指令使用的信号表。

表 6 流水微程序控制器使用的控制信号表

微指令	KT	KRD			KWE			KLD					PR	QE	KT1	KRR					STP		
当前微地址	00	17	3F	3E	27	3D	3C	37	33	32	31	30	07	05	40	47	46	45	44	43	46	16	26
下一微地址	07	3F	3E	3F	3D	3C	3D	33	32	31	30	33	05	10	47	46	45	44	43	46	10	10	
P0	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
P1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	1	1
P2	.	.	.	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	1	.
P3	1	.	.	.	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	.
*INTS	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
*INTC	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
LD_IR	.	.	.	.	.	.	.	.	.	1	.	.	.	1	.	.	.	1	.	.	.	1	1
LD_PC	.	.	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	.	.	.
PC_ADD	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
PC_INC	.	.	.	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	1	1
M4	.	.	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	.	.	.
*LD_IAR	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
LD_AR1	.	1	.	.	1	.	.	1	.	.	.	.	.	1	.	1	.	.	.	.	.	1	1
AR1_INC	.	.	.	1	.	.	1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
M3	.	.	.	.	.	.	.	1	.	.	.	.	.	0	.	1	.	.	.	.	.	0	0
LD_ER	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	.	.	.	.	.
*IAR_BUS#	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
SW_BUS#	1	0	1	1	0	0	1	0	0	1	0	1	0	1	1	0	0	1	1	1	1	1	1
RS_BUS#	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	
ALU_BUS	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
CEL#	1	1	0	1	1	0	1	1	0	1	1	1	1	1	1	1	0	1	1	1	1	1	1
LRW	.	.	1	.	.	0	.	.	0	.	.	.	.	.	.	.	0	.	.	.	.	.	.
WRD	.	.	.	.	.	.	.	.	.	.	.	1	.	.	.	.	.	.	.	.	.	.	1
LD_DR1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
M1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
S0	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
S1	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
S2	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
TJ	.	.	1	.	1	.	1	1	.	1	.	1	.	.	.	1	.	.	.	1	1	1	1

注：1. 带 *的信号为中断信号，本方案中不使用。

2. “.” 代表 0。

表 6 微程序流水控制器使用的控制信号表（续）

微指令	ADD			SUB			MUL			AND			LDA			STA			JMP		JC			
当前微地址	10	20	3B	11	21	3A	12	22	39	13	23	38	15	25	36	14	24	35	18	28	19	29	0F	1F
下一微地址	3B	3B	20	3A	3A	20	39	39	20	38	38	20	36	36	20	35	35	10	05	05	0F	0F	10	05
P0	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	1	1	.	.
P1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	.	.	1	1
P2	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	.	.	1	.
P3	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
*INTS	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
*INTC	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
LD_IR	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	.	.	1	.
LD_PC	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	1	1	.	.	.	1
PC_ADD	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	1
PC_INC	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	.	.	1	.
M4	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	1	1	.	.	.	0
*LD_IAR	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
LD_AR1	.	.	1	.	.	1	.	.	1	.	.	1	1	1	1	1	1	1	.	.	.	.	1	.
AR1_INC	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.
M3	.	.	0	.	.	0	.	.	0	.	.	0	.	.	0	.	.	0	.	.	.	.	0	.
LD_ER	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	.	.	.	.	.	.	.
*IAR_BUS#	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
SW_BUS#	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
RS_BUS#	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	0	0	1	0	0	1	1	1	1
ALU_BUS	.	.	1	.	.	1	.	.	1	.	.	1	.	.	.	.	.	1	.	.	.	.	.	.
CEL#	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	1	1	0	1	1	1	1	1	1
LRW	.	.	.	.	.	.	.	.	.	.	.	.	.	.	1	.	.	0	.	.	.	.	.	.
WRD	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	.	1	.	1	.	.
LD_DR1	1	1	.	1	1	.	1	1	.	1	1	.	.	.	.	1	1	.	.	.	.	.	.	.
M1	0	0	.	0	0	.	0	0	.	0	0	.	.	.	.	0	0	.	.	.	.	.	.	.
S0	.	.	0	.	.	1	.	.	0	.	.	0	.	.	.	.	.	1	.	.	.	.	.	.
S1	.	.	1	.	.	1	.	.	0	.	.	0	.	.	.	.	.	0	.	.	.	.	.	.
S2	.	.	0	.	.	0	.	.	1	.	.	0	.	.	.	.	.	0	.	.	.	.	.	.
TJ	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.

注：1. 带 *的信号为中断信号，本方案中不使用。

2. “.” 代表 0。

在本表中，同样将下列信号进行了归并：LDIR 和 CER 归并为 LDIR，M1 和 M2 归并为 M1，LDAR1 和 LDAR2 归并为 LDAR1，LDDR1 和 LDDR2 归并为 LDDR1，LDPC 和 LDR4 归并为 LDPC。同样，从本表中可以很容易发现，M1 的值为全 0。因此可以固定接 GND，而从表中删掉。由于本方案中不使用中断，有关的信号是无用的，本表中保留这些信号，是为了和常规的微程序控制器微指令格式一致，以方便接线。表中用“.”代表 0，是为了非 0 信号醒目，突出微程序流程图中出现的信号。

(3) 控制存储器代码表

根据微指令信号表 6，得到控制存储器代码表 7

表 7 流水微程序控制器代码表

地 址	CM4	CM3	CM2	CM1	CM0	地 址	CM4	CM3	CM2	CM1	CM0
00	00	05	C0	02	07	20	00	35	C0	00	3B
01						21	00	35	C0	00	3A
02						22	00	35	C0	00	39
03						23	00	35	C0	00	38
04						24	00	34	C4	00	35
05	00	05	C4	91	10	25	00	14	C4	00	36
06						26	04	15	C4	91	90
07	00	05	41	20	05	27	04	05	44	00	3D
08						28	00	14	C1	20	85
09						29	00	15	C0	00	4F
0A						2A					
0B						2B					
0C						2C					
0D						2D					
0E						2E					
0F	00	05	C4	91	90	2F					
10	00	25	C0	00	3B	30	04	15	C0	00	33
11	00	25	C0	00	3A	31	00	05	60	00	30
12	00	25	C0	00	39	32	04	05	C0	10	31
13	00	25	C0	00	38	33	00	01	40	00	32
14	00	24	C4	00	35	34					
15	00	04	C4	00	36	35	00	83	C4	91	90
16	04	05	C4	91	90	36	00	09	E4	91	0A0
17	00	05	44	00	3F	37	04	05	54	00	33
18	00	04	C1	20	85	38	00	07	E4	91	0A0
19	00	05	C0	00	4F	39	02	07	E4	91	0A0
1A						3A	01	87	E4	91	0A0
1B						3B	01	07	E4	91	0A0
1C						3C	04	05	C8	00	3D
1D						3D	00	01	40	00	3C
1E						3E	00	05	C8	00	3F
1F	00	05	C0	60	85	3F	04	09	C0	00	3E
						40	00	05	C0	02	47
						41					
						42					
						43	04	04	C0	00	46
						44	00	05	C0	00	43
						45	00	05	C0	10	44
						46	00	01	40	00	45
						47	04	05	54	00	46

代码表内有若干空白项，理论上这些空白项填充任何值都可以，不过以填充空操作对应的数值

最好。所谓空操作就是对每一个信号都取它的非有效值，下一微地址使用 00H。因此，对于任意一个微地址，填充的值应为 00 05 0C0 00 00。根据控制存储器代码表，可以很容易生成二进制文件，将它写入 28C64 芯片即可。

（4）控制器微地址生成方案

在本方案中，使用的微地址生成方案和常规微程序控制器微地址生成方案完全一致。因此，还是使用常规微程序的控制器，只是改变了控制存储器的内容。电路图参看第五节图 11《微程序控制器的组成》。

IR、IRO 经过图 15 所示的两级缓存才能变成 WR1、WR0，与寄存器堆所需的 WR1、WR0 连接。

（5）接线

本实验的接线比较多，需仔细。

7. 将跳线开关 J1 用短路子短接。时序发生器的输入 TJI 接控制存储器的输出 TJ。控制器的输入 C 接运算器 ALU 的 C。控制器的输入 IR7、IR6、IR5、IR4 依次指令寄存器 IR 的输出 IR7、IR6、IR5、IR4。共 6 条线。
8. 控制器的输出 LDIR (CER)、LDPC (LDR4)、PC_ADD、PC_INC、M4、LDIAR、LDAR1 (LDAR2)、AR1_INC、M3、LDER、IAR_BUS#、SW_BUS#、RS_BUS#、ALU_BUS、CEL#、LRW、WRD、LDDR1 (LDDR2)、M1 (M2)、S2、S1、S0 依次与数据通路的对应信号连接。共 27 条线。
9. 指令寄存器 IR 的输出 IRO 接双端口寄存器堆的 RD0，经两级缓存后接 WR0，IR1 接 RD1，经两级缓存后接 WR1，IR2 接 RS0，IR3 接 RS1，共 8 条线。

课程设计三 流水硬布线控制器的设计与调试

一、教学目的、任务与实验设备

（一）教学目的

- （1）通过知识的综合运用，进一步加深对并行、流水的认识，从而建立流水计算机的整机概念。
- （2）培养科学研究的独立工作能力，充分发挥创造力，进一步加强运用 ISP 技术进行工程设计和组装调试的经验。

（二）设计与调试任务

- （1）在硬布线控制器的基础上，设计一台硬布线控制器的指令级标量流水模型计算机。
- （2）根据设计图纸，在通用实验台上进行组装，或对硬布线控制器方案的模型机进行改装，并调试成功。
- （3）在组装调试成功的基础上，整理出设计图纸和其他文件。包括的文件：
 1. 总框图（数据通路图）；
 2. 硬布线控制器逻辑模块图；
 3. 模块 ABEL 语言源程序（如果有的话）；
 4. 硬布线控制器流程图；
 5. 模拟向量测试方程（可以和 ABEL 源程序合并）；
 6. 元件排列图；
 7. 设计说明书；
 8. 调试小结。

如果没有第 3 点的 ABEL 源程序，则文件中必须包含控制信号的译码函数方程组，用逻辑表达式表示出每个控制信号。

（三）实验设备

- （1）TEC—4 计算机组成原理实验系统一台
- （2）双踪示波器一台
- （3）直流万用表一只
- （4）逻辑测试笔一支
- （5）集成电路若干片，取决于设计方案。

集成电路建议使用 ISP 芯片（一片 ispLSI1032），若采用 ISP 器件，则需要一台 PC 机运行设计自动化软件（例如 ispEXPERT）作设计、编程和下载使用。

二、数据格式和指令系统

采用与模型机相同的指令系统，即 12 条机器指令。考虑到复杂度和时间问题，一般应降低难度，采用该指令系统的子集：去掉中断指令后的 4 条机器指令，只使用 9 条机器指令。

三、设计要求

设计流水方案时，牢记设计的目的是提高系统性能。没有性能改善、为流水而流水的方案是毫无意义的。

实验系统的时序发生器将一个微指令周期分为 T1 至 T4 四段，原则上，本次实验只利用四段划分，即仍使用 T1—T4 作为时序脉冲。

设计时，应充分考虑控制信号的综合和化简，出厂时的模型机提供了这方面的某些化简实例，但还可以进一步化简。

四、总体设计、组装、调试

理解第九节提到的实现流水所需要具备的两个必要条件。对于数据通路部分，无须再作更多的说明，本实验的重点在控制器的设计上。

由于采用 ISP 技术，设计控制器的自由度非常大，模拟调试验证也很方便，相信只要发挥各人的聪明才智，实现的方法可以是多种多样的。本实验鼓励每个人充分创新，提出尽量多的解决方案。

硬布线控制器的设计方法请参考第十节，时空图的设计请参考第九节，实现流水的方案请参考前面几节的总体设计，自由发挥，不再赘述。数据通路沿用图 4，在已有的数据通路上（或者稍加改进），设计出自己的流水硬布线控制器方案，控制器主要在 ispLSI1032 内完成。这实际上只是一个具有流水功能的硬布线控制器设计。建议在分调试时，利用开发软件的模拟测试功能，对所设计的控制器进行充分的调试，这样可以使后面总调试的成功率大大增加，避免无谓的返工。在组装调试成功的基础上，整理出设计图纸和其他文件。

五、参考方案

(1) 流水硬布线控制器流程图

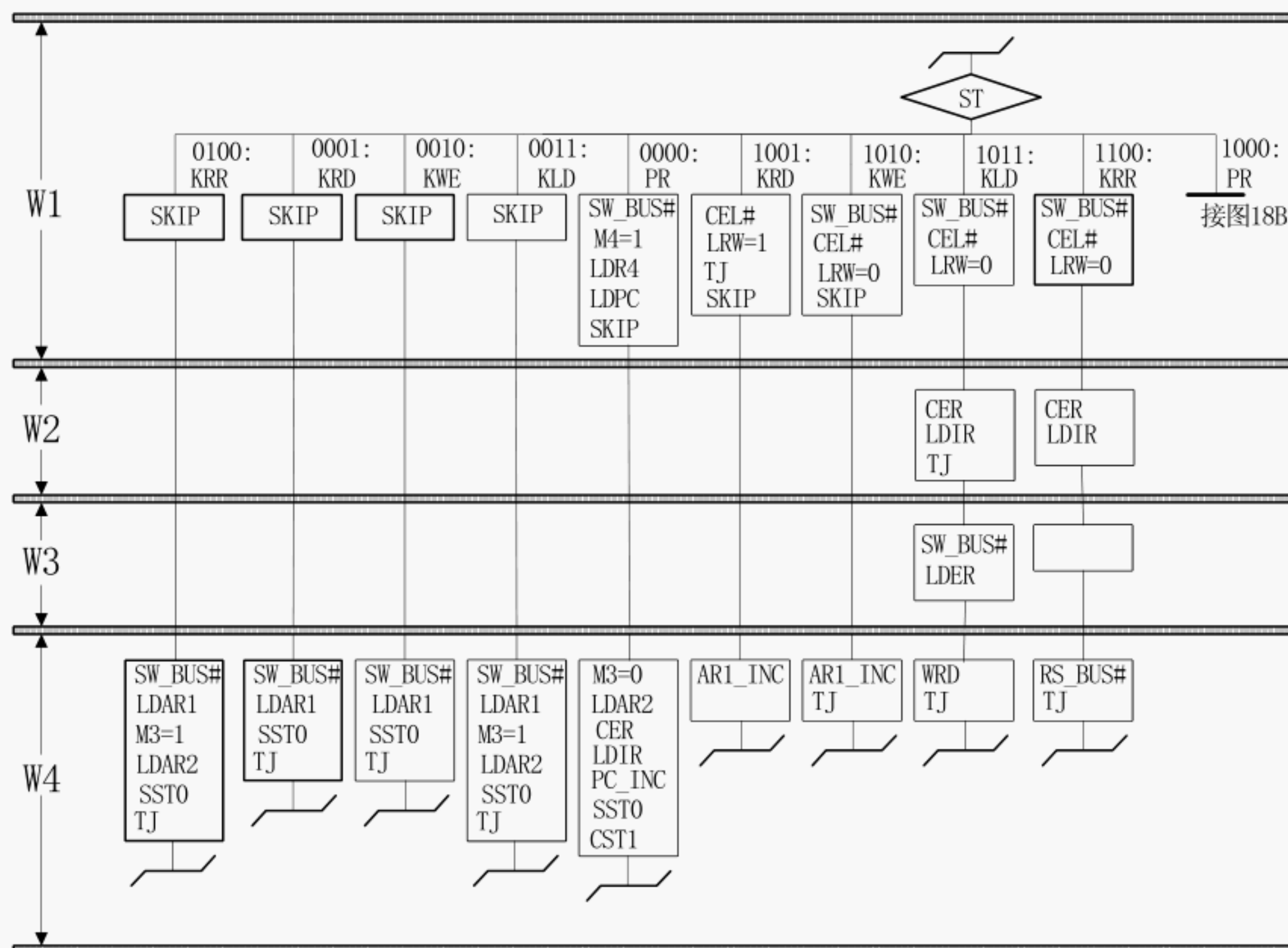


图18A 流水硬布线控制器流程图

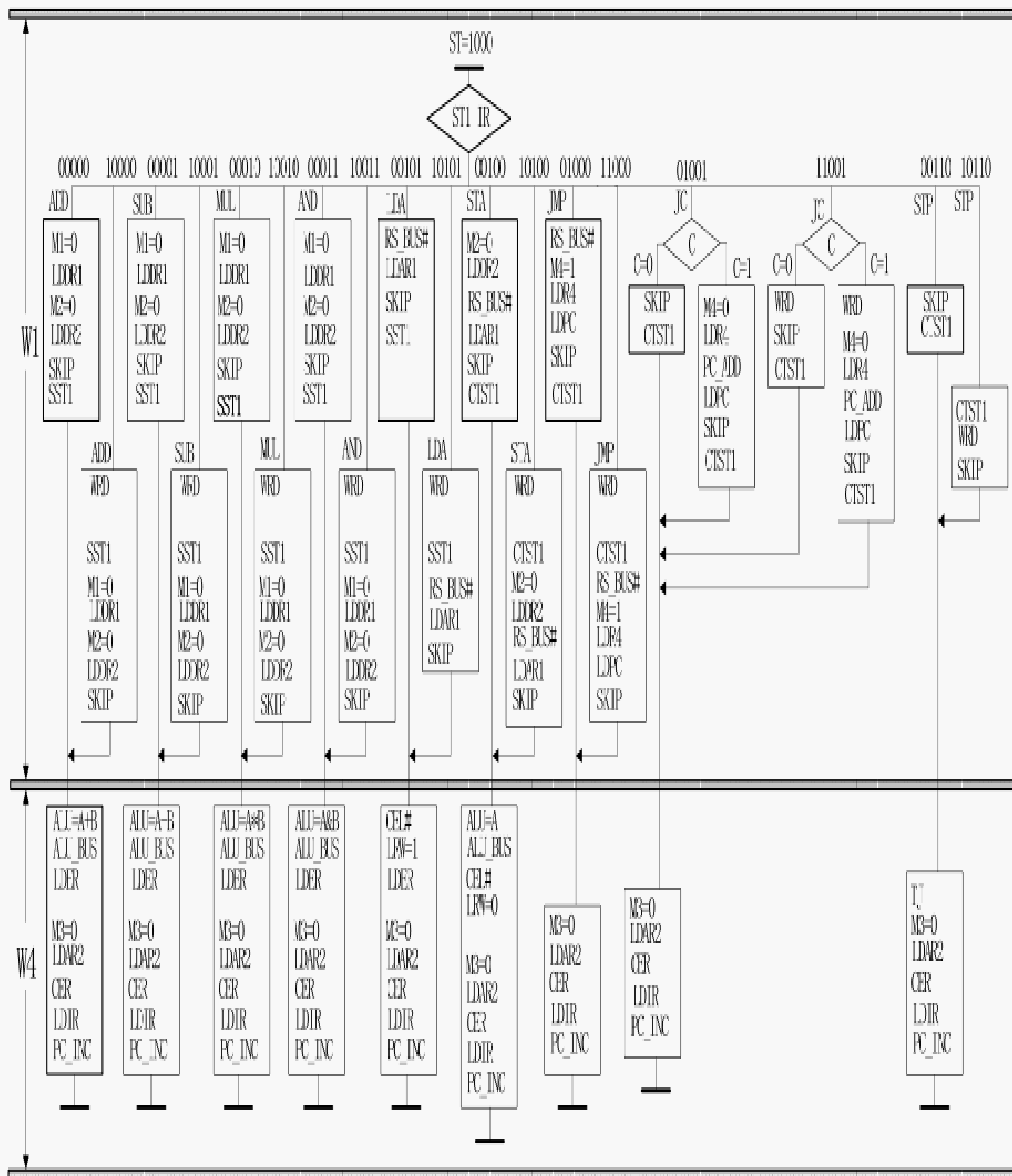


图18B 流水硬布线控制器流程图 (续)

图中, SST0 是控制台标志 ST0 的置 1 信号。ST0 用于区别是否是控制台操作的第一组 W1、W2、W3、W4 信号。按 CLR#按钮后, ST0 的初始状态是 0。当 ST0 = 1 时, 随后的 T1 的上升沿时 ST0 置 1。CST1 和 SST1 是指令状态标志 ST1 的复位和置 1 信号。当按下 CLR#按钮后, ST1 复位到 0。当 SST1=1 时, 下一个 T1 的上升沿将 ST1 置 1; 当 CST1 = 1 时, 下一个 T1 的上升沿将 ST1 清 0。ST1 和指令的操作码 IR7—IR4 共同确定一条指令的分支, 以便执行流水中的相应操作。ST1 = 1 时, 在 W1 节拍, 产生 WRD 信号, ST1 = 0 时, 在 W1 节拍, 不产生 WRD 信号。

(2) ABEL 语言源程序

对于硬布线控制器的 ABEL 语言源程序, 在实现的过程中, 我们对下列信号进行了归并。

LDAR1 和 LDAR2	归并为 LDAR2。
LDDR1 和 LDDR2	归并为 LDDR1。
M1 和 M2	由于在流程图内没有 1 的值出现, 只有 0 的值, 因此 ABEL 语言源程序中不出现这两个信号。
CER、LDIR 和 PC_INC	归并为 LDIR。
LDPC 和 LDR4	归并为 LDPC。

在用 ABEL 语言实现硬布线控制器用 ABEL 语言时, 我们增加了 1 个标志位 RUN。由于按 CLR#按钮复位后, 实验系统的时序停止在 T4、W4, ST0 的值为 0, 这样 SST0 = !ST0 & W4 的值为 1。按 QD 启动按钮后, 由于立即产生 T1 信号, 在 T1 的上升沿使 ST0 置 1, 在第一组 W1、W2、W3、W4 时, ST0 的值为 1, 这是不希望看到的。增加了标志位 RUN 后, 按 CLR#按钮复位, 使 RUN 为 0。由于 SST0 = !ST0 * W4 * RUN, 因此复位后的 SST0 = 0。按 QD 启动按钮, 在 T1 的上升沿, 使 RUN = 1。根据 SST0 的布尔表达式, 在 W1、W2、W3 时, SST0 = 0, 直到 W4 时, 才使 SST0 = 1。由于 ST0 := CLR# * SST0 # CLR * ST0, 在 W4 过后的下一个 T1 的上升沿, 才使 ST0 置 1, 从而将控制台操作的两种状态区分开来。

```
MODULE CTL_2
```

```
TITLE 'PIPE HARD WIRED CONTROL UNIT'
```

```
"INPUT
```

```
CLR PIN 3; "CLR=CLR#
MF, T1, W1, W2, W3, W4 PIN 5..10;
IR4, IR5, IR6, IR7 PIN 12..15;
SWC, SWB, SWA, C PIN 25..28 ;
IRO, IR1 PIN 72, 73; " USE FOR PIPE
```

```
"OUTPUT
```

```
MF1, SST0 NODE ISTYPE 'COM' ;
RUN, ST0 NODE ISTYPE 'REG' ;
SST1, CST1 NODE ISTYPE 'COM' ; "USE FOR PIPE
ST1, TJA NODE ISTYPE 'REG' ; "USE FOR PIPE
WROA, WR1A NODE ISTYPE 'REG' ; "USE FOR PIPE
LDIR, LDPC, PC_ADD, M4 PIN 30..33 ISTYPE 'COM' ;
```

```

LDAR1, AR1_INC, M3, CEL, LRW PIN 35..39 ISTYPE 'COM' ;
LDER, SW_BUS, RS_BUS, WRD PIN 45..48 ISTYPE 'COM' ;
"RS_BUS=RS_BUS#, SW_BUS=SW_BUS#, CEL=CEL#
LDDR1, S2, S1, S0, ALU_BUS PIN 52..56 ISTYPE 'COM' ;
TJ, SKIP PIN 69, 70 ISTYPE 'COM' ;
WRO, WR1 PIN 74, 75 ISTYPE 'REG' ; "USED FOR PIPE
CLK = .C. ;

```

EQUATIONS

```

MF1 = !CLR & MF # T1;
RUN := CLR & !ST0 # CLR & RUN;
RUN.CLK = MF1;
SST0 = !ST0 & RUN & W4;
ST0 := CLR & SST0 # CLR & ST0;
ST0.CLK = MF1;
CST1 = !ST0 & !SWC & !SWB & !SWA & RUN & W4
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & !IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & IR7 & !IR6 & !IR5 & !IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & IR7 & !IR6 & !IR5 & IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & IR5 & !IR4 & W1;
SST1 = ST0 & !SWC & !SWB & !SWA & !IR7 & !IR6 & !IR5 & !IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & !IR7 & !IR6 & !IR5 & IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & !IR7 & !IR6 & IR5 & !IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & !IR7 & !IR6 & IR5 & IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & IR4 & W1;
ST1 := CLR & SST1 # CLR & ST1 & !CST1;
ST1.CLK = MF1;
WROA := IR0;
WR1A := IR1;
WRO := WROA;
WR1 := WR1A;
[WROA, WR1A, WRO, WR1].CLK = T1;

LDIR = !ST0 & !SWC & !SWB & !SWA & W4
      # ST0 & !SWC & SWB & SWA & W2
      # ST0 & SWC & !SWB & !SWA & W2
      # ST0 & !SWC & !SWB & !SWA & W4;
LDPC = !ST0 & !SWC & !SWB & !SWA & W1
      # ST0 & !SWC & !SWB & !SWA & W1 & IR7 & !IR6 & !IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W1 & IR7 & !IR6 & !IR5 & IR4 & C;
PC_ADD = ST0 & !SWC & !SWB & !SWA & W1 & IR7 & !IR6 & !IR5 & IR4 & C;

```

```

M4 = !ST0 & !SWC & !SWB & !SWA & W1
      # ST0 & !SWC & !SWB & !SWA & IR7 & !IR6 & !IR5 & !IR4 & W1;
LDAR1 = !ST0 & !SWC & !SWB & SWA & W4
      # !ST0 & !SWC & SWB & !SWA & W4
      # !ST0 & !SWC & SWB & SWA & W4
      # !ST0 & !SWC & !SWB & !SWA & W4
      # !ST0 & SWC & !SWB & !SWA & W4
      # ST0 & !SWC & !SWB & !SWA & W4
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & !IR5 & !IR4 & W1
      # ST0 & !SWC & !SWB & !SWA & W4;
AR1_INC = ST0 & !SWC & !SWB & SWA & W4
      # ST0 & !SWC & SWB & !SWA & W4;
M3 = !ST0 & !SWC & SWB & SWA & W4
      # !ST0 & SWC & !SWB & !SWA & W4;
LDER = ST0 & !SWC & SWB & SWA & W3
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & IR4;
WRD = ST0 & !SWC & SWB & SWA & W4
      # ST0 & !SWC & !SWB & !SWA & W1 & ST1;
SW_BUS = !(!ST0 & W1 & !SWC & !SWB & !SWA
      # !ST0 & W4 & !SWC & !SWB & SWA
      # !ST0 & W4 & !SWC & SWB & !SWA
      # !ST0 & W4 & !SWC & SWB & SWA
      # !ST0 & W4 & SWC & !SWB & !SWA
      # ST0 & W1 & !SWC & SWB & !SWA
      # ST0 & W1 & !SWC & SWB & SWA
      # ST0 & W1 & SWC & !SWB & !SWA
      # ST0 & W3 & !SWC & SWB & SWA );
RS_BUS = !(ST0 & SWC & !SWB & !SWA & W4
      # ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & IR6 & !IR5 & IR4
      # ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & IR6 & !IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W1 & IR7 & !IR6 & !IR5 & !IR4);
ALU_BUS = ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & !IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & IR4
      # ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & !IR4;
CEL = !(ST0 & !SWC & !SWB & SWA & W1

```



```

# ST0 & !SWC & SWB & !SWA & W1
# ST0 & !SWC & SWB & SWA & W1
# ST0 & SWC & !SWB & !SWA & W1
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & !IR4);
LRW = ST0 & !SWC & !SWB & SWA & W1
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & IR4;
LDDR1 = ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & !IR6 & !IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & !IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & !IR6 & IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & !IR6 & IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W1 & !IR7 & IR6 & !IR5 & !IR4;
S2 = ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & IR5 & !IR4;
S1 = ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & !IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & IR4;
S0 = ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & !IR6 & !IR5 & IR4
# ST0 & !SWC & !SWB & !SWA & W4 & !IR7 & IR6 & !IR5 & !IR4;
TJ = !ST0 & !SWC & !SWB & SWA & W4
# !ST0 & !SWC & SWB & !SWA & W4
# !ST0 & !SWC & SWB & SWA & W4
# !ST0 & SWC & !SWB & !SWA & W4
# ST0 & !SWC & !SWB & SWA & W1
# ST0 & !SWC & SWB & SWA & W2
# ST0 & !SWC & SWB & !SWA & W4
# ST0 & !SWC & SWB & SWA & W4
# ST0 & SWC & !SWB & !SWA & W4
# TJA & W4;
TJA = ST0 & !SWC & !SWB & !SWA & !IR7 & IR6 & IR5 & !IR4;
TJA.CLK = T1;
SKIP = !ST0 & !SWC & !SWB & !SWA & W1
# !ST0 & !SWC & !SWB & SWA & W1
# !ST0 & !SWC & SWB & !SWA & W1
# !ST0 & !SWC & SWB & SWA & W1
# !ST0 & SWC & !SWB & !SWA & W1
# ST0 & !SWC & !SWB & SWA & W1
# ST0 & !SWC & SWB & !SWA & W1
# ST0 & !SWC & !SWB & !SWA & W1;

```

END

由上述源程序可以看出，它和常规硬布线控制器的 ABEL 源程序十分类似，主要是增加了流水处理所需的标志 ST1 和对 IR0、IR2 的两级缓存，其他地方基本相同，有的信号的逻辑表达式还更简洁一些。使用在系统可编成逻辑器件，比较容易实现计算机性能的升级。

(3) 接线

5. 接固定连线和数据通路内部连线

M2 接 M1, M1 接 GND。CER 接 PC_INC, PC_INC 接 LDIR。LDAR2 接 LDAR1。LDDR1 接 LDDR2。IRO 接 RD0, IR1 接 RD1, IR2 接 RS0, IR3 接 RS1。IAR_BUS 接 VCC。共 11 条线。

6. 控制台和 isp1032 的连线

CLR#接 PIN3 (CLR), SWC 接 PIN25 (SWC), SWB 接 PIN26 (SWB), SWA 接 PIN27 (SWA)。共 4 条线。

7. 时序电路和 isp1032 的连线

MF 接 PIN5 (MF), T1 接 PIN6 (T1), TJI 接 PIN69 (TJ), SKIP 接 PIN70 (SKIP)。W1 接 PIN7 (W1), W2 接 PIN8 (W2), W3 接 PIN9 (W3), W4 接 PIN10。共 8 条线。

8. 数据通路和 isp1032 的连线

IR4 接 PIN12 (IR4), IR5 接 PIN13 (IR5), IR6 接 PIN14 (IR6), IR7 接 PIN15 (IR7)。C 接 PIN28 (C)。LDIR 接 PIN30 (LDIR)。LDPC 接 PIN31 (LDPC), PC_ADD 接 PIN32 (PC_ADD), M4 接 PIN33 (M4), M3 接 PIN37 (M3)。LDAR1 接 PIN35 (LDAR1), AR1_INC 接 PIN36 (AR1_INC), CEL#接 PIN38 (CEL), LRW 接 PIN39 (LRW)。LDER 接 PIN45 (LDER), SW_BUS#接 PIN46 (SW_BUS), RS_BUS#接 PIN47 (RS_BUS), WRD 接 PIN48 (WRD)。LDDR1 接 PIN52 (LDDR1), S2 接 PIN53 (S2), S1 接 PIN54 (S1), S0 接 PIN55 (S0), ALU_BUS 接 PIN56 (ALU_BUS)。IRO 接 PIN72, IR1 接 PIN73, WR0 接 PIN74 (WR0), WR1 接 PIN75 (WR1)。共 27 条线。

附录:

